Luiz Gustavo Lino

Implementação de um motor de jogos para microcontroladores

Sao Paulo - Brasil

1 de dezembro de 2020

Luiz Gustavo Lino

Implementação de um motor de jogos para microcontroladores

Monografia apresentada na disciplina Trabalho de Conclusão de Curso, como parte dos requisitos para obtenção do título de Bacharel em Ciência da Computação.

Centro Universitário Senac - Santo Amaro

Bacharelado em Ciência da Computação

Orientador: Fabio Henrique Abenza Freire

Coorientador: Thiago Ribeiro Claro

Sao Paulo - Brasil

1 de dezembro de 2020

Gostaria de dedicar este trabalho a quem eu mais dei ouvidos durantes estes três anos de pesquisa. Muito obrigado a rádio "lofi hip hop radio - beats to relax/study to", do YouTube, por estar lá por mim sempre, 24 horas por dia.

RESUMO

Este trabalho aproxima o movimento Maker da produção de jogos de vídeo games, propondo uma ferramenta de motor do jogos, específica para microcontroladores. Considerando-se as limitações de memória e processamento destes dispositivos — que torna o desenvolvimento de software complexo — verificou-se a necessidade da construção de um Ambiente de Desenvolvimento Integrado para auxiliar na produção de jogos digitais. Para tanto, motores de jogos comerciais foram estudados, bem como as partes que os compõe, e um protótipo de console de video games foi construído utilizando peças facilmente encontradas, como meio de validar a capacidade multiplataforma da ferramenta que foi proposta. Durante o desenvolvimento, utilizou-se uma arquitetura modular e extensível, aproveitando-se os benefícios do uso de contêineres Linux como ambiente de compilação. Por fim, validou-se a ferramenta produzida por meio da construção e mensuração de dois jogos conceituais. Espera-se que, publicando-se este texto, desenvolvedores de jogos sejam atraídos para esta plataforma em ascensão, dos microcontroladores, e que se desperte o interesse em eletrônica, programação e jogos digitais nas novas gerações de estudantes dos cursos de Ciência da Computação e relacionados.

Palavras-chave: video games; motores de jogos; microcontroladores.

ABSTRACT

This work brings together the areas of video game production and the Maker Movement, proposing a game engine, specifically for microcontrollers. Considering the memory and processing limitations of these devices - which makes software development more complex - there was a need to build an Integrated Development Environment to assist in the production of digital games. For this purpose, commercial game engines were studied, as well as the parts that compose them, and a video game console prototype was built using easily found parts, as a means of validating the multiplatform capability of the proposed tool. During development, a modular and extensible architecture was used, taking advantage of the benefits of using Linux containers as a compilation environment. Finally, the tool produced was validated by constructing and measuring two conceptual games. It is expected that, with this publication, game developers will be attracted to this rising platform, of microcontrollers, and that interest in electronics, programming, and digital games will be aroused in the new generations of students in Computer Science and related courses.

Keywords: video games; game engines; microcontrollers.

LISTA DE ILUSTRAÇÕES

Figura 1 – O modelo de sistema-em-um-chip Arduino LilyPad 328 16
Figura 2 – MAKERbuino, um micro console com controlador ATmega328 de 16Mhz 17
Figura 3 – Creations, uma rede social para compartilhamento de jogos criados por usuários do console Gamebuino 17
Figura 4 – Da esquerda para direita: Monument Valley, lançado em 2014 pela ustwo games, utilizando o motor Unity; Fortnite, lançado em 2017 pela Epic Games, utilizando o motor Unreal; Big Fish Casino, lançado em 2012 pela BigFish, utilizando o motor Cocos2D. 21
Figura 5 – O jogo PongMeta, por Aurélien Rodot. 22
Figura 6 – Diagrama de componentes de um motor de jogos, no contexto do desenvolvimento. 23
Figura 7 – Diagrama de módulos de um motor de jogos, no contexto da execução de um jogo. 24
Figura 8 – Máquina de estados finita ilustrando os possíveis estados em que um botão de um controle de console de video games pode estar em um dado quadro da simulação. 25
Figura 9 – Comparação das capacidades de memória dentre video games baseados em microcontroladores. 26
Figura 10 – Possível divisão de cenas para o jogo Super Mario Bros., lançado em 1985 para o console Nintendo NES. 27
Figura 11 – Exemplo de geração de imagem por indexação de cores 30
Figura 12 – Algoritmos para compressão de imagens 31
Figura 13 – Exemplo da redução quando aplicada a codificação run-length 32
Figura 14 – Exemplo da codificação de Huffman 32
Figura 15 – Geração procedural de imagens em um jogo, com exemplo de pseudo-código. 33
Figura 16 – Geração procedural de imagens aplicada a jogos. 34
Figura 17 – Formatação digital de áudio. 35
Figura 18 – Primeira placa Arduino, um protótipo. 37
Figura 19 – Editor de código online Arduino Web IDE. 38
Figura 20 – O modelo micro:bit, da BBC. 39
Figura 21 – Linha de processadores ARM, com destaque para geração atual, chamada Cortex. 40
Figura 22 – Especificações do console portátil ODROID GO. 42
Figura 23 – Diagrama de blocos do hardware ODROID GO. 43

Figura 24 – Especificações do protótipo de videogame LVNDR-01, desenvolvido durante este trabalho. 44
Figura 25 – Diagrama de classes do motor de jogos Gear 2D. 47
Figura 26 – Protótipo de quebra-cabeça tangível 48
Figura 27 – Arquitetura para compilação multiplataforma. 52
Figura 28 – Diagrama UML de Casos de Uso para o desenvolvedor de controlador . 55
Figura 29 – Exemplo do funcionamento de uma função externa definida no motor e implementada no controlador. 57
Figura 30 – Fluxograma do critério de decisão das hipóteses na escolha do formato de representação das cores 59
Figura 31 – Opções de alinhamento de memória na matriz framebuffer. 60
Figura 32 – Exemplificação de transações digitais eletrônicas paralelas e seriais. . . 62
Figura 33 – Circuito dos botões no video game ODROID-GO. 66
Figura 34 – Exemplo do divisor de tensão dos botões direcionais do ODROID-GO. 66
Figura 35 – Circuito dos botões no protótipo LVNDR-01. 67
Figura 36 – Diagrama de classe apresentando as interfaces públicas do motor de jogos para interface do usuário. 69
Figura 37 – Diagrama de classe apresentando as interfaces públicas do motor de jogos para entradas de controles. 70
Figura 38 – O Ambiente Integrado de Desenvolvimento “lvndr” 71
Figura 39 – Delimitação do contêiner e das tecnologias utilizadas nas bibliotecas do motor de jogos. 74
Figura 40 – Diagrama UML de Casos de Uso para o desenvolvedor de codificadores. 75
Figura 41 – Mecanismo de troca de cenas e seu ciclo de vida. 76
Figura 42 – Um exemplo de arquivo no formato lv-code. 77
Figura 43 – Comparação entre as versões com e sem uso de bitplanes para codificação run-length em uma imagem (menos símbolos = melhor). 79
Figura 44 – Exemplo da leitura do tamanho de repetição de uma codificação run-length utilizando a estratégia de representação de tamanho variável. . . 80
Figura 45 – Algoritmo de compressão de imagem utilizando Bitplanes. 81
Figura 46 – Trecho da configuração do projeto que especifica os codificadores e editores. 82
Figura 47 – Captura de tela do editor de código sendo executado dentro do ambiente integrado de desenvolvimento. 83
Figura 48 – Captura de tela do editor de imagens sendo executado dentro do ambiente integrado de desenvolvimento. 84
Figura 49 – Diagrama UML de Casos de Uso para o desenvolvedor de jogos. 85
Figura 50 – “Pac-man”, para o console NES, da Nintendo. 86

Figura 51 – Interface projetada para prova de conceito 1. Utiliza gráficos de domínio público, sob licença CC-0, criados por “GrafxKid”, “ansimuz” e “surt”, do portal online “Open Game Art”. 87
Figura 52 – Medição de quadros por segundo nos jogos conceituais. 88
Figura 53 – Gráfico da memória RAM livre durante a troca de cenas. 89
Figura 54 – “Double Dragon”, para o console NES, da Nintendo. 90
Figura 55 – Interface projetada para prova de conceito 2. Utiliza gráficos de domínio público, sob licença CC-0, criados por “GrafxKid”, “ansimuz” e “surt”, do portal online “Open Game Art”. 91
Figura 56 – Processo de construção de cenários utilizando peças. Utiliza gráficos de domínio público, sob licença CC-0, criados por “surt”, do portal online “Open Game Art”. 92
Figura 57 – O jogo conceitual 2 sendo executado no protótipo LVNDR-01. 93
Figura 58 – Programa de 1986, da TV japonesa NHK, sobre o desenvolvimento de jogos, mostrando a digitalização de imagens para vídeo games por meio de uma ferramenta, a Sega Digitizer System. 95

SUMÁRIO

1 INTRODUÇÃO . . . 15
1.1 Contexto . . . 15
1.2 Objetivo . . . 18

2 REVISÃO BIBLIOGRÁFICA . . . 19
2.1 Video games como produto . . . 19
2.2 Motores de jogos . . . 19
2.2.1 Motores de jogos em microcontroladores . . . 22
2.2.2 Partes de um motor de jogos . . . 23
2.2.3 Dispositivos de interface humana (HID) . . . 24
2.2.4 Gestão de conteúdo . . . 25
2.2.5 Arquitetura de cenas . . . 26
2.2.6 O conceito de Gameloop . . . 27
2.2.7 Editores de jogos . . . 28
2.3 Formatos digitais de multimídia . . . 28
2.3.1 Formatação digital de imagens . . . 29
2.3.2 Codificação run-length e codificação de Huffman . . . 31
2.3.3 Conteúdo procedural . . . 33
2.3.4 Formatação digital de áudio . . . 34
2.4 Microcontroladores e consoles micro controlados . . . 36
2.4.1 Arduino . . . 37
2.4.2 Arquitetura ARM Cortex . . . 39
2.4.3 Arquitetura XTensa . . . 41
2.4.4 ODROID GO . . . 41
2.4.5 Protótipo LVNDR-01 . . . 43
2.5 Trabalhos relacionados . . . 46
2.5.1 Biblioteca para Desenvolvimento de Jogos para o Nintendo DS . . . 46
2.5.2 Construindo um motor de jogos: um conto de engenharia moderna baseada em modelos . . . 46
2.5.3 Gear2D: um motor extensível de jogos baseado em componentes . . . 47
2.5.4 Um protótipo de quebra-cabeça tangível para crianças com transtorno de déficit de atenção com hiperatividade . . . 48

3 DESENVOLVIMENTO . . . 51
3.1 Arquitetura do motor de jogos . . . 51

3.2 Controladores . . . 53
3.2.1 Infraestrutura de compilação . . . 54
3.2.2 Funções externas do motor de jogos . . . 56
3.2.3 Interface para desenho da tela . . . 56
3.2.4 Escolha do formato de cores no framebuffer . . . 58
3.2.5 Implementação da interface com a tela no ODROID . . . 61
3.2.6 Implementação da interface com a tela no Protótipo LVNDR-01 . . . 64
3.2.7 Interface para os botões físicos . . . 65
3.2.8 Implementação da interface com os botões no ODROID . . . 65
3.2.9 Implementação da interface com os botões no Protótipo LVNDR-01 . . . 67
3.3 Bibliotecas do motor de jogos . . . 68
3.3.1 Interfaces públicas da aplicação . . . 68
3.3.2 Ambiente integrado de desenvolvimento . . . 70
3.3.3 Interface via linha de comando . . . 72
3.4 Codificadores . . . 74
3.4.1 Codificador de código . . . 75
3.4.2 Codificador de imagem . . . 77
3.5 Editores . . . 80
3.5.1 Estratégia de associação por extensão . . . 80
3.5.2 Utilização de Webviews . . . 82
3.5.3 Editor de código . . . 83
3.5.4 Editor de imagem . . . 84
3.6 Jogos conceituais . . . 85
3.6.1 Conceito 1 - Jogo Arcade . . . 86
3.6.2 Conceito 2 - Jogo de ação . . . 90

4 CONCLUSÃO . . . 95

5 TRABALHOS FUTUROS . . . 97
5.0.1 Integrações com o ecossistema do Arduino . . . 97
5.0.2 Codificação de áudio para microcontroladores . . . 97
5.0.3 Adição de consoles baseados em Linux ao ecossistema . . . 98

REFERÊNCIAS . . . 99

1 INTRODUÇÃO

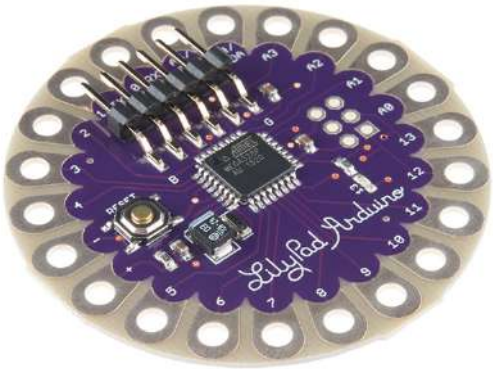
Neste trabalho examinamos a intersecção entre a produção de jogos de vídeo game e o movimento Maker, que utiliza microcontroladores para criar todo tipo de gadgets. Exploramos o funcionamento dos motores de jogos, citando exemplos modernos, sua relação na indústria e como afetam a produtividade dos times de desenvolvimento. Aprofundando no tema, são estudados arquivos multimídias – peças fundamentais na construção de jogos – bem como opções de algoritmos de compactação. O potencial dos consoles micro controlados é investigado, citando as arquiteturas de processador ARM e Xtensa, aparelhos já existentes e demonstrando novas configurações. No desenvolvimento, é proposto um motor de jogos multiplataforma para microcontroladores. Sua arquitetura modular é detalhada. Dois aparelhos são escolhidos para validar a ferramenta, um deles construído durante a pesquisa. Por fim, dois jogos são implementados na ferramenta e validações são feitas.

1.1 Contexto

Lançamentos de novos modelos de smartphones ano após ano têm elevado o nível da computação portátil a ponto de estarmos próximos de ter em nossos bolsos aparelhos tão potentes quanto os computadores pessoais, de mesa. Kaczmarek, Lees e Bennett (2018) relatam que o processador A11 bionic de um Apple iPhone X, lançado em 2017, contém mais de 4 bilhões de transistores e Baig e LeVitus (2019) dizem ainda que seu sucessor, o A12, é capaz de realizar 5 trilhões de operações por segundo. Com isso, o desenvolvimento de aplicações se torna mais ágil e facilitado pela quantidade de recursos de hardware e velocidade de processamento disponíveis. Estes avanços permitem também que os smartphones façam uso de linguagens de programação de alto nível, como `C#`; embarquem máquinas virtuais, como as das linguagens Lua e Python, para código compatível entre diferentes sistemas operacionais; suportem softwares editores de jogos de ponta, que antes só eram utilizados para produção de títulos lançados para os consoles de video games de última geração, como, por exemplo, a ferramenta Unreal Engine 4, da Epic Games.

Por outro lado, uma área em ascensão tem encontrado novamente as dificuldades de lidar com processadores e memória limitada, são os aparelhos movidos por microcontroladores. De dispositivos inteligentes de ar condicionado a vasos de plantas conectados a internet, estes pequenos computadores podem conter sensores, botões, telas e conectar-se a redes sem fio para se atualizar, buscar e enviar informações. Tudo isso graças a microcontroladores de baixo custo, pouco consumo de bateria, baseados em tecnologias de

Figura 1 – O modelo de sistema-em-um-chip Arduino LilyPad 328



Fonte: Arduino AG, da Itália

sistema-em-um-chip e empoderados por processadores ARM Cortex, Microchip AVR e Tensilica Xtensa, por exemplo. O modelo Lily Pad 328, encontrado por 15 dólares em 2019, é mostrado na figura 1. Com processador de apenas 8MHz e 1 Kilobyte de memória SRAM, este diminuto hardware programável tem apenas 0.8 milímetros de espessura e 50 milímetros de diâmetro (PEREA, 2015).

A disponibilidade de novos componentes eletrônicos de baixo custo, distribuição digital de aplicativos e sites de financiamento coletivo, fez surgir nesta intersecção uma nova categoria de aparelhos de video game, os micro consoles, como elucida Grubb (2013). São, basicamente, iniciativas de plataformas de jogos abertas a desenvolvedores e paralelas as oferecidas pelas gigantes da área, Sony, Nintendo e Microsoft. Entre elas, consoles portáteis de video game por preços acessíveis com telas de LCD (coloridas ou não), suporte a cartões de memória, processadores com velocidades em torno de uma centena de MHz, memória volátil de poucos Kilobytes e baterias de lítio.

É o caso do MAKERbuino, ilustrado na figura 2, lançado em 2017 por meio do site de financiamento coletivo Kickstarter e vendido como um kit de aprendizado de eletrônica (COMPUTER ACTIVE, 2017; 24 SATA, 2018). Seus criadores, da empresa croata CircuitMess, afirmam no site do produto que "milhares de crianças montaram o MAKERbuino e aprenderam sobre eletrônica, solda, software, hardware e muito mais"[1].

Todo console de video games precisa de jogos. Por esse motivo os projetos de hardware sempre são acompanhados de bibliotecas e softwares entregues aos desenvolvedores interessados, sejam nos consoles das grandes marcas ou nos independentes. Atualmente,

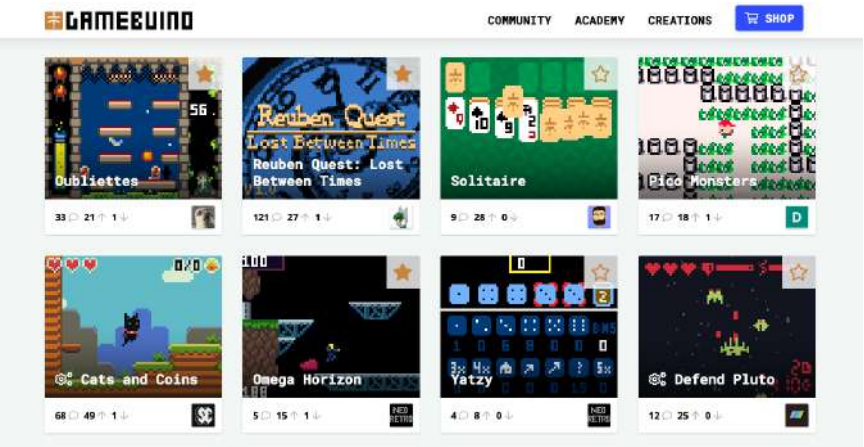
[1] "Thousands of kids have assembled the MAKERbuino and learned about electronics, soldering, software, hardware and so much more" com tradução do autor.

Figura 2 – MAKERbuino, um micro console com controlador ATmega328 de 16Mhz

Fonte: CircuitMess, da Croácia

redes sociais criadas pelos próprios fabricantes de micro consoles são um ponto de conexão entre os criadores. Nestes fóruns são discutidos projetos, algoritmos e soluções para problemas. Isso motiva os próprios usuários com algum conhecimento em programação a criarem seus jogos e compartilha-los com o restante da comunidade (figura 3).

Figura 3 – Creations, uma rede social para compartilhamento de jogos criados por usuários do console Gamebuino



Fonte: Gamebuino, da França

1.2 Objetivo

Para Gregory (2018) os jogos digitais podem ser vistos como “simulações de computador baseadas em agentes interativos em tempo quase real”[2]. Projetá-los historicamente tem sido uma tarefa multidisciplinar e complexa que envolve muita matemática e otimizações. Por esse motivo motores de jogos são construídos, para facilitar tarefas corriqueiras de um projeto de jogo digital. A empolgação da comunidade de desenvolvimento em relação a novos dispositivos precisa ser acompanhada da criação de ferramentas para dar suporte a produção de conteúdo para estas plataformas em ascensão.

Este trabalho tem como objetivo implementar um software motor de jogos para microcontroladores. Para tanto, serão estudados dois hardwares de video game: ODROID-GO, um portátil projetado pela empresa Coreana HardKernel Co. Ltd, baseado em um chip ESP32; LVNDR-01, um protótipo construído durante esta pesquisa como meio de validar o potencial multiplataforma do motor de jogos, baseado em um microcontrolador ARM Cortex M3.

Serão discutidas as partes de um motor de jogos, além de expostos exemplos de motores comerciais disponíveis atualmente. Em seguida, avaliadas as técnicas de processamento, formatos de representação e compressão de dados para gráficos, no contexto de um motor de jogos. A arquitetura de microcontroladores será abordada e contextualizada historicamente, bem como detalhadas as especificações dos consoles escolhidos. Quatro trabalhos relacionados são descritos, de modo a apresentar os resultados de estudos acadêmicos sobre o tema. Por fim, é proposto um software motor de jogos intitulado "lvnder", preparado para video games baseados nas arquiteturas estudadas, juntamente com dois jogos conceituais que têm como objetivo validar e mensurar a viabilidade da ferramenta.

[2] “soft real-time interactive agent-based computer simulations”, com tradução do autor

2 REVISÃO BIBLIOGRÁFICA

2.1 Video games como produto

Jogos eletrônicos, conhecidos também pelo termo em inglês *video games*, são criados desde os anos 70, inicialmente por equipes pequenas e até individualmente. Loguidice e Barton (2012) relatam que Toshihiro Nishikado, da Taito Corporation do japão, lançou em 1978 o jogo Space Invaders e Alexey Pajitnov programou sozinho em 1985 o jogo Tetris, usando um mainframe soviético Elektronika 60 com um processador de 16bits a 0.25MHz.

Com o passar do tempo, grandes empresas de jogos foram fundadas e os times de desenvolvimento cresceram, tornando-se multidisciplinares (GREGORY, 2018). Segundo Boltjes (2018), *Horizon Zero Dawn* da *Guerrilla Games*, vencedor do prêmio de jogo do ano no *4º Game Awards*, chegou a alocar mais de 150 pessoas trabalhando ao mesmo tempo, no mesmo software, em uma das etapas do desenvolvimento. Matéria do site *Push Square*, citando Kist (2017 apud BARKER, 2017), alega que apenas neste projeto foram investidos 45 milhões de euros ao longo de 6 anos.

Neste contexto de expansão, Weiller (2015) comenta sobre a crescente notoriedade e importância social dos jogos eletrônicos, afirmando que 67% dos americanos jogam video games de maneira recorrente e Gregory (2018) diz que "hoje, os jogos são uma indústria multi-bilionária que rivaliza com Hollywood em tamanho e popularidade"[1].

Os motores de jogos são protogonistas nessa cena. Em texto na revista americana CGW, Meloni (2015) dedica-se a comentar o assunto, relatando que o benefício rotineiro ao se utilizar uma ferramenta destas durante o desenvolvimento de um jogo é reduzir custos de produção, permitindo que os criadores consigam focar nos personagens, histórias, cenários, visual e gameplay.

2.2 Motores de jogos

Para Zarrad (2018), motores de jogos, tradução do termo popular em inglês *game engine* ou apenas *engine*, são ferramentas que colaboram com a redução dos custos, tempo de desenvolvimento e necessidade de força de trabalho. Ainda, em suas palavras, "um motor é uma parte essencial de um jogo; ele influencia a estrutura e a organização dos

[1] "Today, games are a multi-billion-dollar mainstream industry rivaling Hollywood in size and popularity", com tradução do autor.

gráficos do jogo, arquivos de configuração e todas as outras entradas, como entradas do usuário, mapas e sons[2]".

Oferecem um kit de desenvolvimento de software (em inglês e abreviado, SDK) e aplicações com interface gráfica que facilitam a gestão de atividades complexas. Costumam ter características especificas de um estilo. Existe *engine* dedicada, por exemplo, apenas a construção de jogos do estilo RPG [3].

Do ponto de vista estrutural, Gregory (2018) sugere que uma arquitetura *data-driven* é o que separa um motor de jogos de um software que é um jogo, porém não um motor. Isso porque codificar lógicas ou gerar casos especiais de tratamento para objetos específicos dificulta a reutilização de parte do código-fonte para a criação de um novo jogo. Kelly (2012) complementa nos dizendo que uma game engine é uma coleção de códigos-fonte que compõem os elementos fundamentais comuns à maioria dos projetos.

Pode-se também utilizá-los para outros fins, que não exatamente a criação de jogos. A *Nasa* mantém um laboratório denominado *Hybrid Reality Lab* que emprega motores comerciais para conceber ambientes de simulação de missões espaciais e com isso treinar seus astronautas, como relatado por Moltenbrey (2017).

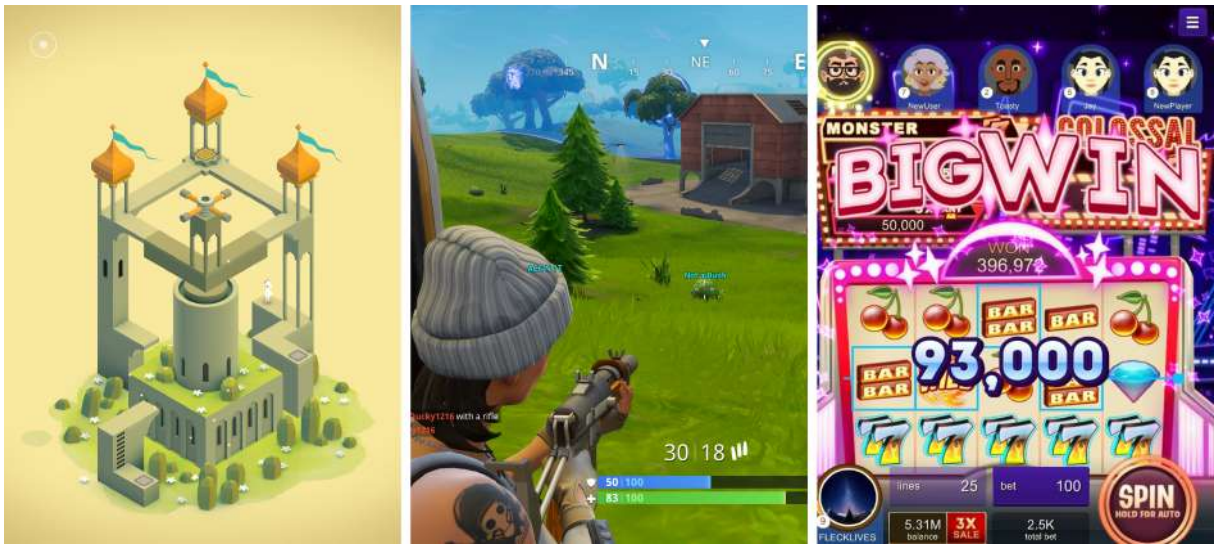
Analisando o panorama da geração atual de motores destacamos três deles, de modo a ilustrar os diversos objetivos que um desenvolvimento destes persegue.

Unity, da *Unity Technologies*: fundada em 2004, na Dinamarca, ganhou espaço inicialmente no mercado de smartphones, quando os jogos mobile estavam começando a se destacar. Em 2007, com o lançamento do primeiro *iPhone*, Meloni (2015) afirma que os esforços da empresa foram direcionados para suportar este aparelho. A estratégia foi um sucesso e assim que o desenvolvimento de jogos para smartphones tornou-se relevante a *Unity* estava na vanguarda, tomando para si a missão de "democratizar o desenvolvimento de jogos". Em 2015, haviam 3.3 milhões de desenvolvedores registrados. Ainda para Meloni (2015), a facilidade de se desenvolver um jogo que será suportado em múltiplas plataformas, como smartphones, consoles e computadores de mesa, é o que atrai novos desenvolvedores para ferramenta.

Unreal, da *Epic Games*: destaca-se em relação a qualidade dos gráficos. Meloni (2015) diz que desde sua fundação, em meados dos anos 90, sua criadora, a *Epic*, preocupa-se em estar à frente neste quesito. Atualmente é parte da *Tencent*, que em 2012 adquiriu 48% da *Epic* num acordo de 300 milhões de dólares. Andy Hess, evangelista das tecnologias

[2] "an engine is an essential part of a game; it influences the structure and the organization of game graphics, configuration files, and all other inputs such as user inputs, maps, and sounds", com tradução do autor.
[3] é o caso do software RPG Maker MV, da Enterbrain.

Figura 4 – Da esquerda para direita: Monument Valley, lançado em 2014 pela ustwo games, utilizando o motor Unity; Fortnite, lançado em 2017 pela Epic Games, utilizando o motor Unreal; Big Fish Casino, lançado em 2012 pela BigFish, utilizando o motor Cocos2D.



Fonte: Apple App Store.

relacionadas a *Unreal*, apresenta a Unreal como um motor de jogos moderno, que permite aos desenvolvedores criar gráficos com qualidade de Hollywood, devido ao grande esforço que a empresa faz em relação a melhorar constantemente seu renderizador gráfico[4].

Cocos2D, de código livre sob a licensa MIT: inicialmente desenvolvido por Ricardo Quesada, na Argentina, escrito na linguagem Python, depois portado para Objective-C e finalmente C++, este motor gráfico open-source é atualmente gerenciado pela Chuckong Technologies de Beijing. Meloni (2015) cita grandes títulos utilizando o motor *Cocos2D*, como Big Fish Casino, da Big Fish Games.

Contudo, apesar de sua popularidade e adesão da comunidade, os motores de jogos acima descritos não suportam micro consoles baseados em microcontroladores. Um smartphone, computador pessoal ou console de última geração, que são as plataformas mais utilizadas pelos jogadores (GDC, 2018), tem um potencial gráfico para ser explorado de níveis de magnitude maior. Acrescenta-se o fato de que os jogos que existem para microcontroladores são feitos por hobistas, gratuitos e sem viés comercial, o que gera hesitação nos novos jogadores – pois, quando se investe na compra um vídeo game, é comum pesquisar os jogos disponíveis para ele.

[4] entrevista disponível online, por Meloni (2015)

2.2.1 Motores de jogos em microcontroladores

Assim como aconteceu nos anos 70, dos Ataris e Odysseys, os jogos atuais para microcontroladores são trechos pequenos de código, muitas vezes totalizam algumas centenas de linhas em um único arquivo. O jogo *PongMeta* (Figura 5), por exemplo, para o *Gamebuino Meta* tem um único arquivo de 270 linhas de código na linguagem C (RODOT, 2017).

Figura 5 – O jogo PongMeta, por Aurélien Rodot.

Fonte: site oficial do Gamebuino, 2019.

Historicamente, nos anos 80 do Nintendo NES e SEGA Master System, gráficos coloridos aprimorados e inimigos baseados em inteligência artificial primitiva marcaram presença, com eles os primeiros softwares facilitando o desenvolvimento de series de jogos de mesma temática. Entre eles estava o Gold Box, desenvolvido pela californiana SSI e lançado em 1988. Matéria da revista americana Antic, na quarta edição de 1985, mostra em entrevista com o então presidente e fundador da SSI, Joel Billings, a evolução das ideias daquela geração abrindo caminho para o aparecimento dos motores de jogos:

> Mantendo o núcleo do sistema e alterando as armas e o mapa, um novo jogo é criado. Gary Grigsby é seu autor mais produtivo, em partes porque ele dominou seu ferramental e o conceito de jogos clonados. "uma nova matemática, um novo banco de dados e um jogo novo é criado"[5] (POWELL, 1985)

De maneira similar, os micro console seguem hoje a mesma trajetória. Ferramentas para facilitar a geração de novos jogos e reaproveitar trechos comuns de código começaram

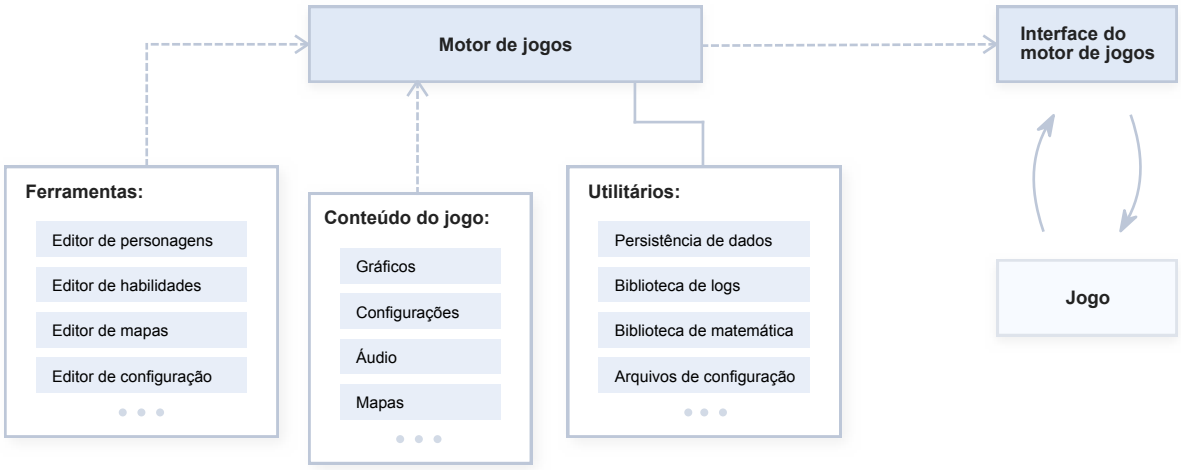
[5] "By keeping the core system and changing the weapons and the map, a new game is created. Gary Grigsby is their most prolific author partly because he's mastered their utility tools and the concept of clone games. "New math, new database, and you've got a whole new game"", com tradução do autor.'

a surgir[6]. Este trabalho pretende aproveitar esse momento e valer-se do conhecimento adquirido pelas gerações anteriores de consoles de video games para contribuir com uma ferramenta que facilite a criação de novos jogos, um motor de jogos para microcontroladores.

2.2.2 Partes de um motor de jogos

Segundo, Gregory (2018), existem duas maneiras de se ver um motor de jogos: em um primeiro momento, onde se está produzindo o jogo, o motor oferece ferramentas de produtividade e formatação de conteúdo; num segundo momento, em que o jogo está sendo jogado, o ambiente de execução do motor vai consumir arquivos em formatos específicos, compor a cena, receber comandos do jogador e seguir um roteiro pré-definido. Zarrad (2018) comenta sobre o assunto, nos diz que as ferramentas oferecidas pela engine fazem uma ponte entre o conteúdo do jogo e os componentes que utilizam estes conteúdos em tempo de execução, como ilustra a Figura 6.

Figura 6 – Diagrama de componentes de um motor de jogos, no contexto do desenvolvimento.



Fonte: Zarrad (2018), com ajustes do autor.

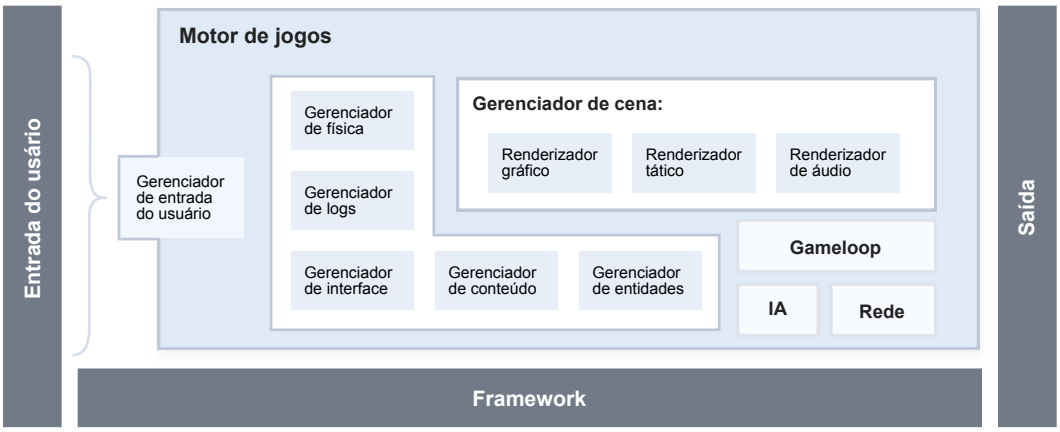
Já sobre as atividades do motor em tempo de execução, ilustrado na Figura 7, existem módulos que podem ou não ser necessários, dependendo do jogo que se quer criar. Nem todos os jogos vão exigir suporte a scripts de inteligência artificial ou simulação de física, por exemplo. Quanto ao módulos mais utilizados, Zarrad (2018) comenta que:

> (...) o gerenciador de entradas e o sistema de interface do usuário são importantes, mesmo para um jogo simples, para processar a entrada do jogador. O gerenciador

[6] O software online Pokitto MicroPython IDE, por exemplo.

de cena é necessário para organizar e controlar o conteúdo da cena e monitorar a lógica do jogo. O gameloop do jogo é usado para controlar um laço infinito que mantém o jogo rodando repetidamente.

Figura 7 – Diagrama de módulos de um motor de jogos, no contexto da execução de um jogo.



Fonte: Zarrad (2018), com ajustes do autor.

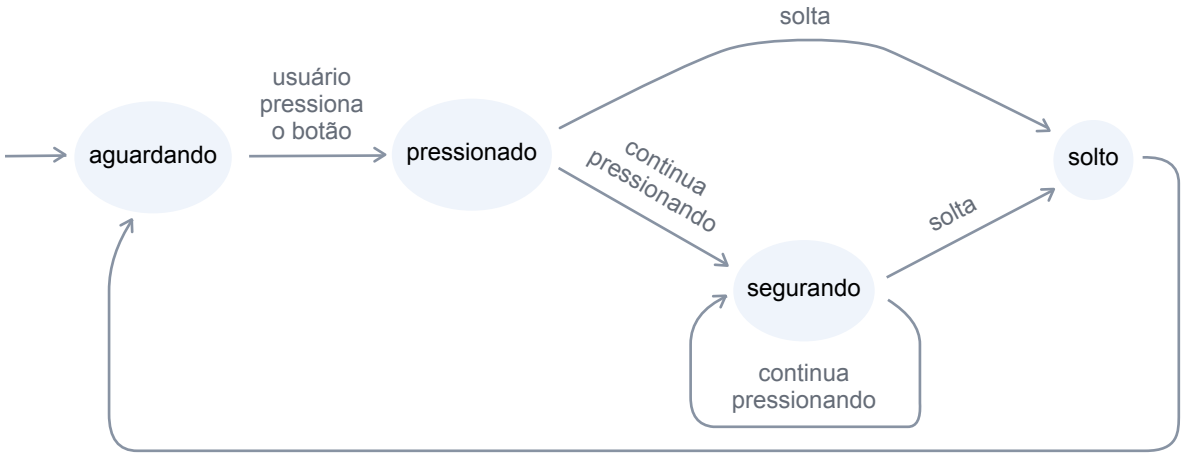
A quantidade e responsabilidade dos módulos de tempo de execução de um motor dependem da plataforma (computadores pessoais, consoles portáteis, celulares etc) e recursos oferecidos (suporte a 3D, realidade aumentada etc). Contudo, como apontado por Zarrad (2018), existem módulos básicos, como o de gestão de conteúdo, que é responsável por processar os arquivos de multimídia utilizados no jogo. Este trabalho, em seu capítulo 3, detalha as ferramentas e módulos que compõe a game engine implementada.

2.2.3 Dispositivos de interface humana (HID)

Gregory (2018) explica que video games são simulações interativas e isso significa que os jogadores devem ser capazes de inserir informações, alterando assim o curso do software. Comenta ainda que tal efeito é alcançado em consoles tradicionalmente por meio de botões em um controle, mas que existe outras maneiras de receber entradas do jogador, como sensores de movimento de um Nintendo WiiMote e a câmera de uma EyeToy, da Sony. A estes dispositivos se dá o nome de interface humana, ou HID, em inglês.

Neste trabalho utilizamos apenas as interfaces por meio de botões táteis – que podem ser vistas na subseção 3.2.7. Em termos de software, uma máquina de estado como a da Figura 8 foi implementada no motor e, sendo assim, os programadores de jogos apenas utilizam funções de alto nível para inspecionar o estado dos botões, sem que seja preciso

Figura 8 – Máquina de estados finita ilustrando os possíveis estados em que um botão de um controle de console de video games pode estar em um dado quadro da simulação.



Fonte: autoria própria.

reimplementar esta lógica a cada jogo. Este tema é discutido na seção 3.3.1. Mesmo sem uso prático nesta pesquisa, o motor já foi construído para suportar mais de um controle.

2.2.4 Gestão de conteúdo

Dada a natureza *data-driven* de um motor, ou seja, sua estruturação baseada em conteúdos multimídia armazenados em arquivos, facilitar e gerenciar o acesso a recursos de imagem, scripts, mapas, efeitos e trilha sonora é essencial. Para Gregory (2018), "Os jogos são por natureza experiências multimídia. Portanto, um motor de jogos precisa ser capaz de carregar e gerenciar uma grande variedade de diferentes tipos de mídia"[7].

Consoles dos anos 80 e 90 utilizavam cartuchos com chips de memória. Um cartucho para o console Super Nintendo, por exemplo, era capaz de armazenar até 8 mega bytes de dados (NINTENDO OF AMERICA INC, 1993). Gradualmente a capacidade de armazenamento foi elevando-se até chegar ao atual disco Blu-Ray dos Sony Playstations 4, capaz de armazenar 50 gigabytes.

Quanto aos video games estudados durante este trabalho, em alguns deles pode-se armazenar conteúdo tanto na memória ROM, quanto em um cartão Micro SD. Uma tabela comparativa é apresentada na Figura 9. Esta é a situação do ODROID-GO, detalhado na sub-seção 2.4.4. O tamanho de um jogo para ele pode ser de gigabytes, devido a capacidade do cartão de memória. Contudo o conteúdo precisa ser antes carregado para memória

[7] "Games are by nature multimedia experiences. A game engine therefore needs to be capable of loading and managing a wide variety of different kinds of media", com tradução do autor.

Figura 9 – Comparação das capacidades de memória dentre video games baseados em microcontroladores.

Aparelho	Memória RAM	Memória Flash	Suporta Micro SD?
Hard Kernel **ODROID-GO**	520 Kilobytes	16 Megabytes	Sim
LVNDR-01	128 Kilobytes	1 Megabyte	Não
Pokitto	36 Kilobytes	256 Kilobytes	Sim
Kittenbot **Meowbit**	96 Kilobytes	512 Kilobytes	Sim
Adrafruit **PyBadge LC**	192 Kilobytes	2 Megabytes	Não

Fonte: autoria própria, com dados das fabricantes.

RAM, limitada em 520 Kilobytes. Claramente a gestão de recursos precisa ser feita de maneira cautelosa. Gregory (2018) comenta sobre o tema, em seu livro sobre motores de jogos afirma que:

> Como a memória geralmente é escassa, um motor de jogos precisa garantir que apenas uma cópia de cada arquivo de mídia seja carregada na memória a qualquer momento. Por exemplo, se cinco modelos compartilham a mesma textura, então gostaríamos de ter apenas uma cópia dessa textura na memória, não cinco.[8].

Uma maneira eficiente de controlar quais recursos devem estar carregados, descompactados e disponíveis para o programa no momento correto é dividir o jogo em cenas.

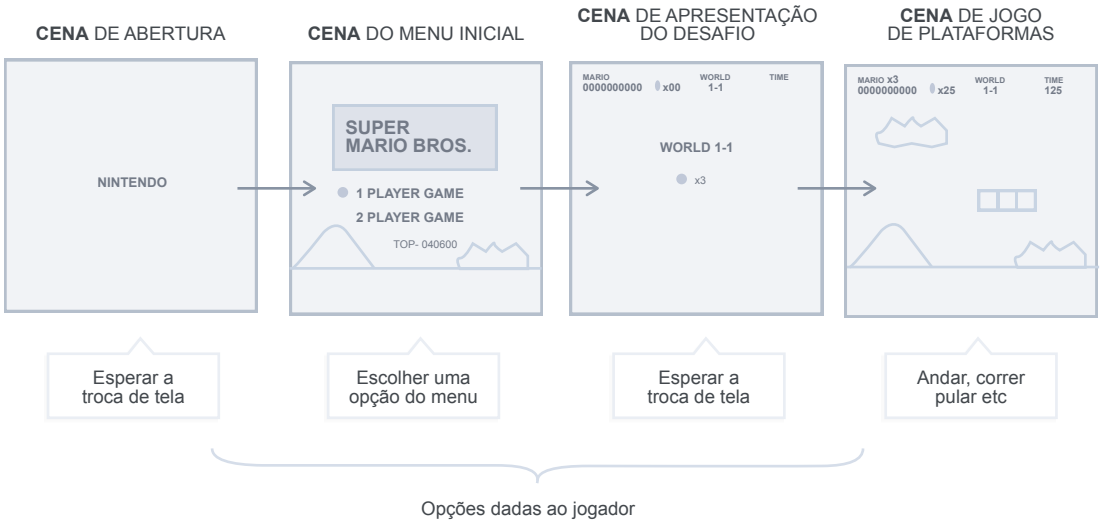
2.2.5 Arquitetura de cenas

Uma das maneiras de se representar o conteúdo que será desenhado em um video game é por meio de cenas, que encapsulam cada área especifica do jogo, nos diz Shekar e Haney (2018). Considerando o clássico Super Mario Bros., ilustrado na Figura 10, podemos pensar, por exemplo, em ter uma cena para o menu, uma para abertura, outra para créditos e uma especialmente complexa para caminhar sobre plataformas e enfrentar os inimigos de um certo mapa.

Cada cena contém objetos em movimento, cenários, trilha sonora e sua própria maneira de interagir. (GREGORY, 2018) nos traz uma analogia, motivando-nos a pensar nos desafios da geração de uma imagem a partir de uma cena em três dimensões:

[8] "because memory is usually scarce, a game engine needs to ensure that only one copy of each media file is loaded into memory at any given time. For example, if five meshes share the same texture, then we would like to have only one copy of that texture in memory, not five", com tradução do autor.

Figura 10 – Possível divisão de cenas para o jogo Super Mario Bros., lançado em 1985 para o console Nintendo NES.



Fonte: autoria própria. Mario é marca registrada da Nintendo.

> Uma cena do mundo real é composta de objetos. Alguns objetos são sólidos, como um tijolo, e alguns são amorfos, como uma nuvem de fumaça, mas cada objeto ocupa um volume de espaço 3D. Um objeto pode ser opaco (nesse caso, a luz não pode passar por seu volume), transparente (nesse caso, a luz passa por ele sem ser dispersada, de modo que podemos ver uma imagem razoavelmente clara do que está por trás do objeto) ou translúcido (o que significa que a luz pode passar pelo objeto, mas está espalhada em todas as direções no processo, produzindo apenas um borrão de cores que sugere os objetos por trás dele).

Em tempo de desenvolvimento, cenas ajudam a organizar o conteúdo e, já em execução, ajudam o motor a decidir qual recurso precisa estar carregado na memória e qual pode ser dispensado, além de limitar a quantidade de processamento. Voltando ao exemplo anterior, quando se está no menu inicial o jogo não precisa considerar se o personagem Mario está tentando subir ou não em uma plataforma, ou seja, cenas limitam as ações possíveis em um dado momento da simulação.

2.2.6 O conceito de Gameloop

Segundo Gregory (2018), nos video games a ilusão de animação é feita da mesma maneira que no cinema, com sucessivas alterações de imagens estáticas e existem maneiras diferentes de atingir este resultado. Jogos 3D dos consoles modernos, por exemplo, utilizam múltiplas regiões da memória para, em milissegundos, alternar entre quadros desenhados ou ainda em processamento, técnica conhecida como double buffering. Já nos consoles antigos, nos jogos em duas dimensões dos anos 90, memória e processamento eram recursos

preciosos. Uma técnica usada era a de invalidação de retângulo, em que apenas regiões que mudaram durante os quadros da animação eram redesenhadas.

Gregory (2018) explica ainda que nos video games a sequência de quadros que formam a animação assume papel fundamental, sendo executada normalmente de 30 a até 60 vezes por segundo. Além de desenhar os gráficos na tela, inclui passos como atualizar a posição dos objetos, receber informações dos controles dos jogadores e processar algoritmos de inteligência artificial. A este processo dá-se o nome de *game loop*. "Um game loop tradicional é dividido em três fases distintas: processamento de entradas, atualização do mundo do jogo e geração de saídas"[9], acrescenta Madhav (2014).

2.2.7 Editores de jogos

Devido a sua natureza multimídia, é comum que os motores de jogos ofereçam não só suporte a múltiplos formatos, mas também suas próprias ferramentas de geração de conteúdo. Gregory (2018) sobre o assunto nos diz que "a maioria dos motores de jogos fornece um editor personalizado para criar mundos de jogo. Ainda assim, alguns motores fazem uso de ferramentas pré-existentes para o layout do mundo do jogo."[10] e comenta que o ferramental não precisa ter o melhor visual, os times de desenvolvimento são capazes de se adaptar e utilizar as ferramentas conforme necessário, porém ressalta que qualquer utilitário oferecido por um motor precisa ser fácil de usar e confiável.

2.3 Formatos digitais de multimídia

Do ponto de vista do software, os gráficos, como fotos, personagens de um jogo e até mesmo os caracteres de um texto, eventualmente serão descritos na memória como uma tabela de pontos coloridos, ou *pixels*, que serão exibidos na tela, como elucida Chen (2011). De maneira análoga, a trilha e efeitos sonoros de um software eventualmente serão representados digitalmente na memória do computador como formatos de ondas.

Toda esta informação, necessária para geração uma imagem completa na tela de cristal líquido ou música nos alto falantes, precisa ser formatada digitalmente de alguma maneira, de modo que possa ser armazenada no programa de computador. Gráficos, mapas, textos e sons de um jogo podem ser compactados com o objetivo de se reduzir o tamanho do executável final que será distribuído digitalmente, pela internet, ou fisicamente, por meio de dispositivos de armazenamento (cds, dvds, cartuchos ou cartões de memória). Posteriormente, em tempo de execução, a descompactação é feita pelo motor de jogos,

[9] "A traditional game loop is broken up into three distinct phases: processing inputs, updating the game world, and generating outputs", com tradução do autor.

[10] "most game engines provide a custom editor for laying out game worlds. Still, some engines do make use of pre-existing tools for game world layout", com tradução do autor.

como elucida Gregory (2018):

> (...) a maioria dos arquivos não é usada em seu formato original pelo motor de jogos. Eles precisam passar por algum tipo de pipeline de transformação de arquivos, cujo trabalho é converter os arquivos para um formato binário, necessário para o motor. Para cada arquivo que passa pelo pipeline, há uma quantidade de metadados que descreve como esse recurso deve ser processado. Ao comprimir uma textura, precisamos saber qual tipo de compactação é mais adequado para essa imagem. [11].

A necessidade de se transportar informação digital em pacotes de tamanho reduzido cresceu tão rapidamente quanto a internet. Empresas e comitês desenvolveram formatos de arquivos digitais contendo algoritmos avançados e complexos, valendo-se dos estudos na área da teoria da informação e entropia. Quanto a compactação de áudio, podemos listar como exemplo o formato patenteado MP3, da Moving Picture Experts Group, e para imagens e gráficos temos o formato livre PNG, da World Wide Web Consortium.

Contudo, estes formatos não se aplicam no contexto dos microcontroladores, em que o binário precisa estar na faixa de Kilobytes. Isso deve-se principalmente ao fato de que apenas o código oficialmente distribuído e já compilado para interpretar uma imagem no formato PNG tem mais de 300 kilobytes, dada a complexidade de seus algoritmos. Roelofs e Koman (1999) argumentam que "Pode-se até ler e escrever imagens PNG diretamente, implementando a própria biblioteca PNG, mas essa é uma tarefa bastante grande e geralmente não é recomendada". O mesmo acontece com o formato MP3, apenas a biblioteca para descompactar o formato excede o espaço disponível em memória de um microcontrolador.

Fica claro que é preciso buscar alternativas de representação que ofereçam alguma capacidade de compressão sem que seja necessário incluir uma biblioteca extensa, que tome um espaço valioso no pacote final.

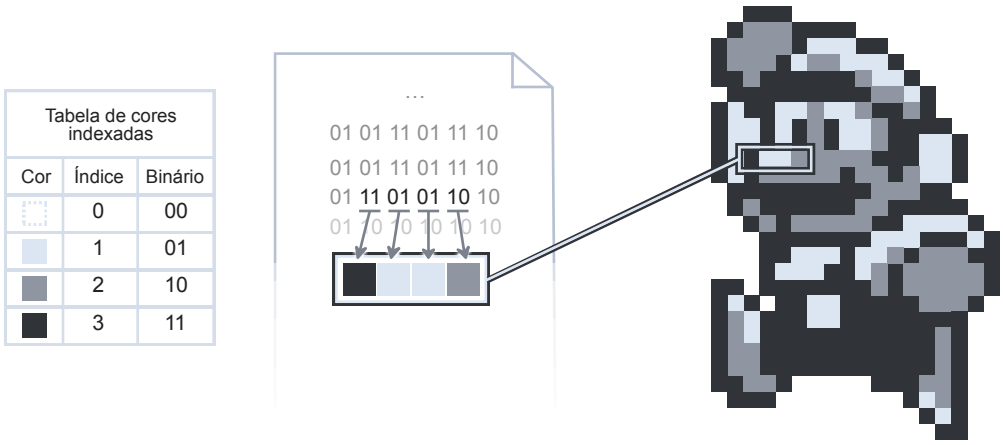
2.3.1 Formatação digital de imagens

Para armazenar imagens digitalmente os consoles das primeiras gerações utilizavam uma técnica baseada em tabelas de cores. No *Super NES*, lançado em 1990 pela *Nintendo*, podia-se definir até 8 tabelas com 16 cores cada uma, escolhidas dentre o total de 32.768 pré-definidas (HUGG, 2019). Em tempo de execução, o processador correlaciona a descrição

[11] "(...) most assets are not used in their original format by the game engine. They need to pass through some kind of asset conditioning pipeline, whose job it is to convert the assets into the binary format needed by the engine. For every resource that passes through the asset conditioning pipeline, there is some amount of metadata that describes how that resource should be processed. When compressing a texture bitmap, we need to know what type of compression best suits that particular image", com tradução do autor.

da imagem com a tabela de cores para gerar a composição final. Desta maneira, cada pixel é representado digitalmente com apenas 4 bits, o suficiente para endereçar a cor na tabela, e não os 15 bits necessários para representar uma entre as 32.768 cores possíveis.

Figura 11 – Exemplo de geração de imagem por indexação de cores



Fonte: autoria própria. Mario é marca registrada da Nintendo™.

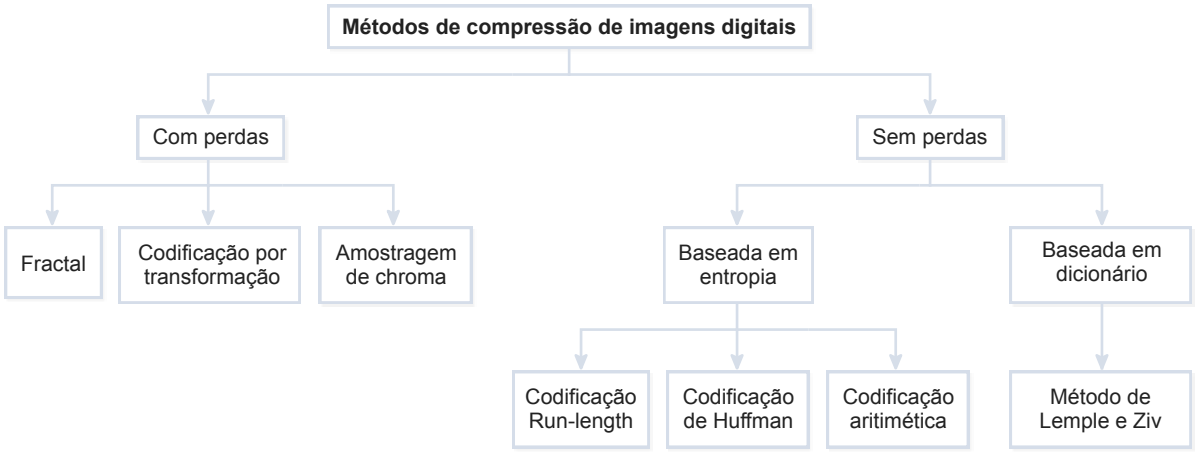
Apesar da limitação na quantidade de cores simultâneas, o uso de uma tabela de cores reduz drasticamente o espaço necessário para armazenar o conteúdo digitalmente. Allen e Triantaphillidou (2012) nos dão uma definição de imagem não comprimida, medida em bits, na equação 2.1. Aplicando a fórmula a uma imagem com 64 pixels de largura por 64 pixels de altura no sistema de cores do *Super NES*[12], vemos que seriam utilizados 61.440 bits para o armazenar o conteúdo de maneira não comprimida. Contudo, são necessários apenas 16.384 bits para uma tabela de 16 cores. Estes 74% de diferença no exemplo dado são chamados de bits redundantes. Um exemplo de imagem com cores indexadas pode ser visto na Figura 11.

$$F = \text{pixels} \times \text{canais de cor} \times \text{bits por canal de cor} \tag{2.1}$$

Na compressão de imagens digitais as redundâncias são identificadas por meio de algoritmos que analisam os pixels, procurando por padrões e informações que possam ser suprimidas. Anitha (2014) faz um comparativo de algoritmos e conclui dizendo que existem duas categorias nas quais estas técnicas se encaixam quando se compacta uma imagem: com perdas e sem perdas, como visto na figura 12.

[12] 15 bits por pixel, sendo 5 bits para cada um dos 3 canais de cores (vermelho, verde e azul).

Figura 12 – Algoritmos para compressão de imagens



Fonte: Chudasama et al. (2015), com ajustes do autor.

Neste trabalho serão comparadas adiante implementações dos algoritmos de codificação run-length e codificação Huffman para compactação de imagens no contexto do hardware estudado. Estes métodos foram escolhidos pela capacidade de compactar imagens sem perdas de definição, ideal para resoluções baixas, como as das telas dos consoles de video games abordados.

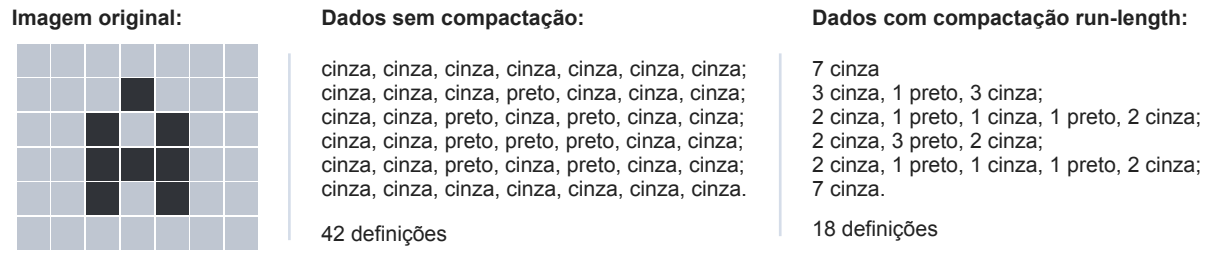
2.3.2 Codificação run-length e codificação de Huffman

Maan (2013) define a codificação run-length como a mais simple forma de compactação de dados. Consiste em apenas anotar o número de repetições de um caractere. A mensagem "AAAABBACCC" seria compactada para "4A2B1A3C", por exemplo. Ainda segundo Maan (2013), "ele substitui as sequencias de dois ou mais do mesmo caractere por um número que representa a duração da sequencia, seguido pelo caractere original"[13]. Mahoney (2012) nos dá outro exemplo, afirmando que "o texto AAAAA será codificado como (5, A)"[14]. Por fim, a Figura 13 ilustra um caso prático. Uma grande vantagem da simplicidade desta técnica, no contexto dos microcontroladores e motores de jogos, é o pouco tempo de CPU necessário para se descompactar a imagem, run-length tem complexidade $\mathcal{O}(n)$. Porém, Ajmire e Khandwani (2018) atentam para o fato de que o uso deste algoritmo pode, no pior caso, em que não ocorrem repetições, gerar um arquivo com o dobro do tamanho em relação a seu original.

A codificação de Huffman usa códigos de tamanho variável, ao invés de códigos de

[13] "It replaces runs of two or more of the same character with a number which represents the length of the run, followed by the original character", com tradução do autor.
[14] "the string AAAAA would be coded as (5,A)", com tradução do autor.

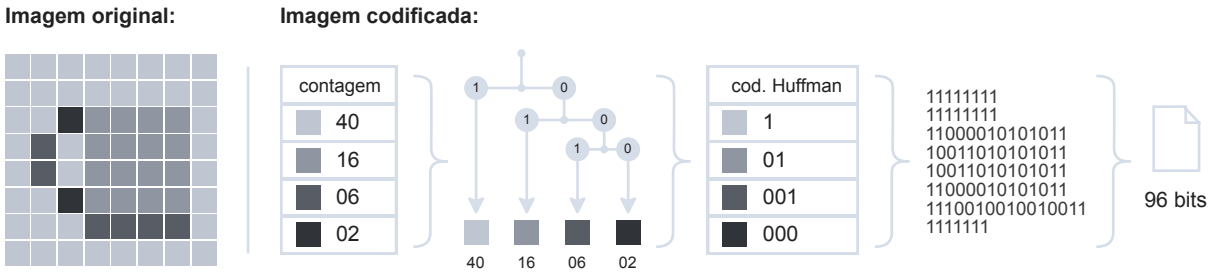
Figura 13 – Exemplo da redução quando aplicada a codificação run-length



Fonte: autoria própria.

tamanho fixo, de modo que a informação que mais se repete dentro da mensagem seja associada a um código de tamanho menor, como detalha Maan (2013).

Figura 14 – Exemplo da codificação de Huffman



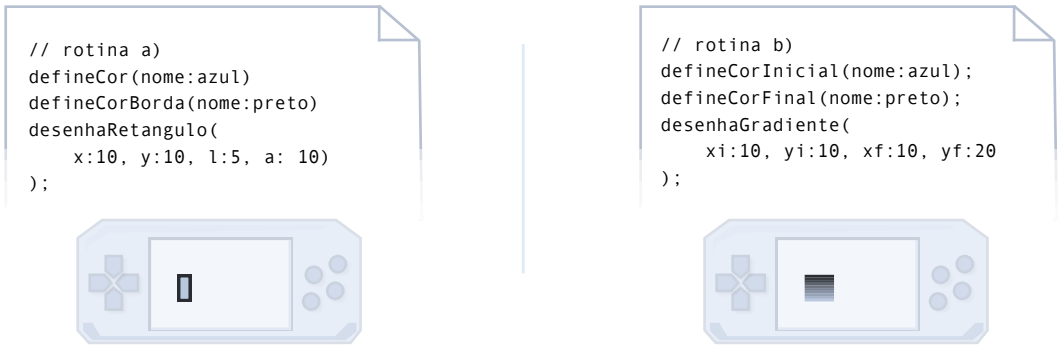
Fonte: autoria própria.

Por exemplo, se em uma imagem baseada em 4 cores indexadas conta-se: 40 pixels azuis, 20 ciano, 3 magenta e 1 vermelho, a codificação Huffman utiliza poucos bits para codificar um pixel azul, que se repete mais, e mais bits para codificar um pixel vermelho, já que ele se repete com menos frequência. Um exemplo pode ser visto na Figura 14. Maan (2013) nos dá um algoritmo para se gerar a codificação dos dados: primeiro é preciso gerar uma tabela com a frequência em que cada dado ocorre. Depois gera-se uma árvore binária de baixo para cima, sendo cada dado distinto uma das folhas, já ordenadas da maior para menor frequência. Em seguida, da esquerda para direita, os nós são unidos por um nó pai, de dois em dois até completar a árvore. Finalmente, de cima para baixo, associa-se o valor 1 para filhos à esquerda e 0 para filhos à direita. O percurso da raiz até a folha dá o código de Huffman para aquele dado associado.

2.3.3 Conteúdo procedural

O dicionário Cambridge define o termo "procedural", no contexto computacional, como "um conjunto de instruções em um programa de computador que executam uma tarefa em particular"[15] (PROCEDURAL, 2013). Green (2016) vai além, dizendo que o "processo de algo ser criado via um algorítimo ou procedimento, ao invés de a partir de algo existente previamente, é chamado de geração procedural"[16].

Figura 15 – Geração procedural de imagens em um jogo, com exemplo de pseudo-código.



Fonte: autoria própria.

Especificamente no caso dos jogos, são rotinas que criam conteúdo com base em procedimentos parametrizados, como as ilustradas na Figura 15. Podemos citar ainda uma implementação real: a função love.graphics.rectangle[17], do motor de jogos Love2D, que desenha um retângulo a partir dos parâmetros de localização e tamanho na tela, de uma cor previamente configurada.

Desta maneira é possível criar gráficos em tempo de execução, armazenando previamente no programa apenas os parâmetros. Green (2016), sobre o tema, afirma que "música, imagens, jogos e textos podem todos serem gerados proceduralmente"[18] e, quanto aos benefícios da geração procedural, comenta:

> Como criar grandes níveis e armazená-los não era possível no passado, os jogos foram projetados para construir seus níveis e recursos de forma algorítmica. Você colocaria todos os recursos necessários em sua mídia de armazenamento e faria o

[15] "a set of instructions in a computer program that does a particular task", com tradução do autor.
[16] "process of something being created via an algorithm or procedure, as opposed to already existing, is called procedural generation", com tradução do autor.
[17] disponível em https://love2d.org/wiki/love.graphics.rectangle. Acessado em 12 de maio de 2019.
[18] "music, images, games, and text can all be procedurally generated", com tradução do autor.

software montar o nível na ponta final, do jogador[19].

Figura 16 – Geração procedural de imagens aplicada a jogos.

Fonte: autoria própria. Super Mario Bros. é marca registrada da Nintendo™.

Vale notar que é possível também aproveitar a técnica geração procedural para recriar imagens bitmap alterando-se por parâmetro a paleta de cores, rotação e espelhamento. Este recurso foi utilizado nos jogos dos anos 80 e 90 (Figura 16). Sobre o lendário título Super Mario Bros., publicado pela Nintendo em 1985, o time de desenvolvimento, reunido anos depois, comenta em entrevista ao então presidente da Nintendo, Satoru Iwata:

> Toshihiko Nakago: nós usamos a mesma imagem para as nuvens e grama, apenas mudando a cor. Satoru Iwata: as nuvens e a grama parecem objetos separados, mas na verdade ambos usam os mesmos elementos gráficos. Takashi Tezuka: sim. Foi divertido naquela época, quando tínhamos que ter ideias assim. Toshihiko Nakago: mesmo com os cogumelos e flores, nós estávamos procurando reduzir os bytes que usamos, então desenhamos metade do objeto e depois viramos para mostrá-lo por completo. Satoru Iwata: é por este motivo que os objetos são simétricos (NINTENDO, 2009).

Exploramos o potencial de economia de espaço da geração procedural neste trabalho oferecendo no motor de jogos funções para desenho geométrico.

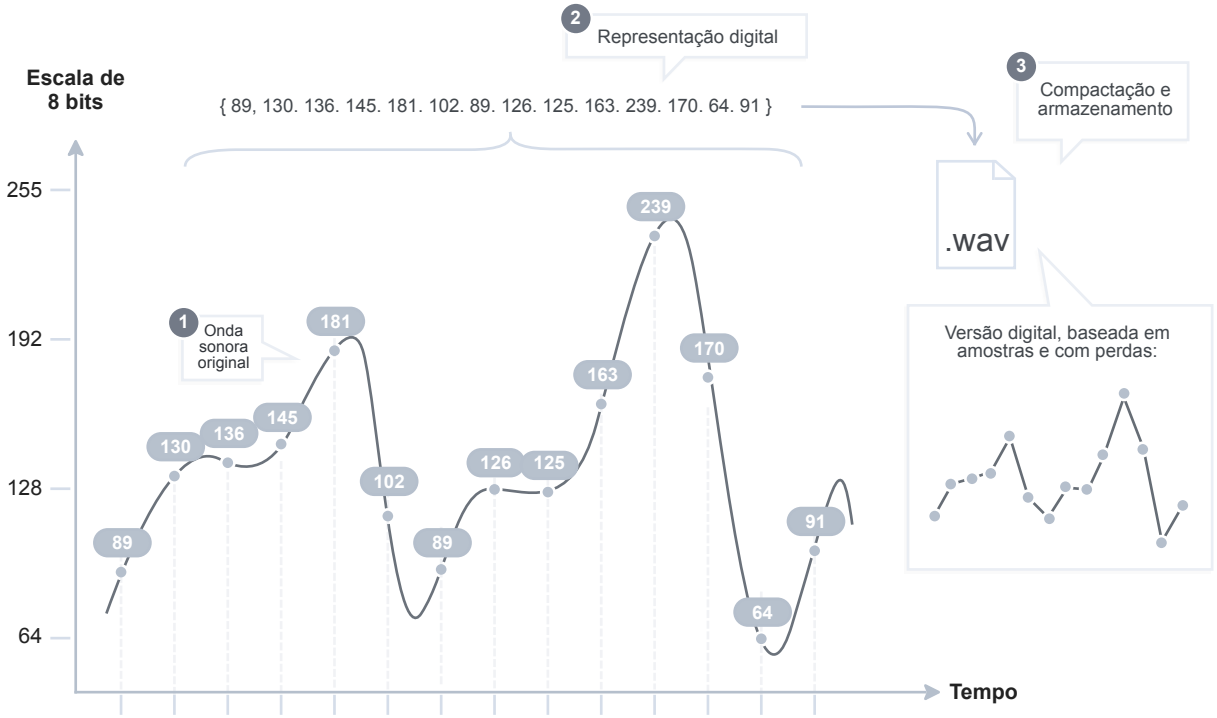
2.3.4 Formatação digital de áudio

Todo áudio pode ser representado graficamente como ondas, que representam, por sua vez, as séries de vibrações carregadas pelo ar. Digitalmente, é possível extrair amostras

[19] "since building large levels and storing them wasn't possible in the past, games were designed to build their levels and resources algorithmically. You'd put all the resources needed on your storage media, and have the software assemble the level at the player's end", com tradução do autor.

do formato de onda e armazenar estes dados como valores binários (SUAREZ; LIMOR, 2018).

Figura 17 – Formatação digital de áudio.



Fonte: (SUAREZ; LIMOR, 2018; POHLMANN, 2011), com ajustes do autor.

Assim funcionam arquivos de áudio digital, como WAV e MP3, armazenando grandes quantidades de amostras do formato de onda e aplicando nelas alguma técnica de compressão. Pohlmann (2011) afirma que quanto mais amostras, maior a fidelidade e diretamente proporcional a quantidade de armazenamento necessária, como ilustrado na Figura 17, em suas palavras: "a amostragem discreta no tempo é o mecanismo essencial que define um sistema de áudio digital, permitindo sua conversão de analógico para digital"[20].

De maneira similar a geração procedural de imagens, os consoles podem oferecer rotinas para sintetização de som em tempo real. É o caso do portátil Nintendo Gameboy, equipado com um processador dedicado a áudio e capaz de gerar ondas a partir de instruções simples em tempo de execução, algo como "gere ondas retangulares numa frequência de 95Hz, aplicando um efeito envelope e que tenha um segundo de duração" em linguagem de máquina. Desta maneira é possível gerar proceduralmente uma música com poucos Kilobytes. Seu manual do desenvolvedor explica em detalhes:

[20] "Discrete time sampling is the essential mechanism that defines a digital audio system, permits its analog-to-digital (A/D) conversion", com tradução do autor.

> O circuito de som consiste em circuitos que geram 4 tipos de sons (...) Som 1: Gera uma forma de onda retangular com funções de varredura e envelope. Som 2: Gera uma forma de onda retangular com uma função de envelope. Som 3: Emite qualquer forma de onda da RAM. Som 4: Gera ruído branco com uma função de envelope (NINTENDO OF AMERICA INC, 1999) [21].

Assim como aconteceu nos anos 90 com o Gameboy, os consoles estudados tem grande limitação de memória RAM, sendo a geração procedural uma alternativa para oferecer suporte a áudio que seja armazenado de maneira compacta.

Considerando a complexidade e quantidade de detalhes específicos do tema visto acima, este trabalho apenas prepara a ferramenta construída para que seja adicionado suporte de áudio futuramente, por meio de funções externas do motor de jogos, de editores e da codificação de controladores, assuntos do capítulo 3.

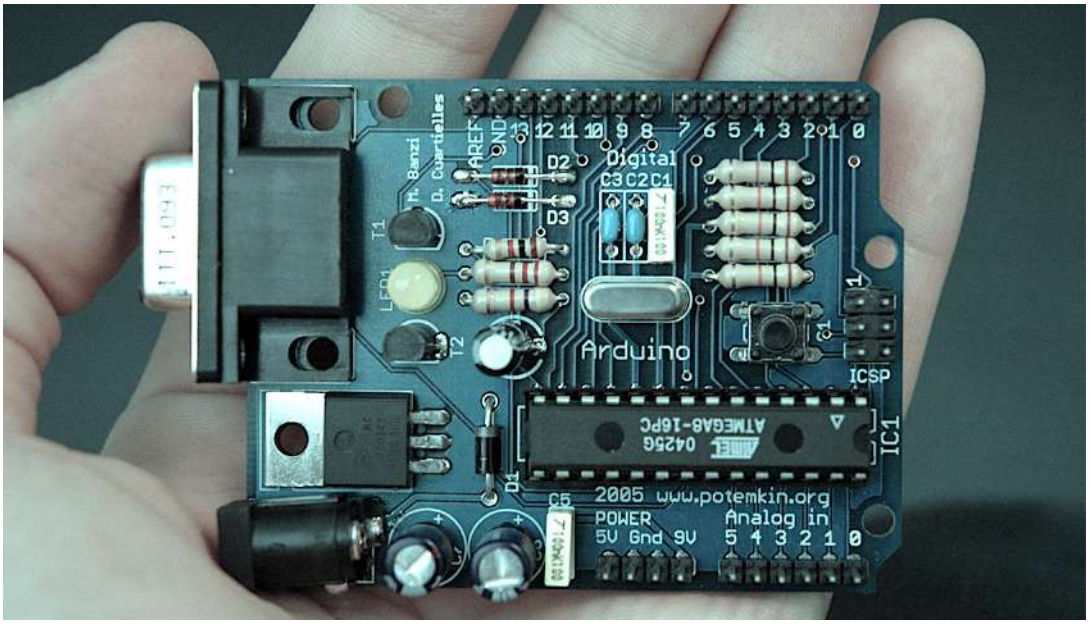
2.4 Microcontroladores e consoles micro controlados

Um microcontrolador é um computador simplificado, capaz de executar um pequeno programa gravado em sua memória flash persistente com auxílio de uma também pequena memória RAM, recebendo e enviando dados por meio de pinos analógicos e digitais de entrada e saída. Eles diferem dos computadores por não terem necessariamente um sistema operacional e serem utilizados para executar apenas uma tarefa específica. São geralmente encontrados integrados a placas que contém os circuitos básicos para o funcionamento, como um conector USB, por onde a memória flash é atualiza com novos programas, e circuitos para fornecer alimentação tanto para o microcontrolador, quanto para os pinos de entrada e saída(YIU, 2015; POUNDER, 2016; HOFFMAN, 2018). Entre as fabricantes, podemos citar as norte americanas Texas Instruments e Microchip Technology Inc, a suiça ST Microelectronics e a chinesa Espressif Systems.

Este trabalho explora o potencial de duas placas com arquiteturas diferentes. Uma baseada em um processador ARM Cortex e a outra utilizando um processador Tensilica Xtensa. Ambas foram encontradas à venda em sites de eletrônicos do Brasil, como Felipe Flop, Auto Core Robotica e Robocore. Também foram escolhidas por integrarem facilmente com a ferramenta Arduino, importante durante a fase de prototipação.

[21] "The sound circuitry consists of circuits that generate 4 types of sounds (...) Sound 1: Generates a rectangle waveform with sweep and envelope functions. Sound 2: Generates a rectangle waveform with an envelope function. Sound 3: Outputs any waveform from waveform RAM. Sound 4: Generates white noise with an envelope function.", com tradução do autor.

Figura 18 – Primeira placa Arduino, um protótipo.



Fonte: foto por Nicholas Zambetti, com ajustes do autor.

2.4.1 Arduino

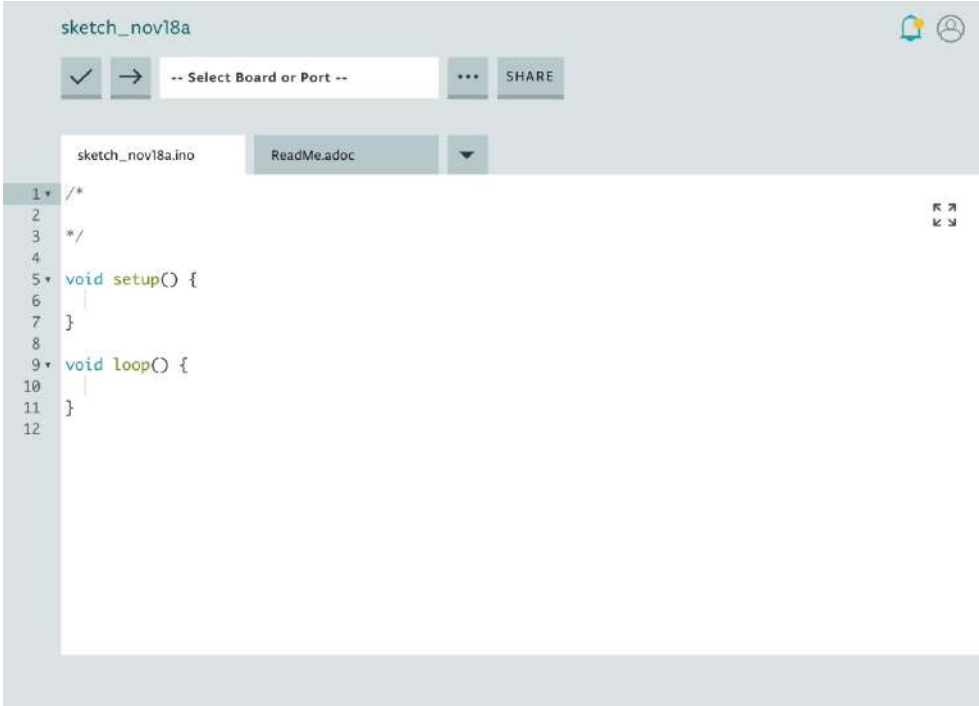
A história do Arduino começa em 2003 na cidade de Ivrea, na Itália, onde Hernando Barragán escreveu sua tese de mestrado intitulada "Wiring: Prototyping Physical Interaction Design". Anos depois, Hoffman (2018), referindo-se a tese de Barragán, diz que seu objetivo foi criar uma ferramenta de prototipagem de baixo custo para projetos de circuitos digitais que pudesse ser utilizada por pessoas sem formação em engenharia. "Isso significava que era preciso funcionar simplesmente conectando o dispositivo a um computador hospedeiro e ter uma interface fácil de usar para programá-lo"[22], complementa.

Hoffman (2018) continua seu relato, listando as tentativas de Barragán em busca de um microcontrolador que atendesse seus requisitos. Inicialmente utilizou um Parallax Javelin Stamp, que suportava a linguagem Java, da americana Oracle Corporation, porém desistiu por questões de licença e termos de uso. Depois fez seus testes em um Atmel ARM 91R40008, que considerou complicado demais e difícil de soldar. Por fim, chegou ao Atmel ATmega128, com o qual obteve sucesso e concluiu com distinção seu projeto de mestrado. Hoffman (2018) acrescenta que "ele vendeu essas placas por aproximadamente US $60. No final de 2005, a Wiring estava sendo usada em várias partes do mundo". Em sua conclusão, Barragán (2004), demonstra interesse em expandir seu projeto, incluindo suporte a versões mais econômicas dos microcontroladores Atmel, e disponibiliza todos os

[22] This meant that it had to be work by simply plugging the device into a host computer and have an easy-to-use interface to program it

detalhes do Wiring, inclusive os diagramas e layouts da placa de circuito.

Figura 19 – Editor de código online Arduino Web IDE.



Fonte: Hoffman (2018).

O projeto Wiring transformou-se em Arduino pelas mãos do orientador de Barragán, Massimo Banzi, que juntamente com David Cuartielles projetou uma placa microcontrolada que custaria apenas USD $30. A Figura 18 mostra o que seria a primeira versão segundo o site oficial, Arduino.cc. O preço baixo era uma maneira de atrair estudantes, motivá-los a criar projetos utilizando suas ferramentas e, eventualmente, receber ajuda em troca. Os fundadores acreditavam fortemente em projetos de hardware e código abertos. Hoje em dia o Arduino é uma plataforma para projetos de eletrônica baseada em placas com controladores de baixo custo, contendo memórias volátil e persistente, portas de entrada e saída de fácil acesso (conhecidas como GPIO, General Purpuse Input Output) e um kit de desenvolvimento de software que se conecta via USB e permite a produção de programas utilizando um editor online, a Arduino Web IDE (Figura 19). Mais do que isso, existe uma grande comunidade online e todos os projetos permanecem com código e hardware abertos (HOFFMAN, 2018).

Vale notar que a escolha por um hardware aberto abre portas para outras fabricantes criarem placas compatíveis com o ambiente de desenvolvimento Arduino e mantém o mercado competitivo. O recente modelo oficial Arduino Uno R3, vendido online pela

Figura 20 – O modelo micro:bit, da BBC.

Fonte: FilipeFlop (2019)

fabricante por USD $22 em 2019, é analisado em detalhes por Hoffman (2018). Ele contém um processador ATmega328P de 16MHz, 32 Kilobytes de memória flash, 2 Kilobytes de memória RAM, 14 pinos de entrada e saída digital, 6 pinos de saída analógica, uma porta USB e conector para alimentação de 5V. Um exemplo de concorrente direto é o modelo Metro 328, vendido pela empresa americana Adafruit Industries. Ele custa apenas USD $17.50, entregando o mesmo controlador e funcionalidades da Uno R3. Sobre esta variedade de modelos e fabricantes de microcontroladores, Pounder (2016) diz:

> Se você está considerando um microcontrolador, então o padrão de fato é o Arduino, graças à sua excelente comunidade. O Arduino é o melhor para aqueles que são novos na cena, mas se você tiver experiência com microcontroladores, dê uma olhada nas novas placas ARM, como a micro:bit (Figura 20), que oferecem mais recursos pelo seu preço e podem ser usadas com múltiplas linguagens de programação. [23]

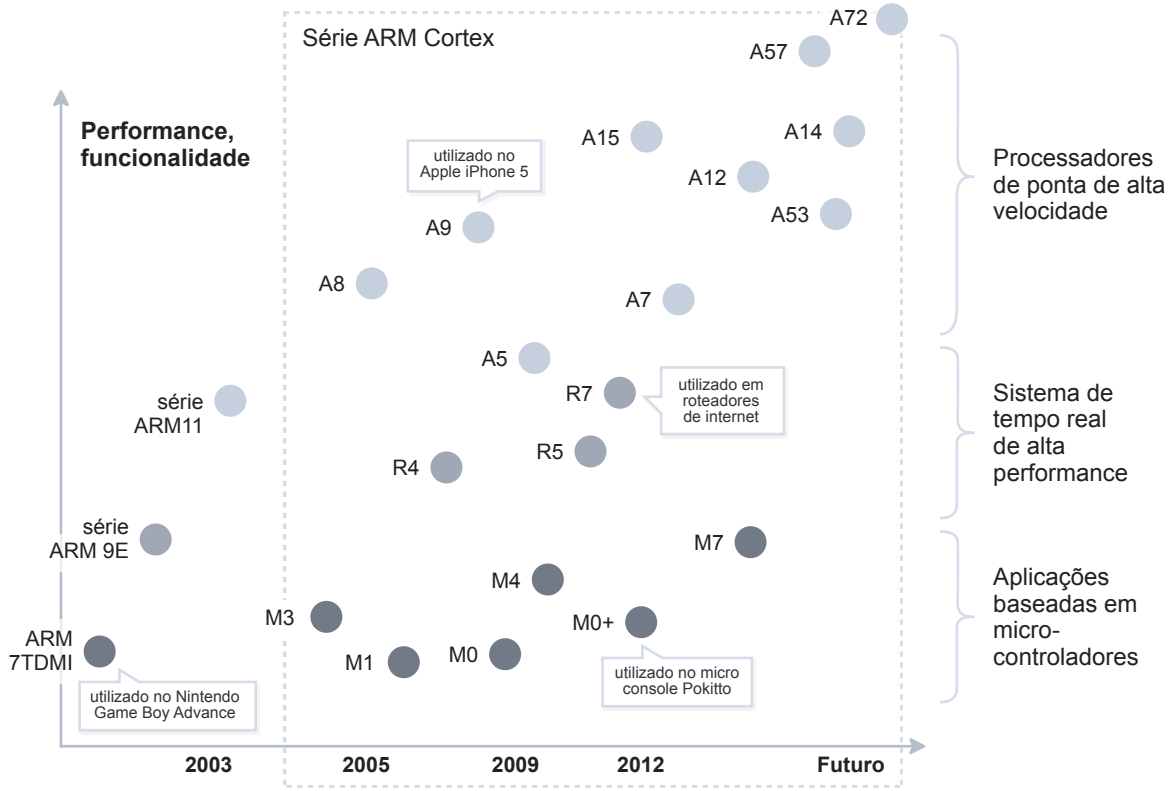
Este trabalho, almejando democratizar a plataforma de microcontroladores como consoles de video games, segue os mesmos passos do Arduino, criando uma ferramenta que opera como entreposto – agregando software e hardware acessíveis.

2.4.2 Arquitetura ARM Cortex

Existem vários modelos de processadores com arquitetura ARM no mercado, para diversas aplicações, como ilustrado no gráfico da Figura 21. Desde processadores de última geração para smartphones a microcontroladores de baixo custo, a Arm Holdings, sediada em Cambridge, vem projetando processadores há mais de 20 anos.

[23] "If you are considering a microcontroller then the de facto standard is the Arduino, thanks to its superb community. The Arduino is the best for those new to the scene, but if you are experienced with microcontrollers then take a look at the new ARM-powered boards, such as micro:bit, which offer more features for their price and can be used with multiple languages", com tradução do autor.

Figura 21 – Linha de processadores ARM, com destaque para geração atual, chamada Cortex.



Fonte: Yiu (2015), com ajustes e anotações do autor.

Após o sucesso inicial do modelo ARM7TDMI, a empresa dividiu seus produtos em três segmentos, Yiu (2015) comenta que "por volta de 2003, a ARM percebeu que precisava diversificar os modelos de processador para atender a diferentes requisitos técnicos em diferentes mercados."[24]. Foram criados os segmentos M, para microcontroladores; R, para aplicações críticas de tempo real e A, para aparelhos que solicitam grande performance. Apesar da variedade de modelos e aplicações, seu principal diferencial, sem dúvida, sempre foi o baixo consumo de energia, proporcionado pela arquitetura e conjunto de instruções proprietários que fazem da linha Cortex a solução ideal para aplicações móveis modernas, que precisam aproveitar ao máximo suas baterias, explica Yiu (2015).

Dentro da linha de processadores M, várias placas micro controladas de diversos fabricantes podem ser encontradas hoje em dia. Um exemplo é a micro:bit, da BBC, com processador ARM Cortex-M0 32 bits de 16MHz, 256 Kilobytes de memória flash e 16 kilobytes de memória RAM. Ela já vem com acelerômetro, sensor de temperatura e suporte a tecnologia sem fio Bluetooth. Na ocasião em que este trabalho foi escrito, o site brasileiro

[24] "In around 2003, ARM realized that it needs to diversify the processor products to address different technical requirements in different markets", com tradução do autor.

FilipeFlop (2019) vendia este modelo, que é importado, por R$ 149,90.

2.4.3 Arquitetura XTensa

Apesar da popularidade dos processadores ARM – advinda do sucesso que o iPhone, da Apple, conquistou – o microcontrolador do ESP32, utilizado no ODROID-GO, é um Xtensa LX6. Desenhado pela empresa americana Tensilica, que foi adquirida posteriormente pela também americana Cadence, o diferencial desta arquitetura é a capacidade de extensão e diversidade de configurações. Ienne e Leupers (2006) comentam sobre a TIE, Tensilica instruction extension; em suas palavras, este tecnologia "permite definir extensões do conjunto de instruções, que são implementadas diretamente por um gerador automático de processador como hardware"[25].

A Cadence comercializa designs de processadores para empresas como a Espressif Systems – que, por sua vez, integram mais componentes no design. Por exemplo, podem ser adicionados ao desenho circuitos de radio frequência e matrizes de memória ROM, criando a especificação de um produto como o ESP32.

2.4.4 ODROID GO

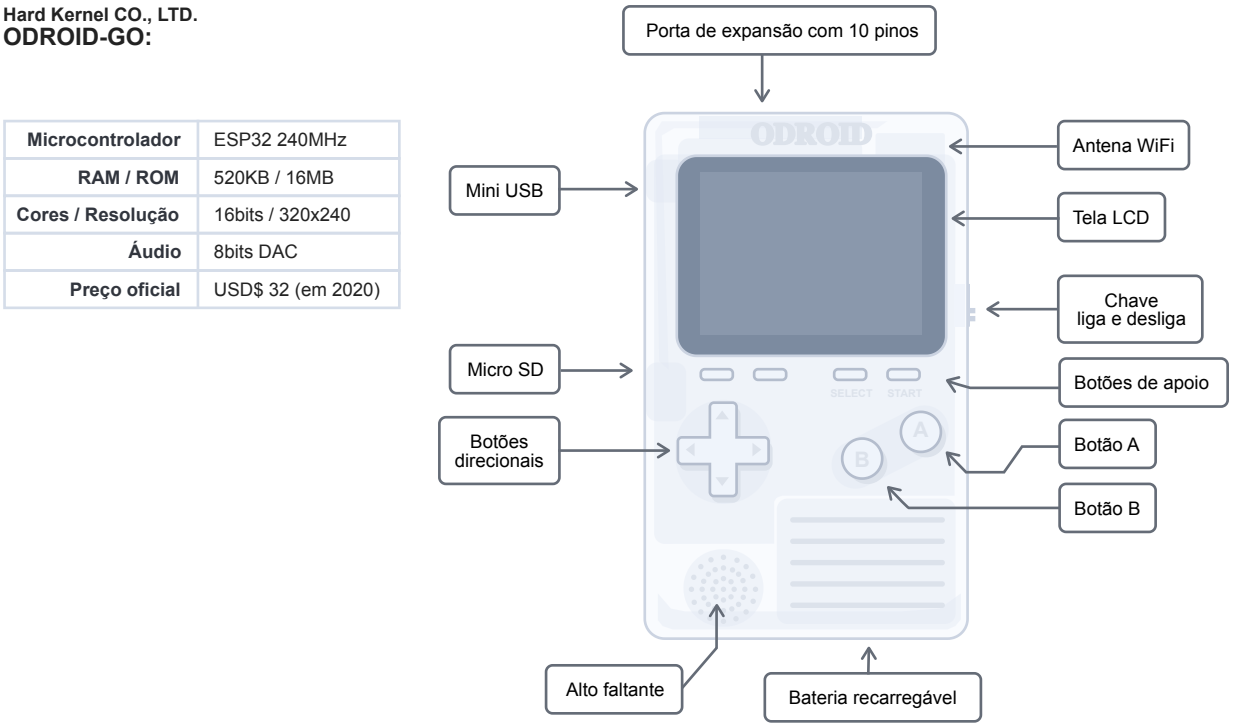
Projetado pela empresa coreana Hard Kernel e lançado em 2018, o vídeo game portátil ODROID GO, apresentado na figura 22 e detalhado internamente na figura 23, é vendido em seu site oficial como um kit para ser montado e pode ser encontrado no Brasil em lojas virtuais. É alimentado por uma bateria recarregável inclusa e utiliza como microcontrolador um módulo ESP32-WROVER-B, da fabricante chinesa Espressif Systems.

Modelos da linha WROVER e WROOM, que incluem um microcontrolador ESP32, têm sido cada vez mais utilizados em projetos da comunidade maker. A plataforma de comércio virtual de kits eletrônicos de desenvolvedores independentes Tindie, em 2020 já listava 143 projetos na categoria ESP32, contra 931 projetos na categoria Arduino e 406 na categoria Raspberry Pi, plataformas mais antigas e consolidadas[26].

O microcontrolador ESP32 é oferecido em configurações com até 520 Kilobytes de SRAM e sua CPU tem velocidade configurável, chegando a 240MHz. Todas as versões incluem o aparato necessário de rádio e criptografia para que seja possível acessar à internet via Wi-Fi e podem ter as memória Flash e RAM expandidas utilizando chips externos (ESPRESSIF SYSTEMS, 2018). Tudo isso por um valor reduzido. No site americano

[25] "(...) allows one to define instruction extensions that are directly implemented by an automated processor generator as hardware (...)", com tradução do autor.
[26] https://www.tindie.com, acessado em 14/11/2020

Figura 22 – Especificações do console portátil ODROID GO.



Fonte: autoria própria, com dados de Hard Kernel Co. Ltd.

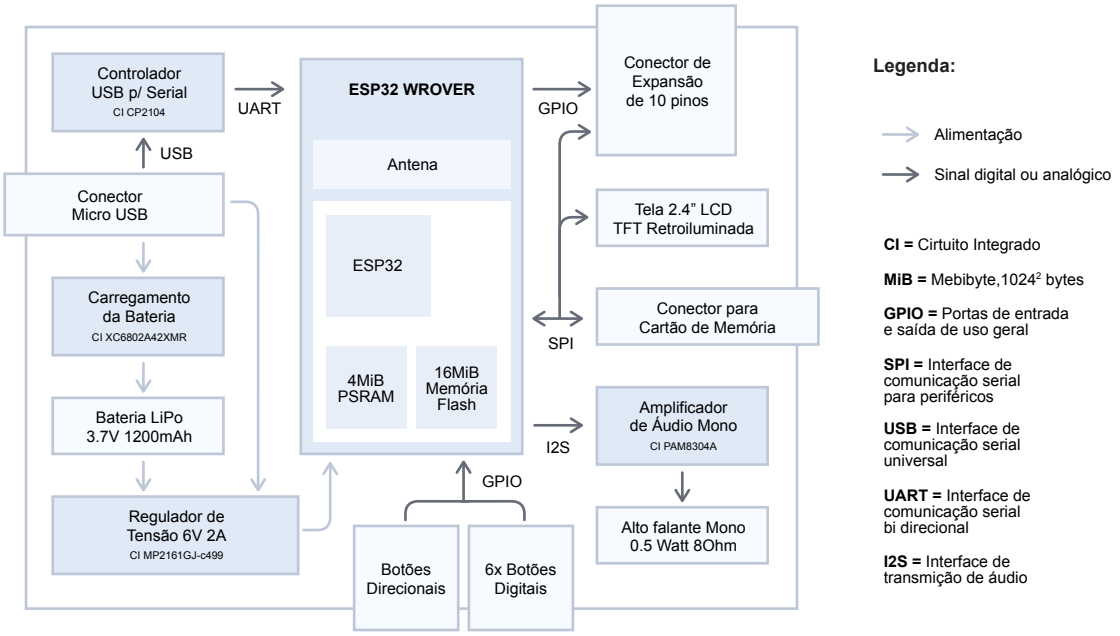
de comércio eletrônico Adafruit, a versão ESP32-S2-WROVER-I pode ser adquirida por menos de 4 dólares[27]. Para ser ter ideia do custo-benefício, um Arduino Nano Every custa mais de 10 dólares, tem um processador Atmel de 20MHz, apenas 6 Kilobytes de SRAM e sem suporte a Wi-Fi[28].

Fisicamente, o console portátil contém dez botões: quatro para os direcionais, A, B, “select”, “start”, “menu” e “volume”. Tem entrada para cartão de memória no formato MicroSD, um conector de 10 pinos para expansão, alto falante e uma chave liga e desliga. Tanto o upload do firmware quanto o carregamento da bateria são feitos por uma porta Mini USB.

A tela é colorida e de cristal líquido, com iluminação traseira. Utiliza um barramento compartilhado com o leitor de cartão de memória para comunicar-se com o microcontrolador, como mostra a figura 23. Isso significa que enquanto o cartão de memória é lido, a tela precisa aguardar e vice-versa. Tem suporte a cores de 16 bits, ou seja, mais de 65 mil cores por pixel, e resolução de 320 por 240 pixels. Apesar do seu controlador, o ILI9341, suportar sincronização vertical na atualização da tela, ou seja, suportar receber do microcontrolador

[27] https://www.adafruit.com/product/4760. Acessado em 14/11/2020.
[28] https://store.arduino.cc/usa/nano-every. Acessado em 14/11/2020.

Figura 23 – Diagrama de blocos do hardware ODROID GO.



Fonte: Hard Kernel Co. Ltd., com modificações do autor.

instruções sobre quando deve começar a atualizar a tela novamente, a placa de circuito impresso do ODROID GO não faz fisicamente esta ligação. Por este motivo sua tela atualiza automaticamente e fora de sincronia com o código que está sendo executado no ESP32, o que causa um efeito chamado screen tearing, onde a tela exibe uma parte do quadro atual sobre o quadro anterior. Seu brilho e curva gama podem ser controlados em tempo de execução.

O kit de desenvolvimento de software oficial, chamado ESP-IDF, inclui uma ferramenta de linha de comando para compilação dos projetos e upload via porta USB para o microcontrolador. O compilador utilizado nestes trabalho é uma variação de GCC com suporte a arquitetura Xtensa, pode ser encontrado em um repositório online da fabricante, no site Github[29]. Foi utilizado também o kernel de sistema operacional suportado oficialmente, o FreeRTOS.

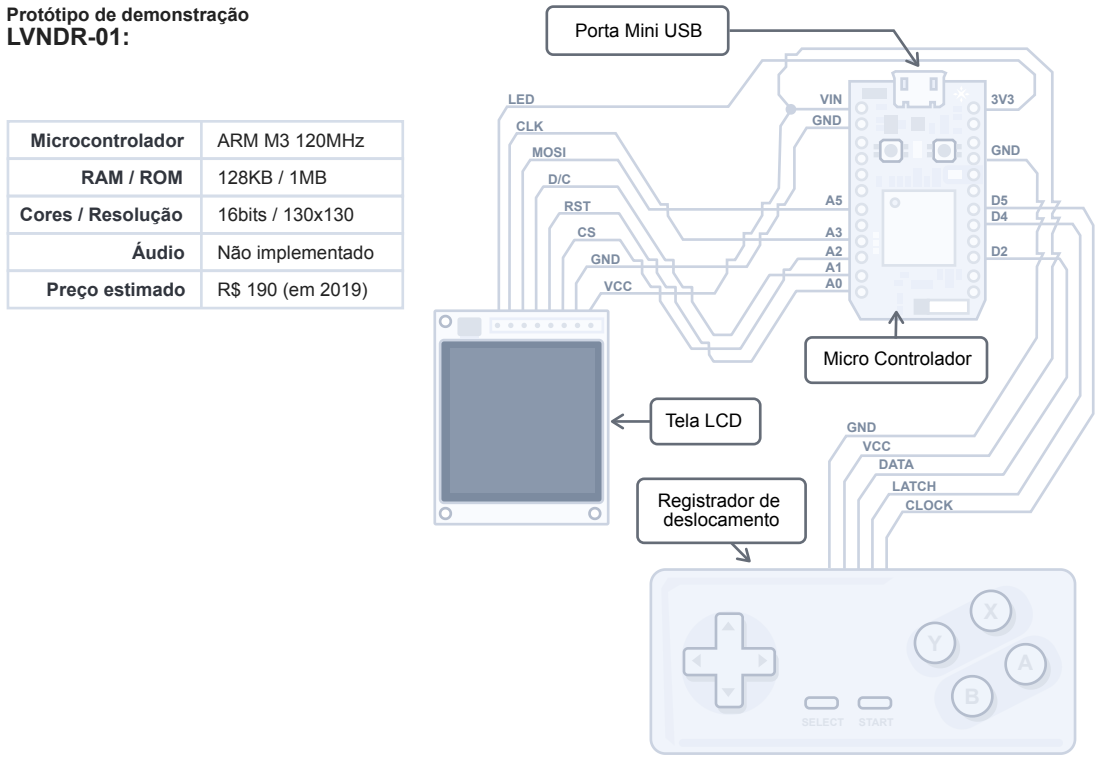
2.4.5 Protótipo LVNDR-01

Durante a fase de pesquisa estudamos uma variedade de video games baseados em microcontroladores (Figura 9). São memória e velocidades de processamento limitadas, mas diversas. Telas de tamanhos e qualidades diferentes, coloridas e monocromáticas. Cada um com seu manual, compilador e ferramentas.

[29] https://github.com/espressif, acessado em 15/11/2020.

Buscando unificar o software mantendo a liberdade de hardware, nossa proposta foi criar um motor que facilite o trabalho do desenvolvedor de jogos e que, ao mesmo tempo, esteja preparado para a adição de novos consoles sem que o código dos jogos já desenvolvidos precise ser alterado ou escrito novamente.

Figura 24 – Especificações do protótipo de videogame LVNDR-01, desenvolvido durante este trabalho.



Fonte: autoria própria.

Para explorar e estressar essa hipótese criamos nosso próprio console baseado em microcontrolador, o LVNDR-01, apresentado na figura 24. Em sua construção foram utilizados:

- Dois kits de placas de ensaio, conhecidas pelo termo em inglês *protoboard*, acompanhadas de fios coloridos e adaptados para prototipação.

- Uma placa Photon PØ, da empresa americana Particle, que contém um microcontrolador STM32F205 ARM Cortex M3 de 120MHz com 128KB RAM. Um chip Broadcom BCM43362, que adiciona suporte a Wi-Fi. É alimentada por uma porta Mini USB.

- Uma tela de cristal líquido retroiluminada de 1.6 polegadas, 130 por 130 pixels de resolução.
- Um controle de video game, baseado em um registrador de deslocamento, com 12 botões.

Todos componentes foram comprados no Brasil, em 2019, no site Felipe Flop, com exceção do controle de video game. A seção 3.2 descreve em detalhes como a comunicação entre microcontrolador e seus periféricos foi implementada.

Optamos pelo Photon PØ principalmente porque ele facilita e acelera o fluxo de trabalho. As atualizações são feitas rapidamente e via Wi-Fi. Durante o desenvolvimento medimos o tempo de update do firmware, do início da compilação, passando pela transferência sem fio, até o início da execução do código no microcontrolador. Registramos média aritmética de 13,6 segundos, com desvio padrão de 1,1s, após 10 testes em dias diferentes. Por estar ligado a Internet, esta placa ainda oferece um painel online, onde acompanha-se em tempo real uso de memória e processador. Com apenas uma linha de código é possível também registrar um ponteiro de memória para que seu valor seja monitorado online. Muito útil, por exemplo, para mensurar a taxa de quadros por segundo.

A tela foi escolhida principalmente pela disponibilidade e baixo custo, mantendo o protótipo acessível. Quanto ao tamanho, optamos por algo similar a outros dois consoles micro controlados, Meowbit Kittenbot e Adafruit PyBadge LC, ambos com telas de 160 por 128 pixels.

Por fim, em relação ao controle, que é utilizado pelo jogador para interagir com o video game, inicialmente foram levantadas três opções:

- Utilizar botões conectados as portas de uso geral do microcontrolador, criando um controle na placa de ensaio.
- Implementar o suporte a controles de video game utilizados em computadores de mesa, por meio da interface USB.
- Adicionar suporte a um controle de video game existente, como o do Super NES, da Nintendo.

Escolhemos a última opção pela simplicidade de implementação do software e hardware, também por dispensar o uso de componentes extras, como resistores e capacitores. Detalhes e resultados podem ser vistos na sub seção 3.2.9.

2.5 Trabalhos relacionados

Dada sua atual importância no processo de produção de um jogo eletrônico, motores de jogos já foram muitas vezes material de estudo. Autores em suas pesquisas aprimoram o pipeline gráfico, exploram novas arquiteturas, entre outros temas. Desta maneira a academia contribui para o avanço da industria. Na sequência são apresentados estudos relevantes.

2.5.1 Biblioteca para Desenvolvimento de Jogos para o Nintendo DS

Em sua monografia, Santos (2009) inicia descrevendo o avanço da capacidade dos video games ao longo do tempo. Em seguida apresenta as especificações de um deles em particular, objetivo da dissertação, o portátil Nintendo DS. Há um detalhado estudo sobre processadores, regiões de memória, o sistema de cores suportado e modos de desenho presentes no aparelho. O autor descreve as possibilidades oferecidas, mas direciona seu texto para o que considera ser o uso mais comum, excluindo, por exemplo, o suporte a gráficos 3D.

Por fim, é proposta a FLIB, uma biblioteca para jogos alto nível, em C++, baseada em asserções. Sua API abstrai detalhes complicados de hardware, como ajustes de registradores e alocação de memória de baixo nível, assumindo as configurações e modos selecionados como ideais para cumprir a tarefa proposta. Cita ainda a implementação de um jogo conceito, comprovando a viabilidade e mensurando a eficiência de seu motor, que relata ocupar apenas 100KB no binário final.

Neste trabalho relacionado o autor dedica um capítulo inteiro para o estudo do hardware e suas capacidades. De maneira similar, a presente monografia contém as especificações técnicas das plataformas que pretende atender, visando os mesmos objetivos: escolher as melhores opções de algoritmos quando se tem um equipamento com memória e processamento limitados.

2.5.2 Construindo um motor de jogos: um conto de engenharia moderna baseada em modelos

Neste paper os autores Guana, Stroulia e Nguyen (2015) exploram a capacidade de fazer uma junção entre o motor de jogos e uma linguagem de programação especializada. Logo na introdução, citam os desafios da construção de engines, que precisam ser uma ferramenta simples de usar e, ao mesmo tempo, flexível para conseguir modelar as inúmeras ideias de jogos que os usuários possam ter. Justificam as vantagens que um software destes trás, afirmando que "tornam a implementação de projetos de jogos complexos menos dispendiosos, menos tecnologicamente frágeis e significativamente mais eficientes".

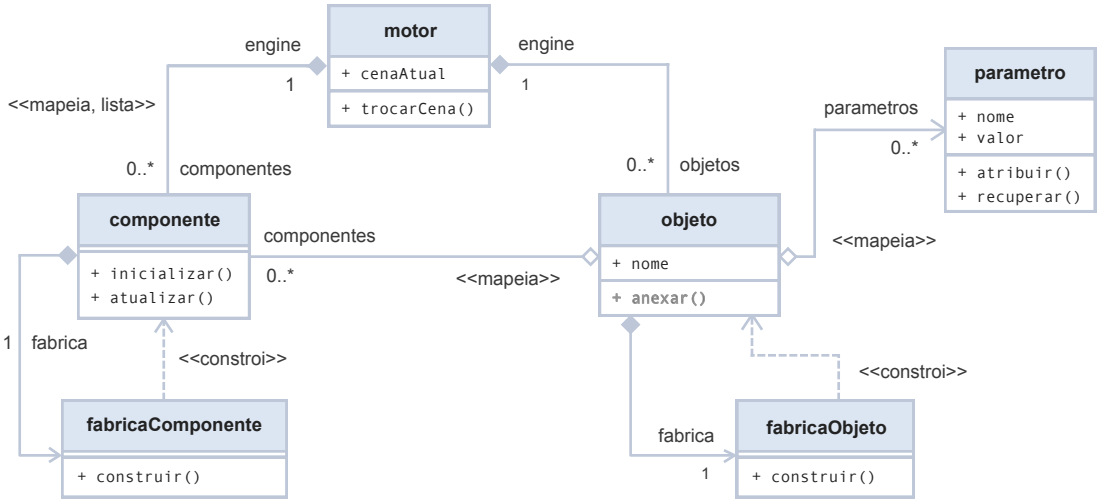
Atribuem parte destes ganhos a maneira como um motor de jogos força os desenvolvedores a raciocinar para criar o jogo dentro da modelagem suportada.

Dada a fundamentação, partem para proposta de uma linguagem de programação que especifica os elementos do jogo e motiva o usuário a pensar em quem são os objetos da simulação, quais ações são capazes de realizar, onde moram e quais objetivos tem. Concluem apresentando seus resultados utilizando um software por eles construído e afirmam que "os motores de jogo impõem uma arquitetura aos jogos que eles produzem e executam".

Assentimos com os autores no sentido de que um motor colabora inclusive para organização dos projetos de jogos, estabelecendo regras de funcionamento e estruturação. Obrigar, por exemplo, que cada cena de um jogo seja construída dentro de uma pasta no sistema de arquivos e que todos os gráficos e imagens necessários a esta cena estejam nesta pasta impõe implicitamente organização.

2.5.3 Gear2D: um motor extensível de jogos baseado em componentes

Figura 25 – Diagrama de classes do motor de jogos Gear 2D.



Fonte: Freitas (2013), com ajustes do autor.

Muitas vezes dada como certa, a orientação a objetos como arquitetura de software é analisada em detalhes nesta monografia por Freitas (2013). O autor exemplifica como herança e polimorfismo prejudicam um projeto de jogos, em que adições no comportamento dos objetos podem ser necessárias num estágio avançado da produção, potencialmente afetando também o comportamento de todas as subclasses envolvidas.

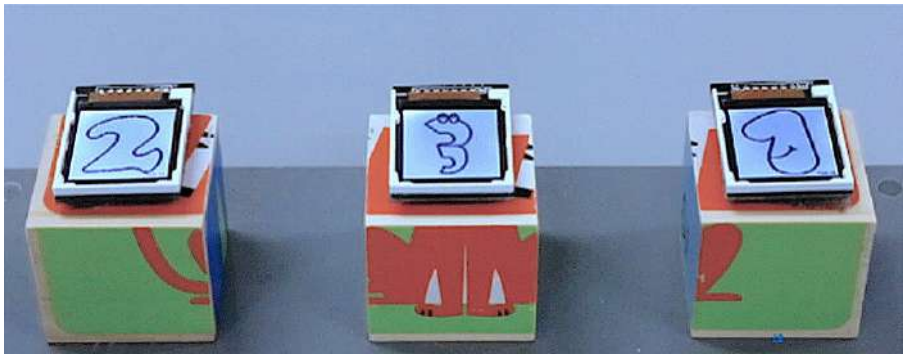
Na sequencia, discorre sobre uma possível alternativa, comentando, em suas palavras, que "alguns motores de jogos apresentam aos desenvolvedores uma arquitetura baseada em componentes, tais como Unity3D, Push Button Engine, entre outras. Nesse caso, entidades são definidas através da agregação de componentes ao invés de herança". Para demonstrar os benefícios desta arquitetura, Freitas (2013) estrutura seu próprio motor de jogos, ilustrado na Figura 25; implementa com sucesso o sistema de componentes proposto, criando elementos básicos de renderização, colisão entre objetos e suporte a scripts, sem que isso altere a taxonomia; relata, por fim, projetos de jogos criados com seu motor e como, com isso, a arquitetura foi aprimorada.

A monografia de Freitas (2013) enriquece as pesquisas na área por detalhar as decisões conceituais do projeto de um motor de jogos. Inspirados por este trabalho, optamos também por utilizar uma estrutura de gestão de cenas.

2.5.4 Um protótipo de quebra-cabeça tangível para crianças com transtorno de déficit de atenção com hiperatividade

Fan et al. (2018) narram em seu artigo a construção de um quebra-cabeça composto por peças eletrônicas. Cada uma delas é equipada com uma tela de LCD e um microcontrolador Intel Edison com capacidade de comunicação sem fio, além de um computador servidor conectado a sensores.

Figura 26 – Protótipo de quebra-cabeça tangível



Fonte: Fan et al. (2018), com ajustes do autor.

O propósito deste experimento, mostrado na figura 26, foi avaliar por meio de biofeedback como comportam-se crianças portadoras de déficit de atenção com hiperatividade enquanto brincavam com as peças do brinquedo eletrônico. Segundo os autores, este método é vantajoso porque gera dados que podem ser gravados durante uma sessão terapêutica,

analisados e comparados posteriormente. Comentam ainda do uso de múltiplos sensores na brincadeira e citam um exemplo de implementação onde a criança pode assoprar um microfone na peça para resolver um enigma. Em sua conclusão, fazem uma reflexão:

> O conteúdo do jogo, até certo ponto, determina o caminho interativo. Além disso, a interação do jogo é determinada por diferentes sensores. Por último, temos que considerar se a experiência do usuário é agradável ou não e se a maneira de interagir do jogo é natural ou não.[30]

O desenvolvimento de jogos simples, mas de grande importância social – como os do protótipo de Fan et al. (2018) – podem ser simplificados por meio de motores de jogos. Eles tornam essa tecnologia mais acessível, colocando os jogos eletrônicos na posição de opção viável para tratamentos como o do estudo mencionado.

[30] "the content of the game to some extent determines the interactive way. And the game's interaction is determined by different sensors. Lastly, we have to consider whether the user experience is enjoyable or not and whether the interactive way of the game is natural or not.", com tradução do autor.

3 DESENVOLVIMENTO

Neste capítulo relatamos o planejamento e codificação de um motor de jogos para microcontroladores, inicialmente oferecendo suporte aos consoles ODROID-GO e LVNDR-01 – apresentados no capítulo 2. São descritos os ambiente de desenvolvimento e o de execução. É documentada a implementação de dois jogos conceituais, mensurando e validando a viabilidade da ferramenta construída. Por fim, o resultado é comentado e são sugeridos trabalhos futuros.

3.1 Arquitetura do motor de jogos

Projetado para continuar crescendo em funcionalidades e suporte a aparelhos, o motor de jogos proposto — ilustrado na figura 27 — cria um ecossistema que contempla quatro frentes de desenvolvimento e aprimoramento. São elas:

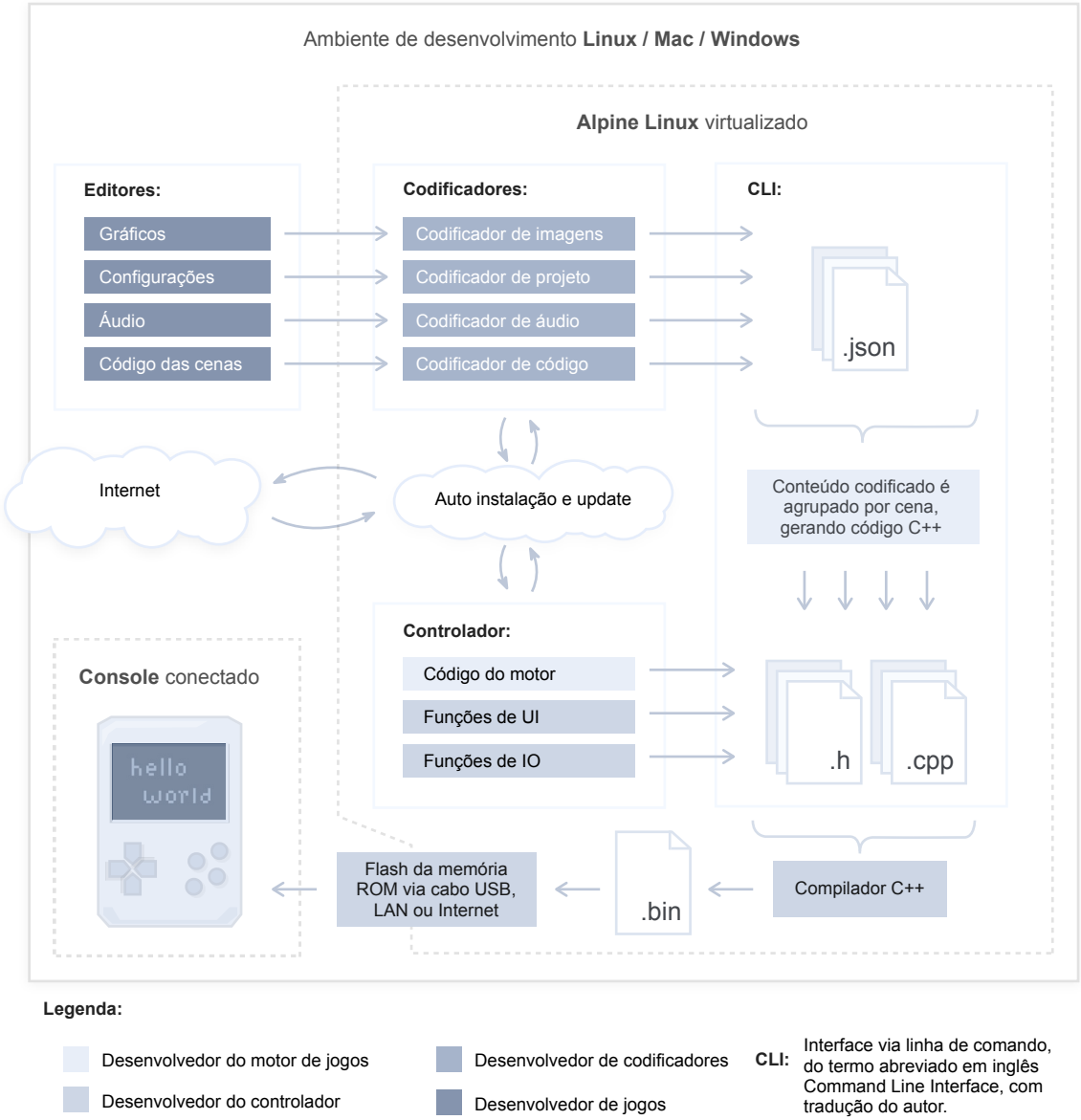
1. Controladores de dispositivo, que viabilizam o suporte aos modelos diferentes de video games. São detalhados na seção 3.2. De maneira geral, fazem a ponte entre o motor de jogos e o hardware. Importante peça na arquitetura, pois novos microcontroladores são lançados constantemente — como visto na subseção 2.4.2.

   Além disso, melhorias nesta área levam a, por exemplo: imagens mais nítidas e estáveis, configurando o curva gama do display e implementando sincronismo vertical; resposta mais rápida dos controles, refinando o algoritmo que monitora o estado dos botões; ganho de agilidade no carregamento do jogo, aprimorando a alocação de memória.

2. Codificadores de multimídia. São programas invocados durante a compilação do projeto para transformar arquivos de multimídia em arquivos de um formato intermediário, já contendo os trechos de código C++ que serão processados pelo microcontrolador em tempo de execução. Por exemplo, um codificador de imagens PNG deve receber um arquivo em seu formato natural, que pode ter sido gerado em qualquer editor da preferência do desenvolvedor de jogos, e transformá-lo em um trecho de C++ declarando uma matriz de cores indexadas, que é o formato aceito para desenho de imagem bitmap pelo motor de jogos. Codificadores são detalhado na seção 3.4.

   Assim como novos controladores adicionam novos aparelhos de video game, novos codificadores adicionam suporte a novos formatos de arquivo. Quando um programador de codificador escreve, por exemplo, um programa que lê arquivos de planilhas e

Figura 27 – Arquitetura para compilação multiplataforma.



Fonte: autoria própria.

transforma em estruturas de dados C++, toda comunidade se beneficia e jogos mais interessantes podem ser criados.

3. Bibliotecas do motor de jogos. Para orquestrar tanto a fase de desenvolvimento, quanto a compilação e execução, o motor de jogos utiliza vários componentes de software.

   Por ser o intermediário entre os demais componentes do ecossistema, uma das tarefas desta frente é manter os demais sistemas informados. A gestão de mensagens, por exemplo, é feita por uma biblioteca do motor. Ela avisa o programador de jogos se

algum problema ocorrer com o código, por exemplo, ou se algum codificador está sendo atualizado.

A interface do usuário para o editor de jogos — que ajuda a organizar os arquivos multimídia em cenas, monitora a conexão com o video game e dá suporte a compilação e execução do projeto de jogo — também é implementada pelos desenvolvedores das bibliotecas do motor.

Aqui também são declaradas as funções que o programador de jogos utiliza-rá para implementar seu jogo. É importante que sejam abstratas o suficiente para que novos consoles possam ser adicionados. Por exemplo, uma função que preenche a tela com uma cor sólida precisa considerar que o tamanho da tela será decidido pelo controlador e os dois precisam utilizar a mesma tabela de cores indexadas.

A seção 3.3 detalha a função e relata a implementação destes códigos.

4. Por fim, os jogos. Como dito na subseção 2.2.1 — um motor apenas dá suporte ao desenvolvedor de jogos, oferecendo ferramentas para facilitar tarefas rotineiras na criação de seu jogo. O sucesso deste projeto será determinado pela qualidade e quantidade de jogos produzidos utilizando a ferramenta.

   Na seção 3.6 demonstramos a capacidade do motor implementando dois jogos conceituais.

3.2 Controladores

O código do controlador, em inglês, *driver*, é visto neste trabalho como a parte responsável por fazer a interface com o *hardware* de um console de videogames. Cada aparelho pode ter telas diferentes ou maneiras distintas de receber comandos do jogador, entre outras particularidades, por isso necessita de um controlador que consiga comunicar-se com seus periféricos quando executado. Citando Corbet, Rubini e Kroah-Hartman (2010), em resumo, controladores são caixas pretas que fazem uma parte particular do *hardware* responder a uma interface de programação bem definida, escondendo completamente os detalhes de como essa peça funciona[1].

Com a diversidade de dispositivos em mente, o motor de jogos foi planejado desde o começo para suportar diferentes controladores. Isso viabiliza a capacidade de compilar um mesmo código de jogo para vários consoles e é desejável porque aumenta a vida útil do projeto, que passa a ser capaz de suportar aparelhos que ainda nem foram lançados, bastando que futuramente um controlador para ele seja escrito.

[1] "They are distinct "black boxes" that make a particular piece of hardware respond to a well-defined internal programming interface; they hide completely the details of how the device works.", com tradução do autor.

Uma implicação negativa desta escolha é a de que, para tanto, torna-se necessária a instalação de uma porção de softwares e bibliotecas de apoio a compilação do projeto para cada console que se deseja suportar, tudo isso na máquina do desenvolvedor de jogos. Apenas nos dois aparelhos de referência utilizados neste trabalho já notamos falta de padrões. Para compilação e execução de um programa, o Particle Photon usa uma interface via linha de comando, chamada Particle CLI, enquanto o ODROID utiliza um programa Python, chamado ESP32 IDF. Mesmo que seja feito um esforço no sentido de normalização, o processador do ODROID executa instruções XTensa, enquanto o Particle Photon executa instruções ARM. São fundamentalmente distintos. Seria pouco produtivo ignorar as ferramentas e compiladores prontos de cada fabricante e tentar escrever algo homogêneo por conta própria.

Para solucionar este problema, das dependências especificas, e ainda assim poder tirar proveito dos kits de desenvolvimento fornecidos pelas fabricantes, utilizamos o recurso de virtualização de sistema operacional . Isso foi feito aproveitando o conceito de contêineres Linux como infraestrutura de compilação. Essa tecnologia de contêineres, tecnicamente chamada de LXC, permite virtualizar um sistema Linux em uma máquina hospedeira sem a necessidade de alocar uma máquina virtual para cada um dos sistemas operacionais hospedados, como explica McKendrick (2020).

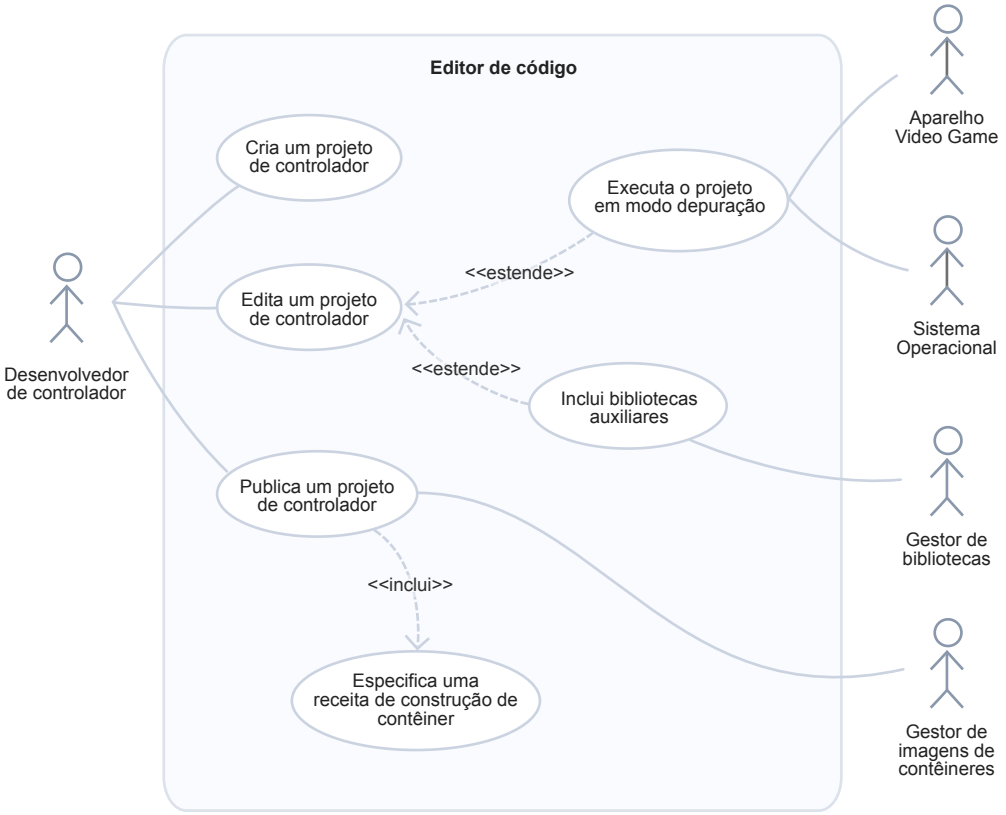
3.2.1 Infraestrutura de compilação

Cada código-fonte de controlador é desenvolvido previamente e está embarcado dentro de uma imagem de contêiner própria, baseada em um Alpine Linux, junto de todas ferramentas necessárias para compilar o jogo na plataforma que o controlador oferece suporte. Isso gera confiança de que o executável do projeto será construído com sucesso independente da máquina hospedeira do desenvolvedor e que não será preciso nem instalar, nem configurar software adicional para criar e executar um jogo, visto que a imagem virtualizada Linux de cada videogame já possui todo software necessário instalado. A Figura 28 ilustra os casos de uso envolvidos na construção e manutenção de um controlador de dispositivo.

Para tornar o editor do motor de jogos multiplataforma, capaz de executar em máquinas Windows, Linux e Mac, foi utilizada a infraestrutura do Docker. Com ele é possível executar múltiplas instâncias LXC em apenas uma máquina virtual, caso o sistema hospedeiro não seja um Linux, ou até sem máquina virtual alguma quando à partir de um Linux, McKendrick (2020) comenta.

A escolha pelo Docker deu-se considerando uma recente pesquisa de satisfação

Figura 28 – Diagrama UML de Casos de Uso para o desenvolvedor de controlador



Fonte: autoria própria.

conduzida pelo site Stack Overflow[2], que aponta Linux e Docker como, respectivamente, primeira e segunda plataforma mais amada pelos desenvolvedores em 2020. Como todo código deste projeto aqui relatado é gratuito e aberto, nos termos da licença GPL v2.0, qualquer desenvolvedor interessado pode colaborar criando controladores para outros videogames, estudando os controladores existentes e implementando melhorarias. Por esse motivo, optamos por plataformas conhecidas e desejadas pela comunidade de desenvolvimento.

As imagens dos contêineres ficam hospedadas em uma plataforma na nuvem chamada Docker Hub, por ser a plataforma oficial da empresa que supervisiona o projeto Docker e gratuita para projetos de código aberto. Durante a criação de um jogo, no editor do motor, o desenvolvedor de jogos aponta quais contêineres quer usar para executar seu projeto. O editor então comunica-se localmente com o Docker, que faz download da imagem, caso seja preciso, e inicializa um contêiner, deixando-o pronto para dar suporte ao projeto. Assim é feita a adição de novos consoles.

Qualquer editor de código pode ser utilizado durante a construção do controlador,

[2] Disponível online em https://insights.stackoverflow.com/survey/2020. Acessado em 2/11/2020.

desde que consiga se comunicar com o hardware, gere um projeto compatível com Linux e automatizável dentro de uma receita de contêiner Docker. Algumas fabricantes de microcontroladores fornecem ambientes próprios, como o Microchip MPLAB Harmony, que facilitam a depuração e configuração de bibliotecas. É comum também entre desenvolvedores da comunidade maker o uso da IDE Arduino — apresentada na subseção 2.4.1 — que tem suporte a Linux e pode ser invocada via linha de comando em um terminal. Durante o desenvolvimento deste trabalho optou-se pelo programa Visual Studio Code, da Microsoft, por conta da sua integração com Docker e também por ser gratuito.

Por fim, dentro do código especificamente, o programa criado pelo desenvolvedor de controlador precisa ser sobretudo capaz de receber instruções do motor de jogos e executá-las. Por serem componentes desenvolvidos separadamente, um protocolo precisa ser estabelecido. Isso é feito por meio de funções externas da linguagem C, similares a interfaces em Java ou funções puramente virtuais em C++.

3.2.2 Funções externas do motor de jogos

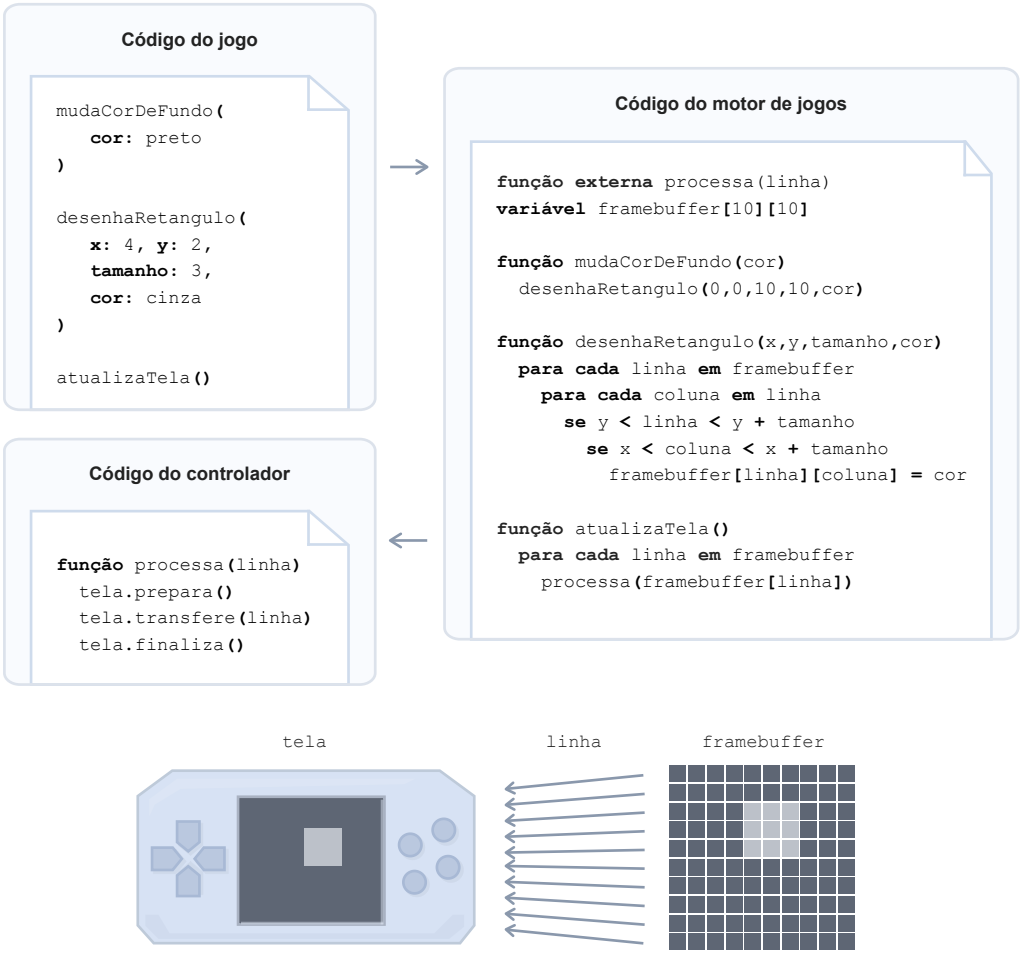
Por meio de declaração de funções externas, o motor consegue lidar com o fato de que algumas implementações serão sintetizadas apenas na compilação, já pelo controlador. De maneira similar, o controlador precisa apenas incluir a biblioteca do motor e concretizar poucas funções para suportar os jogos, sem precisar saber exatamente como elas serão utilizadas ou quando serão chamadas. A Figura 29 ilustra esse processo.

Para alcançar o objetivo proposto para este trabalho, apenas duas funções externas foram necessárias. A primeira delas enviando instruções de desenho para tela e a segunda recebendo o estado dos botões. A mesma estratégia de delegar ao controlador a concretização de função externas pode ser utilizada para adicionar mais capacidades ao motor, como sintetizar áudio ou suportar escrita e leitura de dados em cartões de memória.

3.2.3 Interface para desenho da tela

Para o motor de jogos, gerenciar a tela significa manter atualizada em memória RAM uma matriz de tamanho equivalente a resolução do aparelho de videogame, técnica conhecida como *framebuffer* (GREGORY, 2018). Nesta matriz, cada elemento representa a cor de um pixel da tela naquele instante. Por exemplo, uma tela de cristal líquido de 130 pixels de largura por 130 pixels de altura, no motor de jogos é vista como uma matriz de tamanho 130x130, totalizando 16.900 elementos. Durante a execução, o motor vai receber instruções do código do jogo para aplicar gráficos bitmap em partes desta matriz e gerar um novo quadro, o que idealmente acontece 30 vezes por segundo, dando a impressão de animação.

Figura 29 – Exemplo do funcionamento de uma função externa definida no motor e implementada no controlador.



Fonte: autoria própria.

Gregory (2018) avança no assunto, complementando que nos jogos de computador, acelerados por placas de video, são utilizadas de duas a três matrizes ao mesmo tempo para gerar um fluxo de trabalho constante – técnicas conhecidas como double e triple buffering, respectivamente. Enquanto uma matriz está sendo transportada para a tela pelo processador gráfico, o motor de jogos já começa a processar o próximo quadro. Contudo, quanto mais matrizes, mais memória RAM é consumida, uma desvantagem desta técnica.

Neste trabalho utilizamos apenas uma matriz. Dadas as semelhanças de hardware, optamos por seguir uma estratégia similar a de como faziam os consoles NES e Super NES, da Nintendo, que também eram limitados em relação à quantidade de memória disponível para gráficos e paralelismo de processamento. Enquanto a tela é atualizada com os dados do framebuffer, não se pode alterar seu conteúdo. É preciso esperar que a transferência termine. Apenas os espaços entre quadros, chamado de vsync, ou entre linhas, hsync,

podem ser utilizados para processar o jogo de maneira que altere o framebuffer, como elucida Hugg (2019).

Para se ter uma ideia da escala desta limitação de memória, o microcontrolador utilizado pelo protótipo LVNDR-01, visto neste trabalho, tem apenas 0.0016% do que se tem de RAM em um console de vídeo games moderno, o Playstation 4, da Sony (GREGORY, 2018). São 128 Kilobytes contra 8 Gigabytes. Já um Nintendo Super NES, por sua vez, conta com 128 Kilobytes de memória de uso geral e mais 64 Kilobytes de video, para armazenar o framebuffer. Por isso, em termos de hardware, este trabalho se aproxima muito mais de um Super NES do que de um Playstation 4.

3.2.4 Escolha do formato de cores no framebuffer

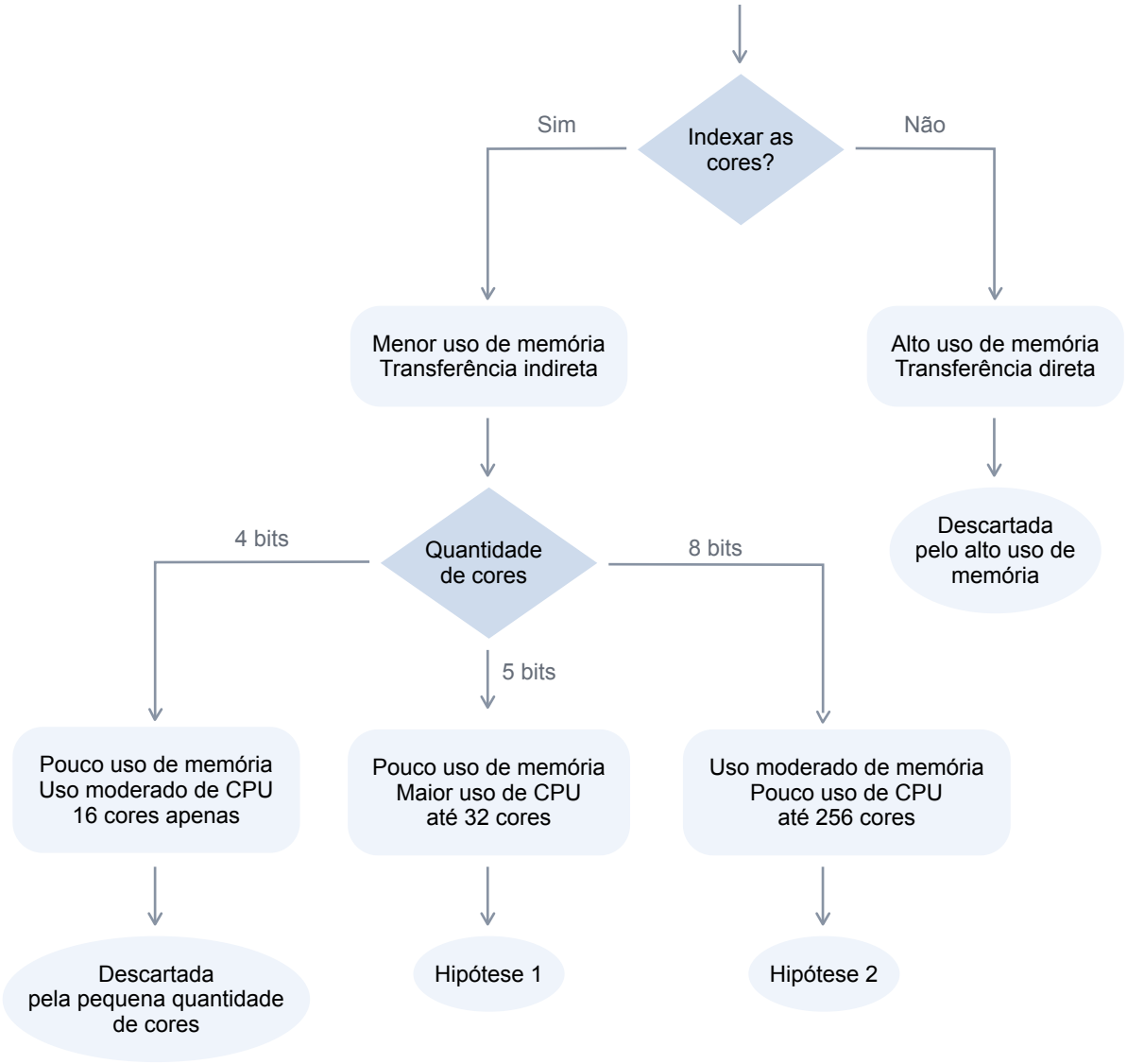
Um experimento foi feito para escolher o melhor formato da representação da cor de cada pixel dentro do framebuffer, considerando as limitações dos aparelhos estudados neste trabalho. As hipóteses foram levantadas seguindo os critérios a seguir e ilustrados no fluxograma da Figura 30.

Num primeiro momento, podemos considerar armazenar as cores do framebuffer já no formato esperado pela tela, tipicamente como cores RGB de 16 bits, ou numa representação intermediária, utilizando uma tabela de apoio para indexar as cores. A primeira opção viabiliza transferências diretas de memória do framebuffer para a tela, porém consome grande quantidade de memória RAM e foi descartada. Cada pixel necessitaria de 2 bytes, representando um consumo de 33,8 Kilobytes em uma tela de 130 por 130 pixels. Para um microcontrolador de 128 Kilobytes de memória RAM, somente o framebuffer alocaria mais de $\frac{1}{4}$ do espaço. Optou-se então pela utilização de cores indexadas no framebuffer – técnica vista no sub-capítulo 2.3.1.

A quantidade de cores simultâneas depende de quantos bits se utiliza para indexar as entradas na tabela de cores. Quanto mais bits, mais cores possíveis e maior o consumo de memória RAM pelo framebuffer. É preciso levar em conta também o alinhamento de memória durante o acesso pelo processador. O endereçamento é feito de byte em byte e utilizar, por exemplo, 5 bits para representar cada cor significa operações de máscara de bits em tempo de execução. Ciente dos fatores envolvidos, três opções foram levantadas: utilizar 4, 5, ou 8 bits (Figura 31). A primeira alternativa porém, apesar de facilitar o acesso a memória por medir exatamente meio byte, foi prontamente descartada por conseguir representar apenas 16 cores, limitação que vai além das concessões planejadas para esse trabalho. Ficamos então com duas hipóteses:

- Hipótese 1: economizar memória por utilizar 5 bits para representar cada pixel,

Figura 30 – Fluxograma do critério de decisão das hipóteses na escolha do formato de representação das cores



Fonte: autoria própria.

resultando em 32 cores possíveis. Acesso desalinhado e feito utilizando estruturas de C e máscara de bits. Esta opção é a mais desejada por conseguir um equilíbrio entre quantidade de cores e uso de memória, se o consumo extra de processamento não for impeditivo.

- Hipótese 2: ganhar performance as custas de consumo de memória, utilizando 1 byte para representar cada pixel. Manter a memória alinhada, facilitando o acesso direto a elementos do framebuffer. 256 cores podem ser representadas simultaneamente. Essa opção será considerada caso a Hipótese 1 falhe, já que ainda significa economia de memória se comparada com não indexar as cores.

Figura 31 – Opções de alinhamento de memória na matriz framebuffer.

Fonte: autoria própria.

A metodologia utilizada para estudar as hipóteses foi implementar o código de cada uma delas, verificar o funcionamento e medir a capacidade de desenho de pixels a uma taxa de atualização de 30 quadros por segundo, 33 milissegundos por quadro. O aparelho utilizado para as medições foi um microcontrolador STM32F205RGY6 contendo um processador ARM Cortex M3 à 120Mhz em uma placa Particle Photon, ligada a um display TFT LCD 130x130 com um controlador Solomon Systech SSD1283A, via interface SPI com clock de 80Mhz.

Os resultados obtidos com a implementação da hipótese 1 foram medidos e confirmaram a capacidade da solução quanto a economia de memória. Apenas 11,05 Kilobytes foram necessários para armazenar os 16.384 pixels da tela de 130x130 pixels, suportando 32 cores, versus os 33,8 kilobytes de um framebuffer sem indexação de cores. Contudo, a demanda por processamento foi elevada drasticamente por conta do arranjo na memória. O tempo necessário para acessar a RAM desalinhada fez com que desenhar apenas uma imagem do tamanho de toda tela, ou seja, 130 por 130 pixels, já tomasse mais tempo do que os milissegundos disponíveis para desenhar um quadro. Por ter reduzido a frequência de atualização da tela para abaixo de 30 quadros por segundo, concluímos sobre a

inviabilidade da estratégia para uma solução de jogos.

Já a hipótese 2, quando implementada, comportou-se como esperado. O uso de memória cresceu, comparando-a com a hipótese 1, porém o acesso alinhado da memória facilitou o processamento e a transferência de imagens para o framebuffer. Conseguimos alcançar 65.536 pixels desenhados em menos de 33ms, mais que o suficiente para suportar os jogos propostos neste trabalho. Optamos por este algoritmo, porém é importante observar que a arquitetura dos registradores do processador tem uma grande influência aqui. Em um console de registradores de 8 bits, por exemplo, seria preciso repetir o experimento para encontrar o melhor equilíbrio entre quantidade de cores e tempo de acesso à memória.

3.2.5 Implementação da interface com a tela no ODROID

A transferência do framebuffer para a tela do ODROID foi implementada em seu controlador utilizando transações SPI[3] em conjunto com um circuito DMA[4]. A atualização da tela é sincronizada pelo gerenciador de tarefas do kernel de sistema operacional de tempo real utilizado pelo ODROID, o FreeRTOS. Estes processos serão detalhados à seguir.

Dentro de um microcontrolador contemporâneo, a memória Flash, RAM e o processador estão contidos no mesmo encapsulamento e trocam informações por meio de barramentos internos (YIU, 2015). Quando o código está sendo executado, porém, o microcontrolador precisa comunicar-se com dispositivos externos – fora do encapsulamento – como uma tela ou um controle de videogame. Isso pode ser feito por meio de um protocolo eletrônico e digital. O microcontrolador cria linhas de comunicação que alcançam fisicamente seus periféricos através de entradas e saídas que monitoram e alteram a voltagem nas trilhas da placa de circuito impresso.

Existem protocolos paralelos e seriais – suas diferenças são exemplificados na Figura 32. No primeiro, vários bits trafegam simultaneamente por meio de linhas paralelas. Já no segundo, a informação é enviada por uma via ao longo do tempo, em série.

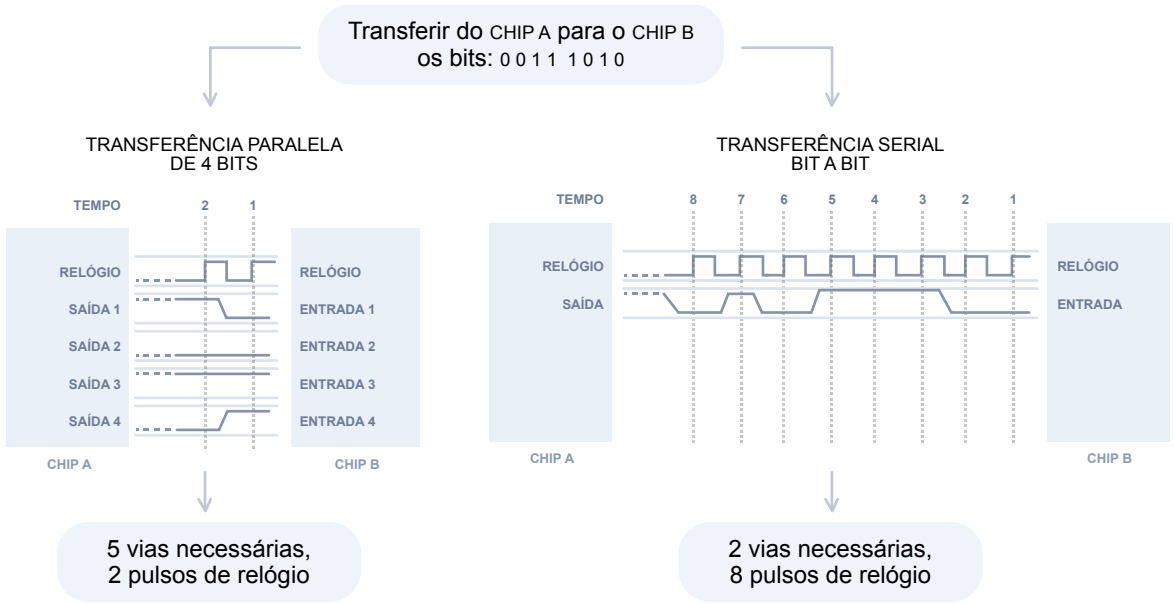
Na família dos protocolos seriais, alguns utilizam apenas uma via para entrada e saída de dados. Citando um exemplo, no protocolo I2C[5], comandos de sincronização e endereçamento são utilizados na via para definir qual chip envia ou recebe sinais em um dado momento, explica Simmonds (2017). Ainda na família dos protocolos seriais, segundo

[3] Abreviação do termo em inglês para Interface Serial de Periféricos, “Serial Peripheral Interface”, com tradução do autor.

[4] “Abreviação do termo em inglês para Acesso Direto à Memória, “Direct Memory Access”, com tradução do autor.

[5] Abreviação do termo em inglês para Integração Intra Circuito, “Inter-Integrated Circuit” com tradução do autor.

Figura 32 – Exemplificação de transações digitais eletrônicas paralelas e seriais.



Fonte: autoria própria.

Amos (2020), existem também os protocolos sem uma via dedicada a sincronização, como o UART[6]. Em suas palavras, o dispositivo UART pega bytes de dados e os transmite por meio de um fio, modulando a tensão de uma linha de sinal a uma taxa pre-determinada[7]. Isso significa que a velocidade da transmissão da informação serial é acordada previamente entre os dois chips.

Especificamente sobre o protocolo utilizado pelas telas dos dois aparelhos estudados, chamado SPI, Simmonds (2017) nos diz que refere-se a uma comunicação serial que utiliza quatro ou mais vias: duas delas para o envio e recebimento de dados, chamadas respectivamente de MOSI e MISO; uma utilizada especificamente para um pulso de sincronia, chamada CLK; por fim, uma ou mais vias para a seleção de chip, nomeadas CS no periférico. Pode ser vista ainda como sendo um meio de comunicação inter-processador altamente determinístico[8], conclui Amos (2020). Na interpretação do autor deste trabalho, o determinismo referido ocorre por se ter no protocolo SPI um pulso de sincronia isolado em uma via e o dado em outra, o que previne defasagens e erros de leitura da informação entre os chips.

No microcontrolador do ODROID-GO, as transações SPI podem ter sua velocidade

[6] Abreviação do termo em inglês para Receptor/Transmissor Assíncrono Universal, "Universal Asynchronous Receiver/Transmitter" com tradução do autor.

[7] "UART hardware takes bytes of data and transmits them over a wire by modulating the voltage of a signal line at a predetermined rate", com tradução do autor

[8] "highly deterministic inter-processor communication medium", com tradução do autor.

de sincronia configurada por software. Quanto maior a velocidade, maior a taxa de transferência e mais rapidamente o processador finaliza a passagem do framebuffer. Utilizamos 80MHz, que é o máximo disponível.

Igualmente importante, um circuito do ESP32 chamado DMA – sigla em inglês para Acesso Direto à Memória – oferece suporte a transferência SPI de alta velocidade. Por meio de um barramento, o DMA esta internamente conectado tanto a memória RAM, quanto a interface SPI. Quando configurado pelo processador, ele consegue executar a transferência dos bytes que estão na memória RAM diretamente para a interface SPI, que, por sua vez, controla a tela. Em seu manual oficial, a Espressif Systems (2018) diz que desta maneira é possível maximizar o desempenho e dá instruções como proceder. Vale observar porém que esta configuração que prepara a transferência DMA tem um custo, Amos (2020) sinaliza que:

> (...) leva algum tempo para configurar a transferência DMA inicialmente; portanto, para pequenas transferências, realmente pode-se levar mais tempo de CPU para configurar a transferência do que se uma interrupção ou método de verificação de estado fosse usado.

Tendo em mente o exposto por Amos (2020), na implementação do controlador foi feito um acumulador para receber várias linhas do motor de jogos antes de realizar a transação SPI via DMA. Este valor é configurável. Quanto maior, mais rapidamente é transferido o framebuffer, mas também mais memória RAM é utilizada.

Por fim, é preciso que o programa do controlador faça a atualização da tela em uma frequência idealmente constante para que aconteça o efeito de animação – conceito visto na subsecão 2.2.6. Para tanto, foram utilizados mecanismos de tarefas e temporizadores do sistema operacional oficial do ESP32, o FreeRTOS. No início do programa – quando o video game é iniciado – uma tarefa responsável por atualizar o jogo é criada. Ela contém um laço que roda infinitamente. Dentro dele acontecem – em conjunto com o motor de jogos – a captura do estado dos controles, a atualização da cena atual e o desenho do quadro na tela. A última instrução do laço invoca o sistema operacional pedindo para que suspenda sua própria tarefa. Também na inicialização do video game, um evento recorrente é agendado em um temporizador do sistema, que libera a tarefa de atualização da tela a cada 33 milissegundos. Os dois mecanismos em conjunto criam a repetição principal do motor de jogo.

Esta definição de arquitetura foi atingida após o aprendizado adquirido em uma implementação anterior, que foi uma tentativa frustrada de solução minimalista. Inicialmente

apenas o temporizador era utilizado, agendando no sistema operacional a chamada de função que gerava o quadro a cada 33 milissegundos, mas há um problema com esta abordagem: quando a função que gera o quadro leva mais do que 33 milissegundos para executar, o temporizador dispara e o sistema precisa lidar com trocas de contexto. Eventualmente o jogo era interrompido devido a um estouro de pilha. Na solução atual este problema foi resolvido com o uso do sistema de tarefas. O temporizador apenas resume a tarefa que gera o quadro, caso ela esteja suspensa. Do contrário, caso ela atrase e ainda esteja sendo executada, resumir a tarefa simplesmente não tem nenhum efeito. A geração do quadro é finalizada e o sistema coloca a atividade em modo suspenso, até que o temporizador seja novamente acionado.

3.2.6 Implementação da interface com a tela no Protótipo LVNDR-01

A implementação da interface com a tela no protótipo LVNDR-01 acontece de maneira muito similar a do ODROID-GO – que foi detalhada na subseção anterior. A seguir são destacadas as semelhanças e diferenças.

Via protocolo SPI, novamente são feitas as transferências do microcontrolador para a tela, como no ODROID. O barramento, contudo, é dedicado e adiciona uma via extra para especificar se a transação atual leva comandos ou dados, chamada D/C. Configuramos a velocidade para a máxima suportada no Photon, de 60MHz. Um diagrama de funcionamento pode ser visto na Figura 24.

Seu processador, um ARM Cortex M3, igualmente suporta DMA. Inicialmente a implementação não o utilizava e transacionava pixel a pixel trazidos da RAM para CPU e dali para a SPI. Atingimos rapidamente o limite de processamento e a taxa de quadros por segundo começou a cair. Ao utilizar o DMA percebemos aumento de performance.

Mensuramos 10 transações de uma linha de framebuffer com e sem uso de DMA para anotar o ganho de performance. A primeira estratégia tomou 462,4 microssegundos, numa média aritmética, com desvio padrão de 0.4. Já na segunda, que utiliza DMA, o tempo gasto foi, em média aritmética, de 81,4 microssegundos, com desvio padrão de 1,77[9]. Com esse resultado, recomenda-se que controladores de novos consoles explorem o DMA para aliviar o consumo de CPU.

Quanto ao sincronismo, no LVNDR-01 utilizamos também um temporizador. Contudo foi preciso configurar um relógio de hardware e interrupções de baixo nível. Visto que ele não possui um kernel de sistema operacional, o que foi feito no ODROID-GO com

[9] O experimento pode ser reproduzido com o código armazenado no repositório online no endereço https://github.com/lv-e/driver-photon, ativando a variável *lvk_measuring* e manipulando a configuração *USE_SPI_W_DMA*.

gestão de tarefa, no LVNDR-01 foi feito com uma variável booleana volátil de C, que sinaliza a liberação para o desenho do próximo quadro. A desvantagem deste método é que o código fica preso em um laço infinito, que utiliza toda CPU para inspecionar a variável booleana indefinidamente, aguardando a liberação para desenhar o próximo quadro que virá por meio da interrupção. Isso não causa efeito de perca de quadros, nem impacta a jogabilidade, apenas não lida bem com o consumo energético – tema que será abordado na seção de trabalhos futuros.

3.2.7 Interface para os botões físicos

Video games são, por definição, interativos – como visto na seção 2.1 e defendido por Gregory (2018). Os consoles criam estas experiências de interatividade por meio de dispositivos de interface humana, matéria abordada em 2.2.3. Nos aparelhos estudados, isso é feito por meio de botões.

Se nos gráficos a transferência do framebuffer ocorre do motor de jogos para uma conexão externa, a tela, aqui o caminho inverso é percorrido. O estado dos botões é capturado pelos controladores de dispositivo e enviado ao motor de jogos. Nas próximas subseções, 3.2.8 e 3.2.9, serão descritas as implementações específicas de cada um.

3.2.8 Implementação da interface com os botões no ODROID

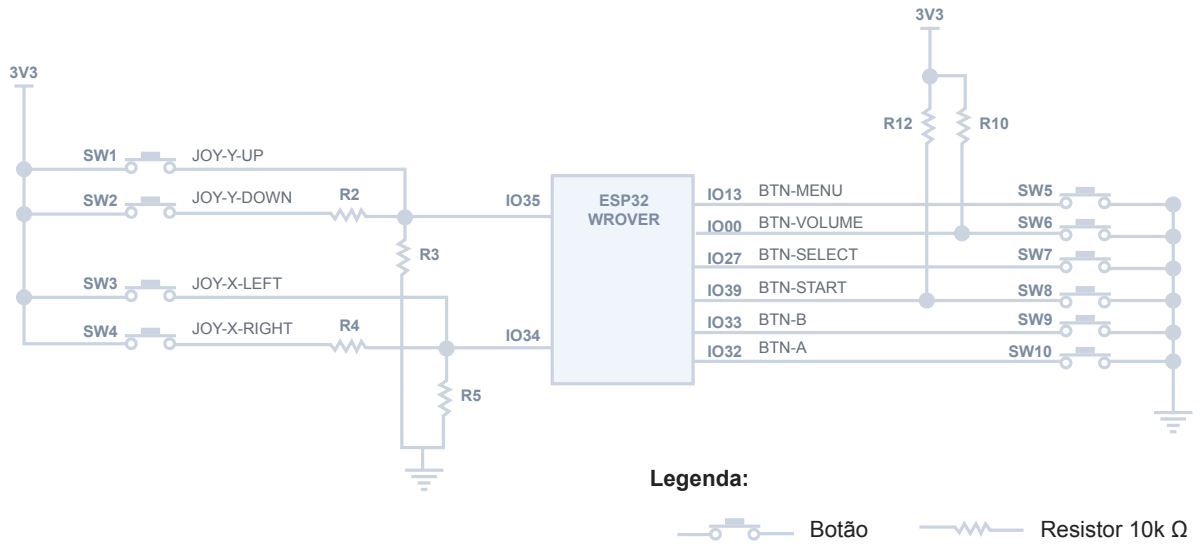
Microcontroladores oferecem um grupo de conexões externas conhecido como GPIO – assunto visto no capítulo 2. No ODROID, interligam o ESP32 a botões táteis, posicionados em sua placa de circuito impresso. Um diagrama pode ser visto na Figura 34.

A leitura dos botões é trivial, pois a biblioteca oficial do microcontrolador oferece uma função para captura do sinal digital ou analógico presente na porta GPIO. É preciso apenas configurar previamente todas elas como portas de entrada.

Os botões de Menu, Volume, Select, Start, A e B são portas digitais. Quando o botão está pressionado, o nível lógico é baixo – um inteiro com valor 0 no código. Quando solto, o nível é alto, 1, no código. A função externa declarada no motor de jogos espera um valor zero para botões que não estejam pressionados. Para implementar essa interface foi preciso apenas inverter o valor capturado.

Já os botões direcionais são implementados com um divisor de tensão. Na prática, não existe apenas um estado lógico, alto ou baixo. É preciso enquadrar a medição dentro de uma faixa de voltagem. Os quatro direcionais são divididos em dois grupos, horizontal e vertical, cada um conectado a uma porta de conversor analógico digital do ESP32. Este arranjo demandou um cuidado extra. Inicialmente o motor de jogos recebia esporadicamente

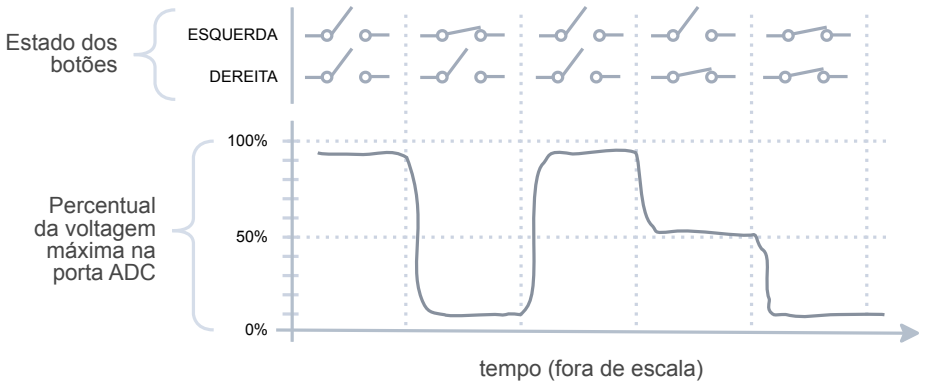
Figura 33 – Circuito dos botões no video game ODROID-GO.



Fonte: Hard Kernel Co. Ltd, com ajustes do autor.

falso positivo de que o botão para direita havia sido pressionado, quando, na verdade, era apenas a transição de voltagem – que passava por 50%, enquanto caminhava para próximo de zero – quando o botão da esquerda era pressionado.

Figura 34 – Exemplo do divisor de tensão dos botões direcionais do ODROID-GO.



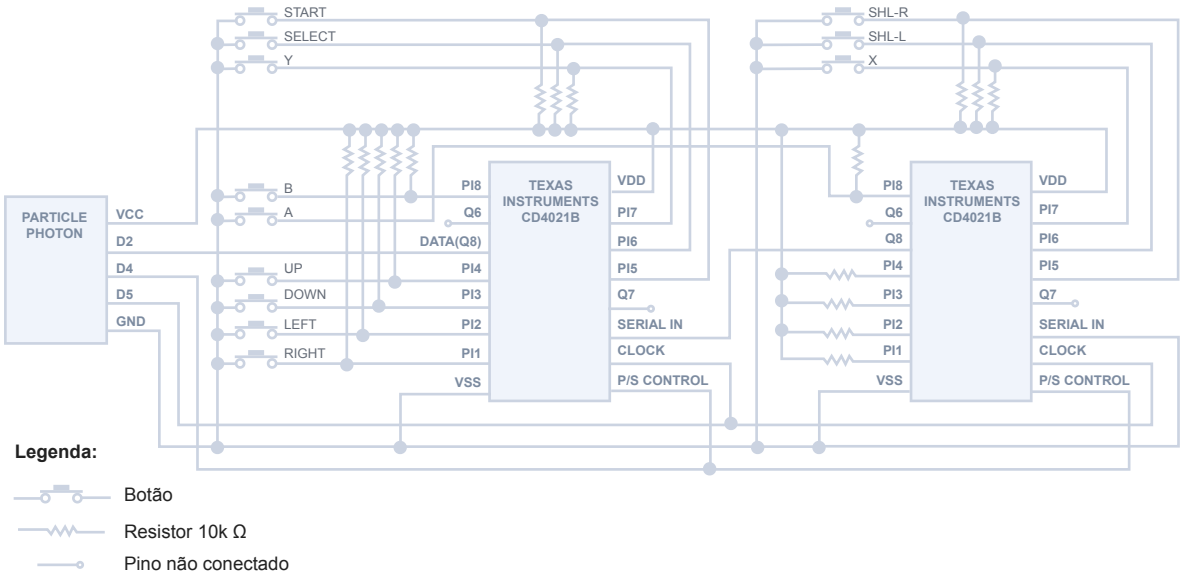
Fonte: autoria própria.

Para mitigar este efeito colateral da arquitetura, utilizamos três medições seguidas espaçadas de 4 microssegundos, utilizando a média aritmética para decidir sobre os estados dos botões.

3.2.9 Implementação da interface com os botões no Protótipo LVNDR-01

Durante a construção do protótipo, foram avaliadas três possibilidades de dispositivos de interface humana – assunto relatado na subseção 2.4.5. Optamos pela interface serial utilizada no NES e SNES, video games da Nintendo. Desta maneira é possível tanto aproveitar um modelo pronto de controle quanto construir um. A Figura 35 ilustra a esquemática de um circuito compatível com o controlador implementado. Esta família de chips, dos registradores de deslocamento, tem valor acessível e são facilmente encontrados. Durante a escrita deste trabalho encontramos modelos por menos de R$ 3,00[10].

Figura 35 – Circuito dos botões no protótipo LVNDR-01.



Fonte: Santiago (2009), com ajustes do autor.

O algoritmo para leitura dos botões é simples e feito de maneira serial. São dois passos. O primeiro dá um pulso de 16 microssegundos na linha P/S CONTROL. Isso faz com que internamente o estado de todas as portas paralelas seja capturado no controle. Nos segundo passo, um laço faz um pulso na via CLK por 8 microssegundos, seguido da leitura na porta de dados. Isso é repetido até que todos os 12 botões sejam verificados.

Chegamos neste algoritmo e intervalos de tempo analisando e traduzindo a versão em linguagem assembly 6502 da mesma lógica, escrita por Hugg (2019) em seu livro sobre desenvolvimento de jogos para o Nintendo NES – que mantém-se igual no controle utilizado, de Super NES, onde a fabricante apenas elevou o número de bits de 8 para 12

[10] Modelo CD4021 com encapsulamento DIP16. Encontrado nas lojas: New Port Componentes Eletrônicos, R$3,00; Baú da Eletrônica, R$2,10; Proesi, R$ 2,09. Valores para novembro de 2020.

no registrador de deslocamento.

3.3 Bibliotecas do motor de jogos

Entre o código do controlador de dispositivos, dos codificadores de multimídia e o do jogo estão as ferramentas do motor de jogos – interligando os três e criando níveis de abstração. Nesta seção estes componentes que ditam a estrutura do ecossistema serão descritos.

3.3.1 Interfaces públicas da aplicação

Ao longo do desenvolvimento de um jogo para video game, os programadores implementam sua lógica escrevendo código-fonte. Criam, por exemplo, máquinas de estado para os inimigos, algoritmos para simular partículas e criar efeitos especiais, controles de vidas disponíveis, inventário do jogador e assim por diante. São códigos específicos de cada jogo. Algumas funções, todavia, podem ser fornecidas pelo motor de jogos – que está mais próximo ao hardware do video game.

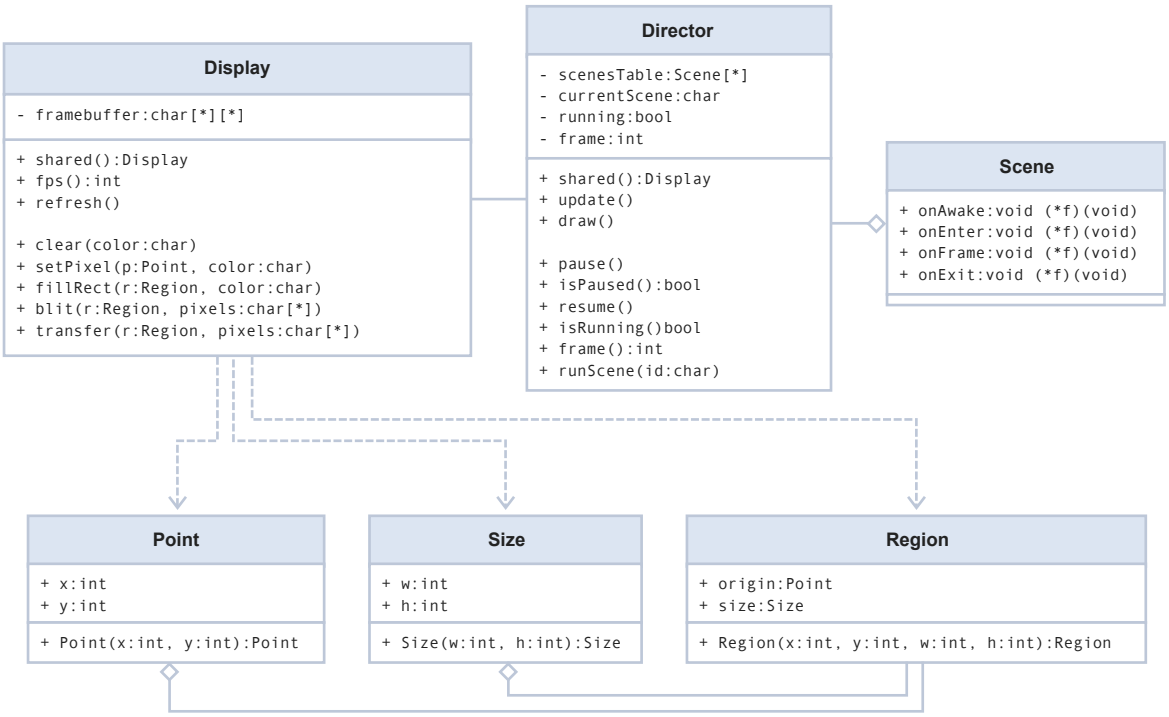
Como saber se o jogador apertou o botão "Start" para que o jogo possa começar? Ou então, depois de decidida a posição de um inimigo, como desenhá-lo na tela? Por outro lado, como um motor pode oferecer para o programador de jogos suporte para múltiplas plataformas se em uma delas os botões são feitos com portas GPIO e em outro um protocolo SPI para um controle periférico é utilizado?

Para resolver este problema o motor de jogos fornece uma biblioteca que é incluída por padrão nos arquivos de código-fonte quando na etapa de compilação de um jogo. São interfaces públicas da aplicação, populares pelo termo abreviado em inglês, API. Por meio delas o jogador pode simplesmente perguntar ao motor: o botão "Start" do primeiro jogador acabou de ser pressionado? Como todos os jogos vão utilizar a mesma função, o motor se preocupa em resolver dali em diante, por exemplo, por meio de declarações de funções externas, que serão implementadas nos controladores de dispositivos – técnica vista na subseção 3.2.2.

Neste trabalho, as interfaces públicas estão agrupadas em dois temas. Um, chamado "UI", concentra as funções atuais e futuras relacionadas a exibição e framebuffer. No outro, ficam as demais funções de entrada e saída, chamado "IO". São detalhados individualmente adiante.

"UI" é abreviação de "User Interface", termo em inglês para "Interface do Usuário". A Figura 36 mostra um diagrama de classes com os componentes desse agrupamento. Por

Figura 36 – Diagrama de classe apresentando as interfaces públicas do motor de jogos para interface do usuário.



Fonte: autoria própria.

tema, temos:

- Desenho. É possível controlar pixels no framebuffer individualmente, dada uma cor desejada e suas coordenadas x e y – bem como especificar um retângulo e uma cor para desenhar em regiões maiores. O motor fornece uma função de apoio para limpar toda tela, pintando-a de uma cor passada de parâmetro. Quanto a imagens, a função de desenho aceita um vetor de cores indexadas e a região onde se deve desenhar. Existe também uma função que faz a transferência direta da imagem descartando a opacidade para ganhar performance – o fato de não haver cálculo de opacidade permite o uso de função de baixo nível para copiar trechos da memória ("memcpy", de C). Todas estas operações internamente manipulam o framebuffer, que posteriormente será transferido pelo controlador para tela, assunto visto em 3.2.

- Estruturas de dados, que são declaradas para dar suporte as funções de desenho. Definem pontos, tamanhos e regiões. Uma estrutura também é definida para representar uma cena, contendo funções para se inicializar, atualizar e finalizar.

- Cenas. Por meio da interface com o motor de jogos é possível pausar, retomar e trocar a cena do jogo.

Já “IO” é a abreviação para “Entradas e Saídas”, do termo em inglês “Input/Output”. Comporta o código que dá acesso ao estado dos controles e declara a estrutura de dados relacionada a eles. É possível verificar cada um dos botões individualmente e há suporte para mais de um controle. Cada botão pode estar em uma de quatro posições possíveis, seguindo a máquina de estados vista em 2.2.3. A Figura 37 ilustra esse agrupamento, apresentando seu diagrama de classes.

Figura 37 – Diagrama de classe apresentando as interfaces públicas do motor de jogos para entradas de controles.

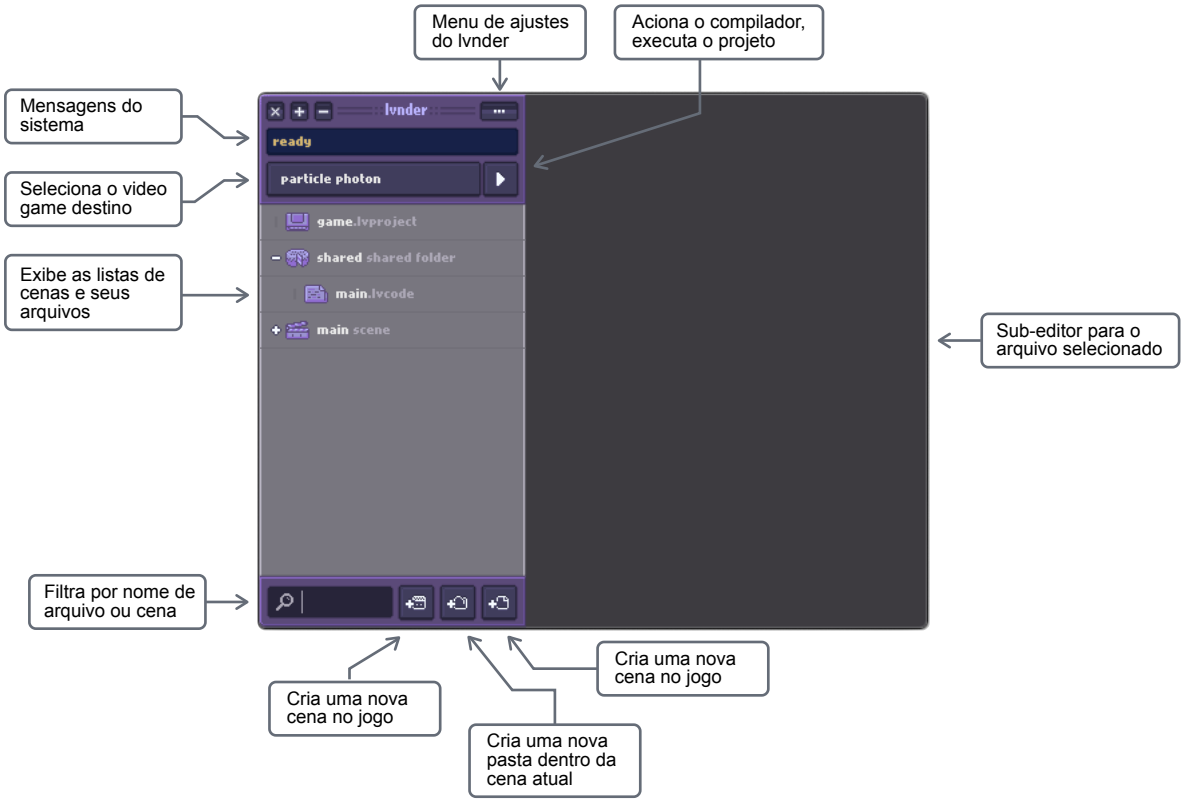
Fonte: autoria própria.

Das partes do motor de jogos, as interfaces públicas são as que mais promovem coesão ao ecossistema. Uma alteração de método público significa que todos jogos – e até mesmo todos controladores – potencialmente precisariam atualizar-se. Por esse motivo, são poucas e versionadas. Cada imagem de contêiner é feita para uma versão especifica das interfaces publicas do motor. Os desenvolvedores de jogos podem optar por versões mais novas ou não do motor de jogos, escolha feita no arquivo de configuração do projeto, durante a construção do jogo, no Ambiente Integrado de Desenvolvimento.

3.3.2 Ambiente integrado de desenvolvimento

Na subsecção 2.2.7, citando Gregory (2018), comentamos sobre os editores dos motores, que são ferramentas com interfaces gráficas para auxiliar nas tarefas do dia a dia do desenvolvimento de um jogo. Para este projeto um editor que cria um ambiente integrado de desenvolvimento foi construído, batizado de “lvnder”. Nele os desenvolvedores conseguem visualizar a estrutura de cenas, configurar seus projetos, editar o código-fonte e disparar o processo de compilação e instalação nos videos games. Internamente o lvnder – que pode ser visto na Figura 38 – é baseado em tecnologias multiplataforma e acessíveis.

Figura 38 – O Ambiente Integrado de Desenvolvimento “lvndr”



Fonte: autoria própria.

Assim como quando da escolha da ferramenta para contêineres, decidimos por uma linguagem cercada de bibliotecas gratuitas e com uma comunidade ativa: a TypeScript, mantida pela Microsoft e de código aberto. Novamente o critério de decisão baseou-se na pesquisa com desenvolvedores feita pelo site americano StackOverflow (2020), que destacou esta linguagem pelo ritmo com que vem conquistando os desenvolvedores. Citando-os, “após um aumento consistente nos últimos cinco anos, Python caiu de segundo lugar no ano passado para o terceiro este ano na lista das tecnologias mais amadas, sendo derrotado pelo TypeScript”[11].

Para interface do usuário, investigamos três bibliotecas gráficas de código aberto: Angular, criado pelo Google; React, do Facebook; Vue.js, criada por Evan You e mantida pela comunidade. Aplicamos a mesma metodologia utilizada no restante deste trabalho, verificando a pesquisa StackOverflow (2020) com desenvolvedores. Desta vez, porém, ela mostrava uma divisão na comunidade. React e Angular são ambos modernos e bem aceitos. Optamos então por desenvolver uma prova de conceito nas duas ferramentas, analisar

[11] “After a consistent rise over the last five years, Python fell from second last year to third this year on the list of most loved technologies, being beat out by TypeScript.”, com tradução do autor.

os resultados e decidir pela que tenha sido mais produtiva. Com os resultados em mãos, escolhemos o React. Isso não significa que uma seja melhor do que a outra para qualquer desenvolvimento, apenas que foi a mais adequada para tarefa e adaptou-se melhor as habilidades prévias de quem escrevia o código.

Com Typescript e React escolhidos, adicionamos na pilha de software uma ferramenta que pudesse conversar com o sistema operacional e com o Docker. Duas opções compatíveis foram levantadas: NW.js e Electron. Tabelamos vantagens e desvantagens e, por fim, a última opção foi escolhida, principalmente pela quantidade de aplicações de mercado em seu portfólio. Slack, Skype, Discord e VSCode são exemplos de aplicações utilizando Electron.

Em seu código, o lvnder é apenas uma interface visual para o que está acontecendo dentro do contêiner Linux. Ele executa chamadas em um terminal remoto, enviando comandos a outra biblioteca do motor de jogos que está instalada no contêiner, a lv-cli – uma interface via linha de comando.

3.3.3 Interface via linha de comando

Quando o "lvndr" abre um projeto no ambiente de desenvolvimento, automaticamente aciona o Docker local e provisiona um contêiner Linux, utilizando a imagem especificada no arquivo do projeto. Dai em diante, o contêiner passa a receber comandos de terminal disparados pelo "lvndr", em resposta a ações do usuário. Assim ocorre a comunicação entre sistema hospedeiro e contêiner.

Para facilitar tarefas rotineiras, foi criada uma ferramenta via interface de linha de comando. Toda imagem de contêiner para o motor de jogos herda este programa, chamado "lv-cli". Isso é feito pelo sistema de imagem base do Docker. O motor de jogos disponibiliza publicamente uma receita inicial que já incluí a instalação desta interface, bastante os controladores indicarem essa base nas suas receitas de contêineres. Os comandos aceitos pela "lv-cli" são:

- "scan": varre a pasta do projeto, suas sub-pastas e arquivos. Separa as entradas por cenas e extensão de arquivos. Agrupa os arquivos de formatos desconhecidos e pastas cujos nomes estão fora do padrão. Isso é possível porque o contêiner consegue mapear o diretório do computador hospedeiro em seu sistema de arquivos. O resultado final é um índice, gravado em um arquivo no formato json. O ambiente integrado usa esse arquivo para construir a interface – que pode ser vista na Figura 38 –.
- "encode": recebe o índice gerado no comando "scan" e invoca os codificadores para cada um dos arquivos, segundo sua extensão. Utiliza a configuração do projeto como

apoio para instalar ou atualizar qualquer codificador que seja necessário. Isso é feito via Internet e o Docker provê o acesso de rede para o contêiner. O resultado, arquivos json com o conteúdo codificado, é salvo em uma pasta dentro do contêiner. O assunto codificadores é abordado em detalhes na subseção 3.4.

- "build": é comando para que o projeto seja construído. Nesta etapa, a "lv-cli" delega a tarefa de compilar o código para implementação específica feita pelo controlador – assunto da seção 3.2. Nesta etapa, ocorre também a instalação do firmware compilado no aparelho de video game. Isto também varia a cada par controlador/video game. No protótipo LVNDR-01 é feita pela Internet, usando o serviço de atualização sem fio da fabricante. Já no ODROID-GO, o controlador mapeaia a porta USB dentro do contêiner para a USB do sistema hospedeiro, atualizando o firmware por ela.

Deslocar estas funcionalidades para dentro do contêiner tem várias vantagens. A Figura 39 mostra a delimitação dos sistemas operacionais e também as tecnologias utilizadas. No Linux o programa roda com acesso privilegiado, podendo instalar dependências livremente e executar scripts localmente. Tudo isso sem comprometer a segurança do computador hospedeiro. O que for instalado e gerado será destruído junto com o contêiner quando o lvndr fechar. Sobre esse tema, nas palavras de McKendrick (2020), "ao configurar namespaces, o Docker mantém os processos isolados em cada contêiner, não apenas de outros contêineres, mas também do sistema hospedeiro"[12].

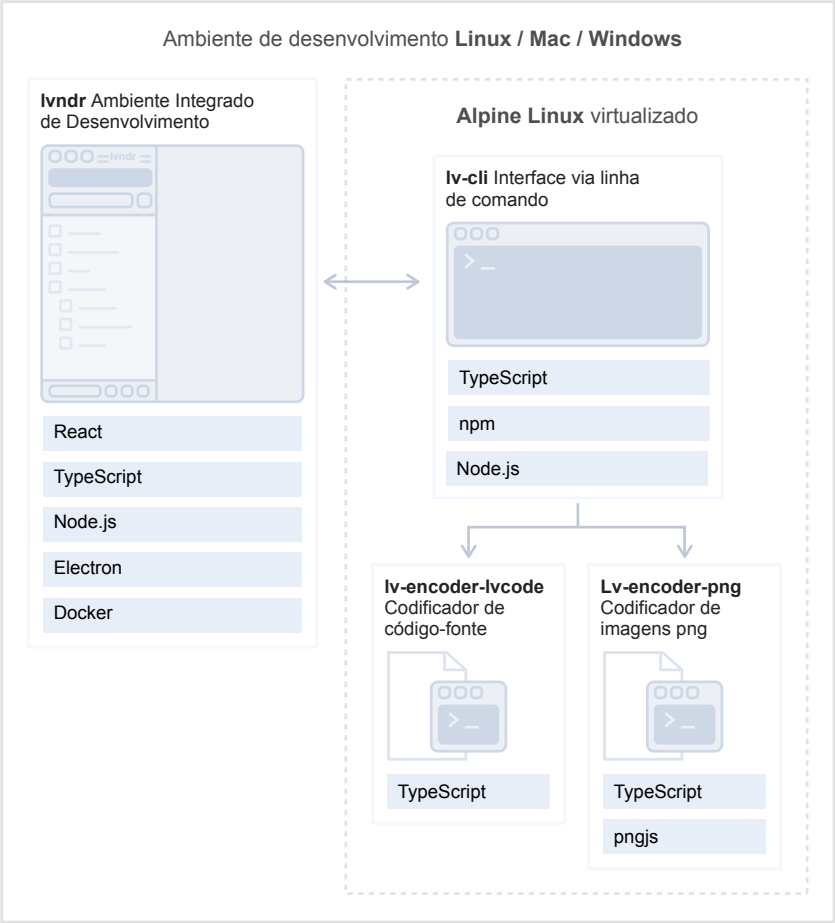
Como a lv-cli não tem interface gráfica e aceita comandos com entradas e saídas bem definidas, foi possível escrever testes unitários para cada um dos comandos. Desta maneira, otimizações internas de código podem ser feitas com segurança, pois o teste unitário valida a conformidade da resposta, e novas funções podem ser adicionadas sem que as antigas sejam afetadas.

Por fim, isso dá a liberdade para que outros ambientes de desenvolvimentos sejam utilizados ou construídos, à critério dos desenvolvedores de jogos. O lvndr é apenas uma interface gráfica para o lv-cli, que roda dentro do contêiner Linux.

O código-fonte é livre, sob a licença GPL v2.0 e está disponível online, no site Github. Os executáveis são versionados e publicados em um repositório de bibliotecas Typescript, o npm.

[12] "By setting up namespaces, Docker keeps the processes isolated in each container, not only from other containers but also from the host system.", com tradução do autor.

Figura 39 – Delimitação do contêiner e das tecnologias utilizadas nas bibliotecas do motor de jogos.



Fonte: autoria própria.

3.4 Codificadores

Além do código-fonte escrito pelos desenvolvedores, um jogo também utiliza arquivos de multimídia – tema explorado na seção 2.3, sobre formatos digitais, e subseção 2.2.4, sobre gestão de conteúdo. Em tempo de execução, o video game precisa lidar de alguma maneira com estes recursos, porém só o que entende são instruções de processador. Em sistemas operacionais de computadores, como Windows e Mac, existem bibliotecas que cuidam de interpretar arquivos de multimídia, lendo seus dados binários e dando sentido ao seu conteúdo. No Linux, por exemplo, temos a libPNG – um executável de 992 kilobytes para ler e salvar arquivos de imagem no formato PNG[13]. Em microcontroladores, todavia, espaço e processamento são escassos, o que torna o uso de bibliotecas do tipo inviável na maior parte dos cenários.

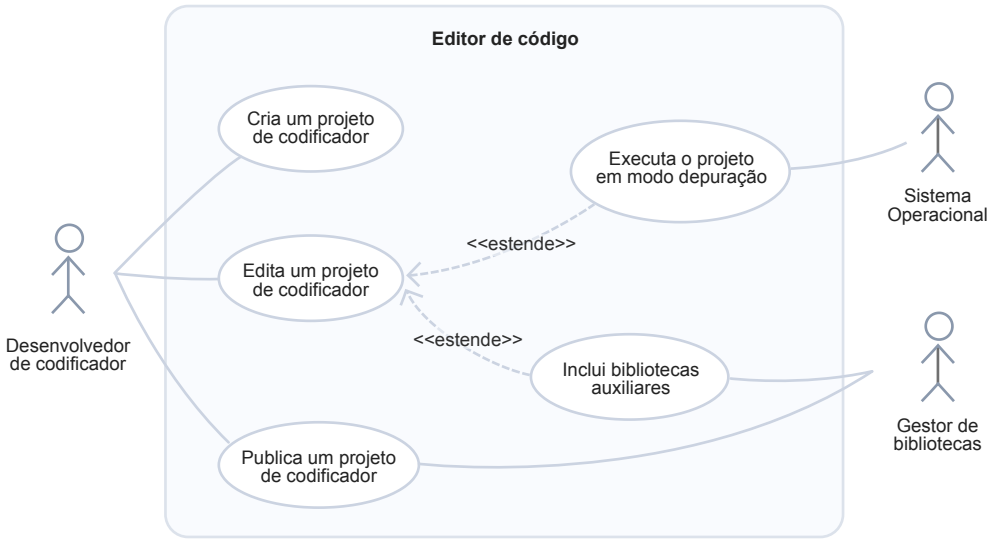
[13] Disponível em https://sourceforge.net/projects/libpng/files. Acessado em 22/11/2020

O motor de jogos proposto utiliza codificadores para lidar com esta situação. Codificadores são programas que, durante a compilação do projeto, transformam um conteúdo multimídia em trechos de código fonte já formatado para conversar com as interfaces públicas do motor de jogos. Esta é uma das frentes de desenvolvimento propostas pela arquitetura, apresentada na seção 3.1.

Não há limites para quantidade de codificadores ou arquivos codificados, por ser uma etapa executada durante a compilação do jogo, mas é importante que o codificador produza uma saída que seja coerente com a quantidade de memória e processamento disponíveis no microcontrolador. Um codificador de arquivos PNG, por exemplo, que apenas descompacte a imagem e codifique-a para um vetor de bytes em C, utilizando 3 bytes para representar cada pixel, irá rapidamente esgotar a memória disponível no controlador. Nestes casos, é preciso que o codificador também forneça algum tipo de compactação e algoritmo de descompressão em tempo de execução – assunto tratado em detalhes na subseção 3.4.2.

A Figura 40 lista os casos de uso para um desenvolvedor de codificador. Na sequência, serão expostos os codificadores de código e imagem.

Figura 40 – Diagrama UML de Casos de Uso para o desenvolvedor de codificadores.



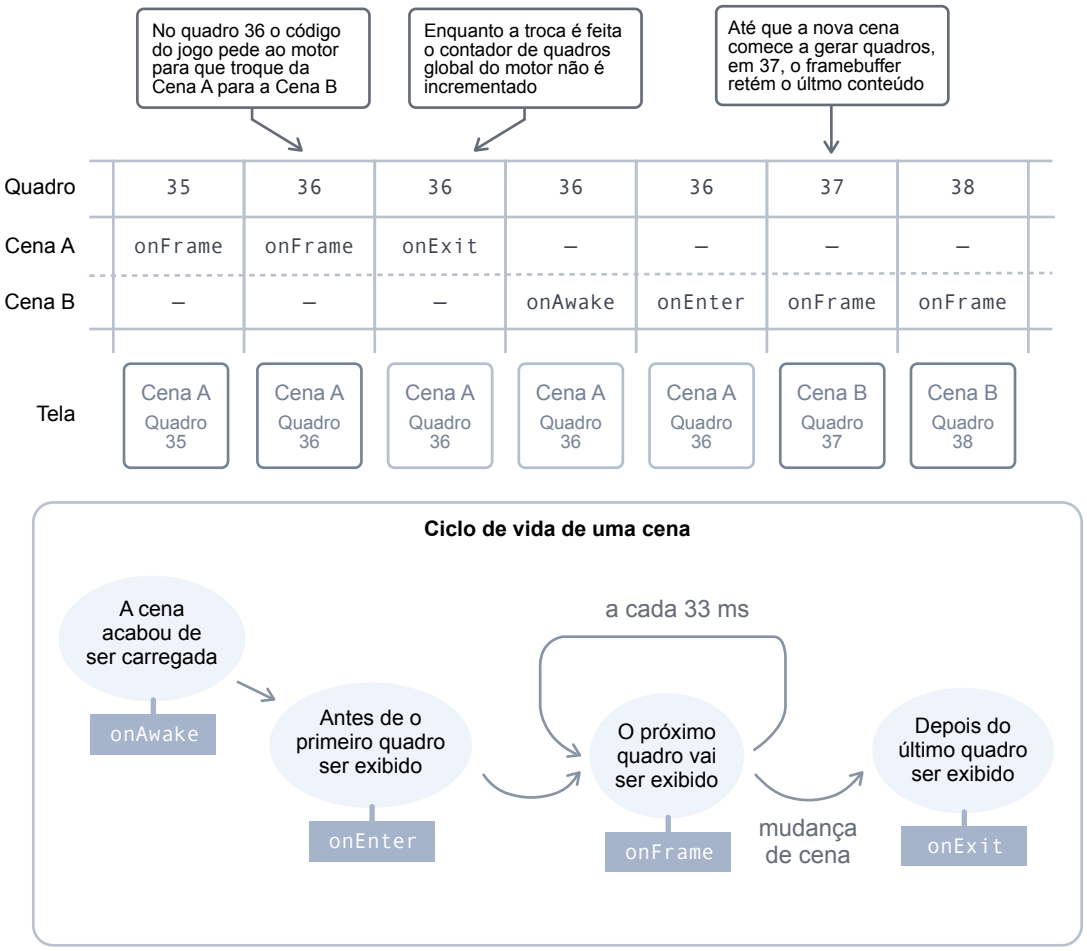
Fonte: autoria própria.

3.4.1 Codificador de código

O motor de jogos reforça uma convenção de ciclo de vida das cenas para que os desenvolvedores de jogos e codificadores possam ter oportunidades de alocar recursos,

configurá-los e, após o fim da cena, liberar o máximo possível de memória. A Figura 41 ilustra este funcionamento.

Figura 41 – Mecanismo de troca de cenas e seu ciclo de vida.



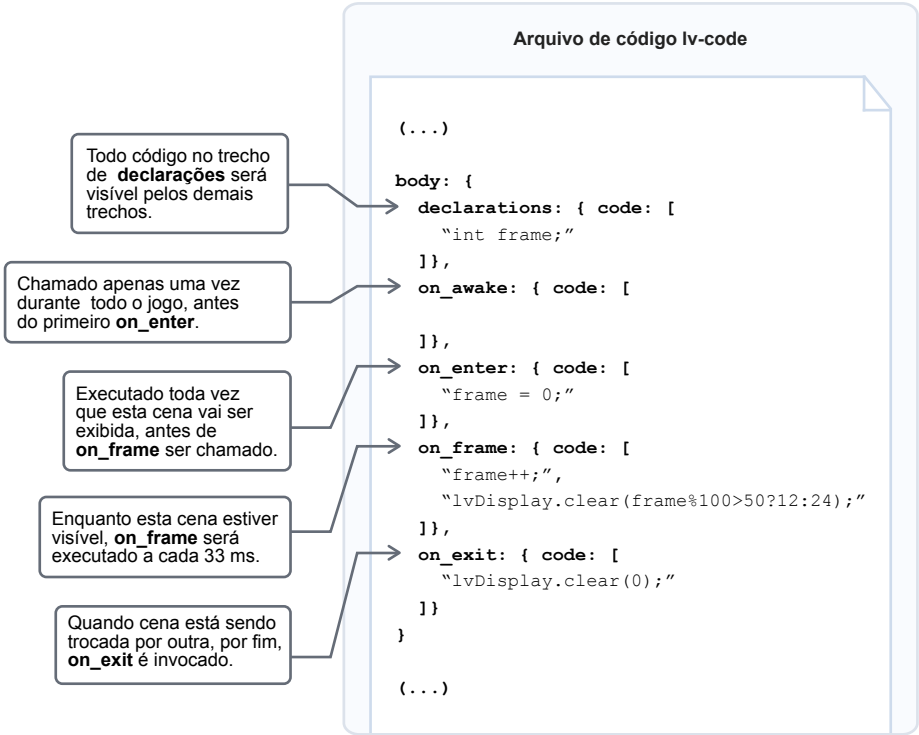
Fonte: autoria própria.

Por esse motivo, os codificadores geram como resultado arquivos intermediários durante a compilação, especificando trechos de código que devem ser executados em cada estágio do ciclo de vida de uma cena. Sendo assim, para codificar um código-fonte em C, apenas um arquivo de texto puro com as instruções não é o suficiente. O programa codificador precisa de alguma demarcação no arquivo original para saber qual trecho deve estar em cada ponto do ciclo de vida.

Para lidar com este cenário, o motor especifica um formato próprio, de extensão ".lv-code". É, basicamente, um arquivo de texto contendo um JSON. Em sua raiz existem objetos para cada estágio do ciclo de vida da cena, além de um objeto para declarações, cada um deles contendo um vetor de strings com o código em C. O codificador apenas

verifica a integridade do arquivo e, caso não encontre inconsistências, dá como resultado a própria entrada. Um exemplo deste mecanismo pode ser visto na Figura 42.

Figura 42 – Um exemplo de arquivo no formato lv-code.



Fonte: autoria própria.

3.4.2 Codificador de imagem

Como visto ao longo deste trabalho, enquanto está executando no video game, o motor de jogos aceita um vetor de cores indexadas como entrada para o desenho de imagens no framebuffer. Este formato intermediário já é mais compacto do que os 2 bytes por pixel utilizados pelas telas com 16 bits de profundidade de cores – caso tanto do ODROID-GO, quanto do LVNDR-01.

Com a dinâmicas de cenas, o jogo não precisa de todos os gráficos carregados na memória RAM ao mesmo tempo. Cada cena busca seu conteúdo multimídia da ROM quando é carregada e libera a memória quando é substituída por outra cena. Desta maneira, trabalhando com a RAM e ROM, funcionavam os jogos para o console NES, da Nintendo. Nele, segundo Hugg (2019), alguns títulos chegavam a ter 512 Kilobytes de ROM em seu cartucho, enquanto a memória de video do console, VRAM, comportava apenas 16 Kilobytes. Sendo assim, o codificador de imagens foi planejado para que, durante

a compilação, defina:

- Uma versão compactada do conteúdo, que residirá na ROM;
- Um código que, quando executado no carregamento da cena, descompacte a imagem na RAM, já no formato aceito pelo framebuffer;
- Por fim, um trecho que libere toda a memória RAM quando a cena for encerrada.

Para versão compactada, dois algorítimos de compressão foram considerados: a codificação run-length, também chamada de RLE, e a de Huffman – vistas em detalhes na subseção 2.3.2. Quanto a complexidade de tempo e espaço para descompressão, as duas são lineares. Após testes com diferentes imagens de jogos, obtivemos melhores taxas de compressão implementando uma solução híbrida, estudada em maiores detalhes a seguir.

No algoritmo run-length, o potencial de compressão está nas grandes sequências de dados repetidos. Quanto maior a sequência, maior o ganho de compressão. É importante que o codificador explore esta característica. Uma das maneiras de fazê-la acontecer é reorganizando a informação. A probabilidade de um próximo caractere ser igual ao atual cresce quando o número de caracteres possíveis é reduzido. No processamento de imagens podemos fazer isso por meio de planos de bits, ou bitplanes, que são matrizes de zeros e uns. Schaathun (2012) explica que "por plano de bits i, queremos dizer a imagem binária que é obtida tomando apenas o bit i da representação binária de cada pixel". A quantidade de planos necessários, p, é proporcional a quantidade de cores possíveis, c, segundo a fórmula:

$$p = \left\lceil \frac{\log(c)}{\log(2)} \right\rceil \tag{3.1}$$

Por mais que seja contra intuitivo dividir a matriz das cores indexadas em várias matrizes binárias e esperar que isso reduza o tamanho do arquivo gerado, a menor quantidade de cores por matriz potencializa a codificação run-length, criando cadeias maiores de repetição. A Figura 43 compara as compressões run-length com e sem bitplanes.

Em codificação Huffman, o ganho está em utilizar tamanhos variáveis, associando códigos de menor tamanho a informações mais frequente. O conceito de variar o tamanho sob demanda tem potencial de ganho de compressão e também precisa ser explorado. Quando gerando o arquivo binário, cada símbolo do run-length, representando a cor ou a extensão da repetição, precisa ser escrito. A maneira mais simples de fazê-lo é utilizando a representação binária do número inteiro.

Figura 43 – Comparação entre as versões com e sem uso de bitplanes para codificação run-length em uma imagem (menos símbolos = melhor).

	Cores True Color	Cores indexadas	Cores indexadas e RLE	Cores indexadas, bitplanes e RLE
	64 milhões de cores 768 símbolos	4 cores 256 símbolos	4 cores 98 símbolos	4 cores 69 símbolos
	64 milhões de cores 768 símbolos	1 cor 256 símbolos	1 cor 2 símbolos	1 cor 2 símbolos
	64 milhões de cores 768 símbolos	4 cores 256 símbolos	4 cores 512 símbolos	4 cores 258 símbolos

Fonte: autoria própria.

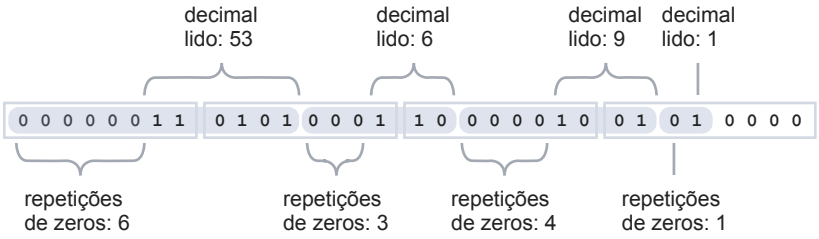
A dificuldade está em posicionar um símbolo na sequência do outro de modo que depois se consiga saber onde um termina e o outro começa, já que são apenas zeros e uns. Uma solução é fixar a quantidade de bits para cada um destes número. Por exemplo, cada um deles utilizando 8 bits, assim o trabalho se resume a ler um inteiro de byte em byte. O problema desta abordagem é que, para cadeias pequenas, com uma ou duas repetições, muitos bits são descartados, apenas preenchidos com zero; para cadeias grandes, com mais de 255 repetições de extensão, um transbordo de inteiro acontece, pois não se consegue representar números maiores apenas com 8 bits.

Encontramos ganho de compressão utilizando um algorítimo para escrever os símbolos utilizando um tamanho variável de bits. Basicamente, antes da versão binária da extensão da repetição, marcamos com zeros seguidos o tamanho do número que vem adiante, em bits. A Figura 44 ilustra como a leitura é feita, determinando onde começam e terminam cada um dos números.

Novamente, apesar de ser contra intuitivo repetir zeros para sinalizar o tamanho em bits do próximo número, houve ganho de compressão fazendo desta maneira. Um passo a passo de caso real pode ser visto na imagem 45.

Sendo assim, conseguimos reduzir significantemente a quantidade de bytes necessária para armazenar as imagens na ROM dos jogos. Para reverter o processo, quando o video game precisa da matriz de cores indexadas que seja transferida para o framebuffer, é preciso saber: qual a largura e altura da imagem; quantos bitplanes ela tem; qual sua tabela de cores. Para tanto, bastou definir um formato de cabeçalho com estas informações

Figura 44 – Exemplo da leitura do tamanho de repetição de uma codificação run-length utilizando a estratégia de representação de tamanho variável.



Fonte: autoria própria.

inserido antes dos bitplanes no arquivo.

O algoritmo para descompactar a imagem é inserido automaticamente no arquivo gerado pelo codificador, dentro da parte que especifica as declarações. Em tempo de execução, é utilizado de forma integrada ao ciclo de vida das cenas, que é imposto pelo motor de jogos. A descompactação é feita na entrada da cena – alocando a memória RAM necessária dinamicamente – acompanhada, em seu encerramento, da devolução da memória previamente alocada. A estratégia de cenas é ilustrada na Figura 10 e comentada na subseção 2.2.5.

3.5 Editores

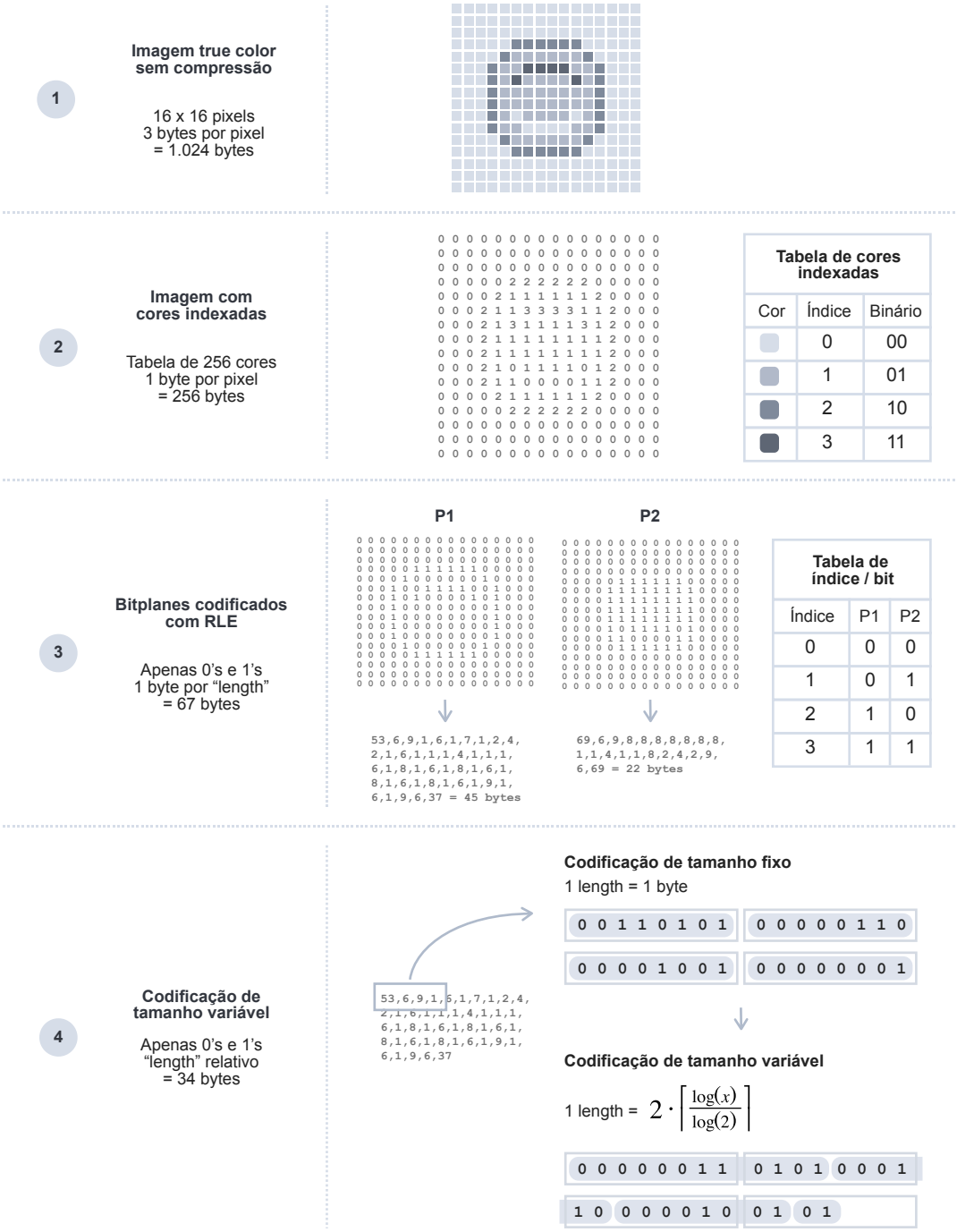
Quando criando um jogo, no ambiente integrado de desenvolvimento – apresentado na subseção 3.3.2 – o desenvolvedor tem acesso a editores de arquivos multimídia, que dão suporte a tomada de decisão e criação de conteúdo. O funcionamento é muito similar ao dos codificadores, um programa que reside no contêiner Linux é indicado para cada extensão de arquivo nas configurações do projeto.

3.5.1 Estratégia de associação por extensão

Durante a compilação, os arquivos são separados primeiramente por cena. Isso é feito usando uma convenção. Cada cena é uma pasta no projeto que tem o nome começando em “scene-”. Esta abordagem foi utilizada visando incentivar a organização e padronização, buscando encontrar os mesmos resultados positivos dos estudos feitos por Guana, Stroulia e Nguyen (2015) – citado na subseção 2.5.2.

Dentro de cada cena, por sua vez, são separados por extensão, dado cada nome de arquivo. Por exemplo, um arquivo “herói.png” será agrupado juntamente com os demais arquivos que serão tratados pelo codificador e editor do formato “png”. Assim, o motor de

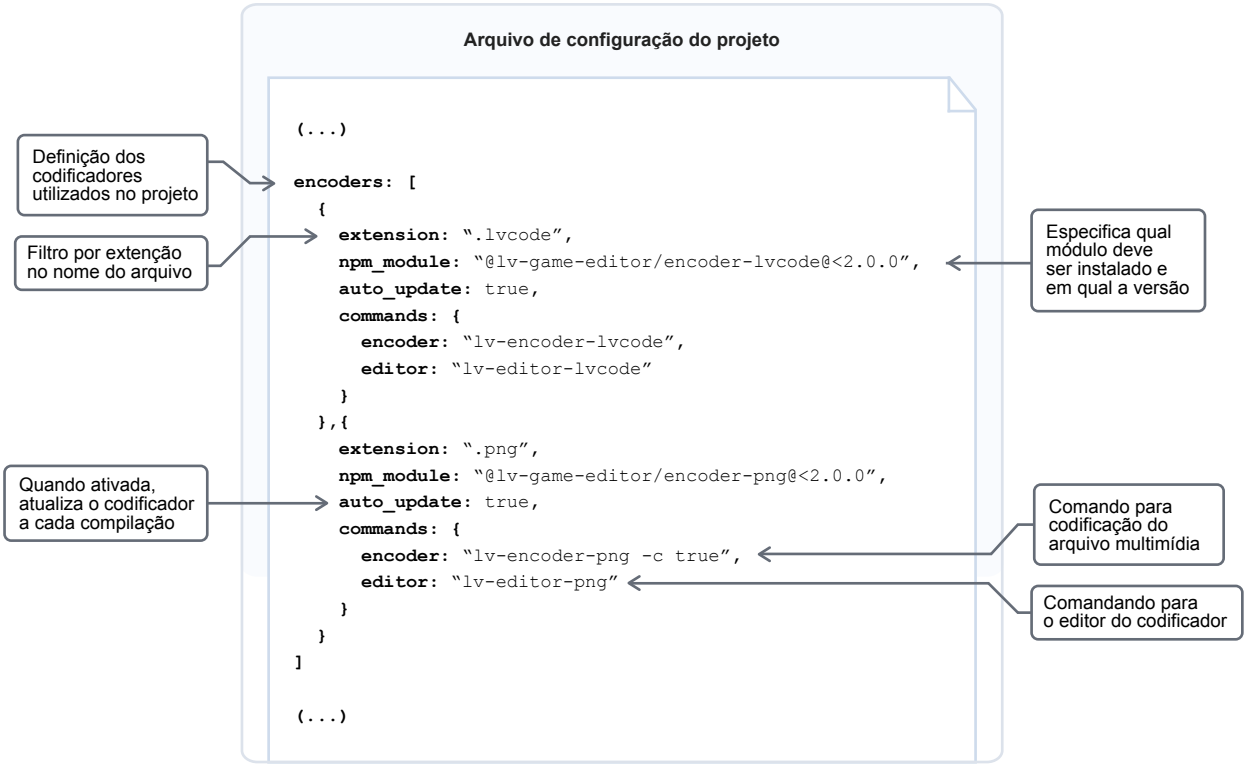
Figura 45 – Algoritmo de compressão de imagem utilizando Bitplanes.



Fonte: autoria própria.

jogos pode buscar nas configurações do projeto – dentro do mapa de codificadores – o trecho que especifica quais pacote e comando devem ser utilizados dentro do contêiner Linux para invocar o editor de cada um deles. Acima de tudo, desta maneira os desenvolvedores podem escolher livremente qual editor usar – incentivando a comunidade a experimentar, incluindo suporte a novos formatos e otimizando os codificadores e editores já existentes.

Figura 46 – Trecho da configuração do projeto que especifica os codificadores e editores.



Fonte: autoria própria.

Na Figura 46, é possível ver um exemplo de configuração. Ilustrando: arquivos de extensão "png", por exemplo, por padrão estão associados ao pacote "@lv-game-editor/encoder-png", desenvolvido pelo autor deste trabalho durante a pesquisa. O motor de jogos consegue, com esta informação, acionar um repositório global de bibliotecas. Via Internet e ainda dentro do contêiner, solicita a instalação destes programas[14]

3.5.2 Utilização de Webviews

Pensando em extensibilidade, fizemos cada editor como um programa próprio, que é invocado pela "lv-cli" – interface de linha de comando do motor de jogos, vista na subseção 3.3.3 – que fica sendo executado como um servidor web local, dentro do contêiner Linux, enquanto o "lvndr" está aberto. Assim, na prática, as interfaces dos editores são apenas páginas HTML que o ambiente de desenvolvimento acessa, servidas pelo contêiner em uma porta TCP específica escolhida pelo motor de jogos. Para evitar que um editor possa afetar outro, ou até afetar o funcionamento do motor de jogos, adicionamos uma proteção fornecida pelo Electron – a ferramenta de interface do motor. Cada editor é exibido dentro

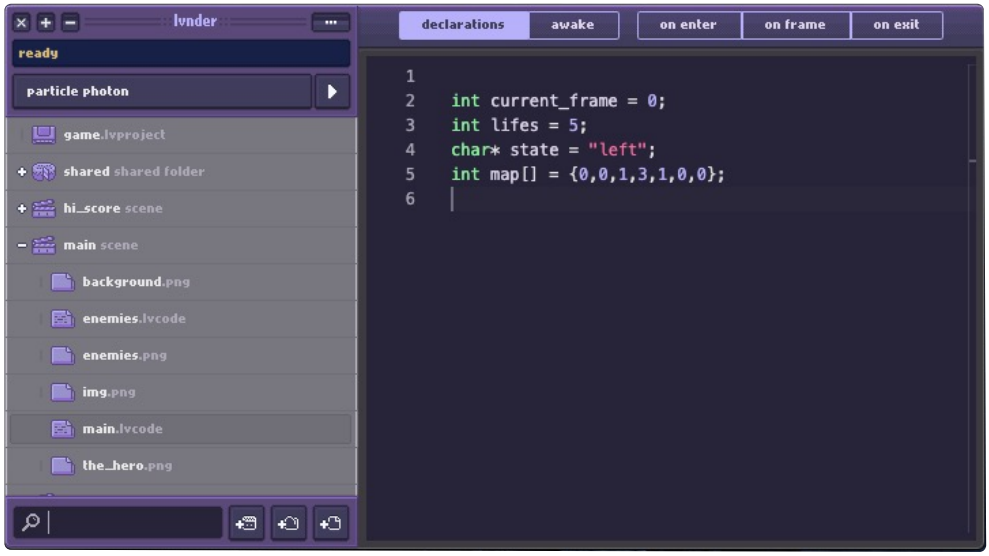
[14] Na ocasião da escrita do projeto, o repositório público de bibliotecas utilizado era o "npm", da empresa americana Github Inc.

de uma separação, que o Electron chama de Webview, e não tem acesso a variáveis ou métodos tanto dos outros editores, quanto do motor de jogos.

Fazendo desta maneira, os desenvolvedores de editores podem escolher suas ferramentas, bibliotecas e até utilizar outras linguagens de programação. Como o servidor do editor é executado dentro do contêiner, consegue invocar a "lv-cli", rodar seu próprio codificador, fornecer interfaces públicas para criar uma interface rica, entre outras liberdades.

3.5.3 Editor de código

Figura 47 – Captura de tela do editor de código sendo executado dentro do ambiente integrado de desenvolvimento.



Fonte: autoria própria.

Sendo a ferramenta central para o programador de jogos, na interface do editor de código buscamos oferecer uma experiência otimizada, que faça fluir o trabalho do desenvolvedor. Felizmente existem ótimas opções de editores sob licença de código livre, que podem ser integradas a outros programas – como o ambiente de desenvolvimento deste trabalho.

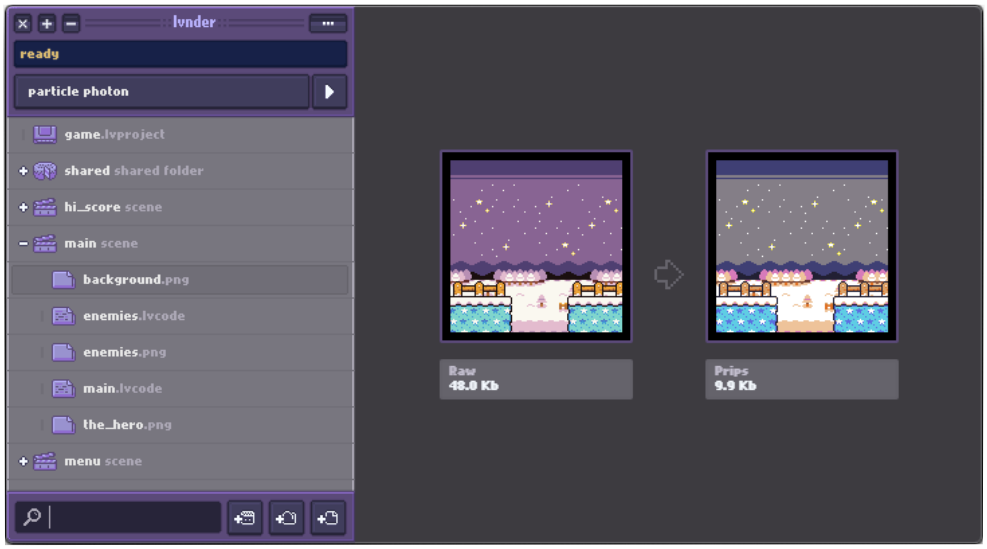
Avaliamos três soluções de editores: Ace, um projeto da comunidade e utilizado pela Amazon AWS sob o nome Cloud9; Monaco, componente core do editor Visual Studio Code, da Microsoft, e mantido por ela; CodeMirror, mais uma opção da comunidade, é o editor utilizado pelo Google Chrome e Mozilla Firefox nas ferramentas de desenvolvedor que vão embarcadas nos navegadores.

Todas opções funcionam muito bem e são utilizadas em grandes projetos. Assim como as demais decisões sobre tecnologia, utilizamos uma grande pesquisa anual com desenvolvedores para ponderar sobre o assunto. Com dados de 2019, StackOverflow (2019) afirma que o Visual Studio Code foi a opção escolhida como ferramenta para edição de código por 50.7%, dos mais de 88 mil entrevistados naquele ano. Optamos, então, por utilizar seu componente de edição de código, o Monaco. Com isso, a experiência de codificar no motor de jogos ficou muito próxima a da codificação no Visual Studio Code. Todos os atalhos de teclado permanecem os mesmos. São oferecidas funções úteis auxiliares, como as de ajuste de tamanho de fonte, completamento de código, busca e refatoração. Na Figura 47 é possível visualizar o resultado obtido com esta integração.

Para deixar clara a estratégia de cenas do motor, adicionamos ao editor de código uma aba para cada ocasião do ciclo de vida da cena. Isso também é um mapeamento direto para os trechos de código do formato de arquivo "lv-code", ilustrado na Figura 42, que será gerado pelo codificador de código – tema da subseção 3.4.1.

3.5.4 Editor de imagem

Figura 48 – Captura de tela do editor de imagens sendo executado dentro do ambiente integrado de desenvolvimento.



Fonte: autoria própria.

Um time de desenvolvimento de jogos é multidisciplinar, envolve programadores, ilustradores, roteiristas, dentre outros. Cada uma destas especializações tem suas ferramentas. Ilustradores, por exemplo, têm a disposições vários programas para edição de

imagens para jogos, como ASE Sprite, de David Capello, e Piko Pixel, de Josh Freeman. Por esse motivo, neste trabalho preferimos não criar mais um editor de imagens – o que seria possível e interessante, até para atrair iniciantes – mas sim auxiliar na tomada de decisão consumindo uma imagem vinda de um editor externo, que os ilustradores já tenham familiaridade.

Na ferramenta criada, é possível ter uma pré-visualização de como serão as cores indexadas escolhidas na compactação de bitplanes e também saber quantos Kilobytes serão necessários na memória ROM para aquela imagem. Também é exibido o tamanho bruto da imagem original, considerando 3 bytes por pixel. Uma exemplo pode ser visto na Figura 48.

3.6 Jogos conceituais

Dentre as frentes propostas pela arquitetura na subseção 3.1, o desenvolvimento de jogos é a última delas e também a principal, sendo a motivação desta pesquisa. Na Figura 49 apresentamos o diagrama de casos de uso para um desenvolvedor de jogos, considerando toda a arquitetura do motor.

Figura 49 – Diagrama UML de Casos de Uso para o desenvolvedor de jogos.

Fonte: autoria própria.

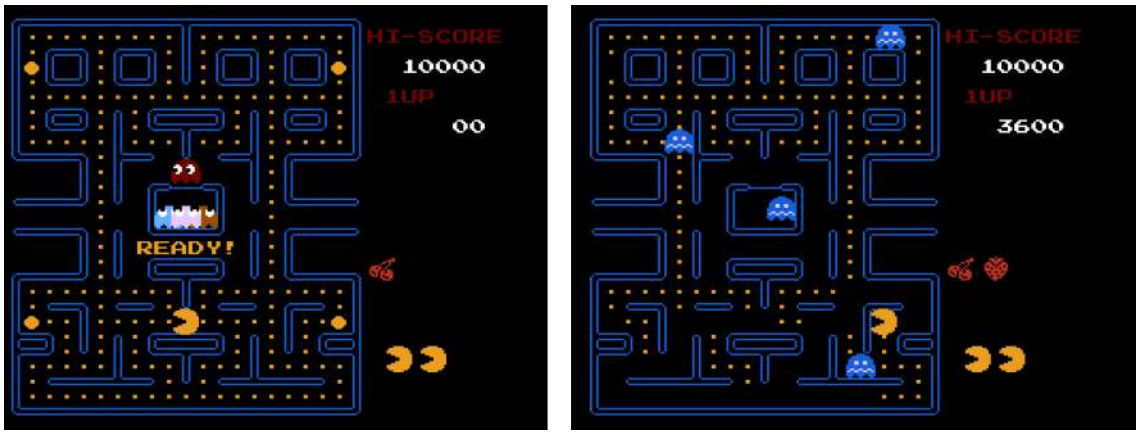
Nesta etapa as ferramentas criadas – controladores, bibliotecas, codificadores e editores – precisam trabalhar em conjunto, dentro do contêiner, para facilitar o trabalho do programador de jogos. Conversando entre si, precisam automatizar tarefas repetitivas, como a compactação de imagens e a compilação do projeto, de modo que o desenvolvedor possa se concentrar em codificar sua ideia de jogo.

Durante o desenvolvimento do projeto, dois jogos conceituais foram implementados – tanto para validar o motor de jogos, quanto para guiar as otimizações e adaptações, por venturas necessárias. A seguir, são descritos em detalhes.

3.6.1 Conceito 1 - Jogo Arcade

Para os testes iniciais, um jogo minimalista foi implementado. Optamos por um título da categoria arcade, em que a quantidade de cores não afeta o andamento do jogo; parte dos gráficos são figuras geométricas primitivas; a quantidade de elementos na tela é controlada; por fim, não são utilizadas animações complexas, de muitos quadros. Exemplo de jogo nesta categoria, Pac-man, na japonesa Namco, é ilustrado na figura 50.

Figura 50 – "Pac-man", para o console NES, da Nintendo.



Fonte: Namco, do Japão. Publicado nos Estados Unidos pela Tengen Inc., em 1989.

Nesta fase, a taxa de quadros por segundo atingida precisa estar entre 20 e 30 e o uso de memória não deve ser um impedimento. O tempo de carregamento deve ser compatível com os demais jogos disponíveis nos consoles micro controlados. O jogador deve ser capaz de interagir com o jogo utilizando os botões físicos sem problemas de latência. O motor deve ser auto suficiente no processo de gerar binários para os dois microcontroladores estudados.

A Figura 51 mostra a interface do jogo desenvolvido. O conceito funciona de maneira similar ao popular Tetris, por Alexey Pajitnov. Peças de formatos diferentes são manipuladas e acumuladas pelo jogador ao longo da partida. Deve-se preencher linhas inteiras de um tabuleiro para marcar pontos, abrindo espaço para que mais peças possam ser adicionadas. O jogo termina quando a pilha de peças passa de um dado limite de altura. Tetris é um jogo com direitos reservados, como esclarece Weinberg (2012). Por este motivo, as peças utilizadas no conceito têm formatos alternativos, o jogo inverte a gravidade – com peças subindo, ao invés de descendo – e o tabuleiro tem dimensões diferentes. Várias partes do motor de jogos foram estressadas e validadas na produção deste jogo conceitual.

Figura 51 – Interface projetada para prova de conceito 1. Utiliza gráficos de domínio público, sob licença CC-0, criados por "GrafxKid", "ansimuz" e "surt", do portal online "Open Game Art".

Fonte: autoria própria.

O fundo azul é criado proceduralmente, em tempo de execução, a cada quadro, com uma função de desenho geométrico do motor. Isso economiza o espaço em memória RAM e ROM que uma imagem para representar o mesmo retângulo azul ocuparia. De maneira similar – utilizando um método que muda a cor de um pixel individualmente – é desenhada a linha que sai da peça e vai até o topo da tela.

Para o desenho das imagens, exercitamos as funções com e sem tratamento de transparência. A figura das ilhas ao fundo e das peças do tabuleiro são transferidas diretamente para o framebuffer, por não conter transparência, economizando tempo de CPU. O algoritmo inserido pelo codificador de imagens faz essa análise por conta própria

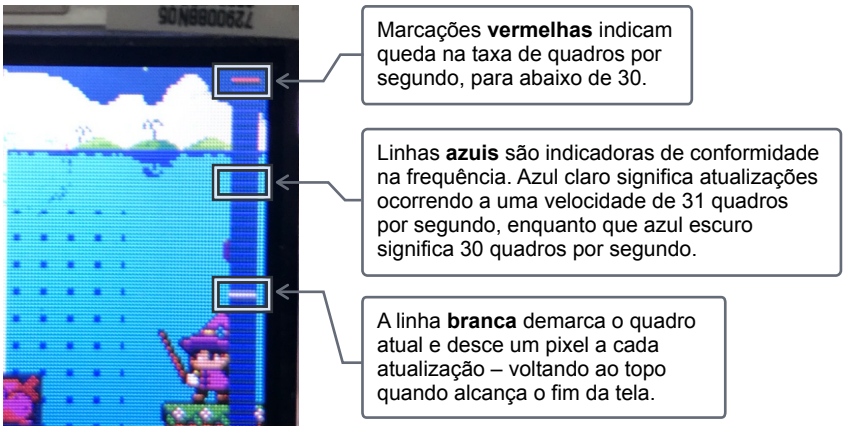
enquanto descompacta o arquivo no carregamento da cena, buscando pela existência ou não de transparência. Isso facilita o trabalho do desenvolvedor, que não precisa se preocupar com esse tipo de detalhe para ganhar o benefício da otimização.

A personagem, em primeiro plano, tem quatro quadros de animação em um mesmo arquivo. Isso favorece a compactação da imagem, neste caso especificamente, já que os quatro quadros utilizam as mesmas cores, que podem ser declaradas apenas uma vez, ao invés das quatro vezes, se fossem imagens individuais. Apesar desta ser uma otimização que depende de ser ter conhecimento sobre o funcionamento interno do codificador de imagens, trazer, na interface do ambiente integrado de desenvolvimento, a informação sobre o tamanho que a versão final do recurso compactado terá na ROM ajuda a guiar o desenvolvedor de jogos sobre a escolha de cores e tamanhos.

Para que o programa execute adequadamente, enquanto o jogo está sendo jogado é necessário analisar, a cada quadro, a matriz que forma o tabuleiro. São 10 linhas por 10 colunas, 100 elementos no total. É preciso, por exemplo, verificar se alguma das partes da peça que está subindo vai colidir com outra peça acima, ou ainda, concluir se a condição de encerramento do jogo aconteceu, inspecionando se há alguma peça ocupando a última linha. O microcontrolador precisa processar estar operações, desenhar toda a cena e ainda fazer a transferência do framebuffer para a tela de maneira constante, 30 vezes por segundo.

Para avaliar como o jogo conceito se comporta neste quesito durante sua execução, foi desenvolvida uma interface especial que, sobreposta ao jogo na tela, indica quando a frequência de atualizações por segundo ficou abaixo de 30 quadros. A Figura 52 apresenta detalhes sobre seu funcionamento.

Figura 52 – Medição de quadros por segundo nos jogos conceituais.



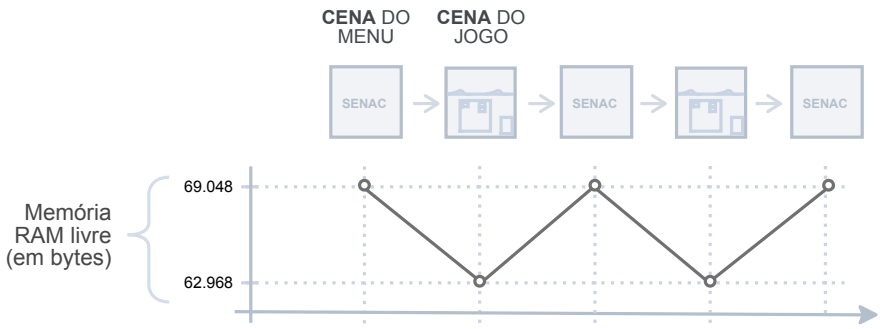
Fonte: autoria própria.

Durantes os testes nos dois vídeo games estudados, apenas durante as trocas de cena foram registrados valores abaixo dos 30 quadros por segundo necessários. Isto era esperado, já que é neste momento de troca de cenas que o microcontrolador executa operações intensivas de memória – liberando os recursos da cena anterior e alocando recursos para nova cena, descompactando imagens, por exemplo. Na opinião dos autores, a queda na taxa de quadros neste momento da execução não é um problema, por ser algo comum nos vídeo games. Popularmente estas ocasiões de transição de cenas nos consoles são conhecidas como telas de carregamento.

De qualquer maneira, a operação de troca de cena nos controladores estudados não chegou a levar um segundo, quando das medições nos testes dos jogos conceituais. Esta é das vantagens de se ter a memória ROM dentro do mesmo encapsulamento da memória RAM. As telas de carregamento em consoles como os, por exemplo, da linha Sony Playstation, que utilizam mídias óticas para armazenar os jogos, são lembradas por levarem minutos. Gregory (2018) explica como o console precisa primeiro deslocar fisicamente um laser montado em um trilho para região correta na mídia ótica durante a leitura dos dados. Diz ainda que isso é um dos três grandes gargalos durante o carregamento de um jogo, juntamente com a descompactação do arquivo e transferência para RAM. Em contraste, a memória Flash ou ROM dos controladores pode ser acessada muito mais rapidamente, randomicamente.

Medimos como se comportava a memória RAM livre durante a troca de cenas. O algoritmo do codificador de imagens deve alocar a memória necessária para os gráficos no início da cena e liberá-la no seu encerramento. Esse comportamento foi observado, como mostra a Figura 53.

Figura 53 – Gráfico da memória RAM livre durante a troca de cenas.



Fonte: autoria própria.

Quanto aos controles, não foi notado atraso na leitura dos comandos. Como ela é

feita de maneira serial tanto no ODROID-GO, quanto no LVNDR-01 – e sendo um dos passos do gameloop do motor de jogos – podemos presumir que o intervalo de tempo entre leituras dos comandos dos controles ficou abaixo dos 33 milissegundos. Do contrário, o tempo para leitura afetaria a métrica de quadros por segundo, comentada anteriormente.

3.6.2 Conceito 2 - Jogo de ação

A ideia neste segundo jogo conceitual é avaliar a capacidade do motor em relação à gestão de recursos, principalmente de memória RAM, ROM e processamento. Escolhemos um estilo de jogo em que: elementos são adicionados e removidos constantemente, estressando rotinas de alocação e liberação de memória; contém inimigos com inteligência artificial baseada em máquinas de estado; a tela se movimenta com frequência e a quantidade de cores é relevante na compactação das imagens utilizadas. A Figura **??** mostra um jogo do gênero, lançado em 1988, para consoles com restrições de hardware similares.

Figura 54 – "Double Dragon", para o console NES, da Nintendo.

Fonte: Technōs, do Japão. Publicado nos Estados Unidos pela Tradewest, em 1988.

O critério de sucesso nesta etapa é: alcançar, durante o todo andamento do jogo, ao menos 20 quadros por segundo, sem variações bruscas na taxa de atualização – visto que isso afeta a experiência do jogador.

Com o objetivo de modificar intensivamente a memória durante a execução, foi planejado um jogo em que, em sua mecânica, muitos projéteis se deslocam pela tela – a interface desenvolvida pode ser vista na Figura 55. Nele, o jogador movimenta seu personagem – representado por uma nave na tela – utilizando os direcionais do controle. Com o botão A pode disparar projéteis, que seguem em linha reta. Quando mais rápido pressionar o botão, mais disparos são feitos – podendo chegar a 15 por segundo. No lado oposto da tela, inimigos são automaticamente posicionados pelo jogo. Cada um deles lança

projéteis em um ritmo próprio. O jogador marca pontos quando um dos seus tiros acerta algum inimigo, removendo-o da tela, e perde quando um dos ataques de inimigos o acerta. O jogo executa indefinidamente, repondo os adversários quando necessário.

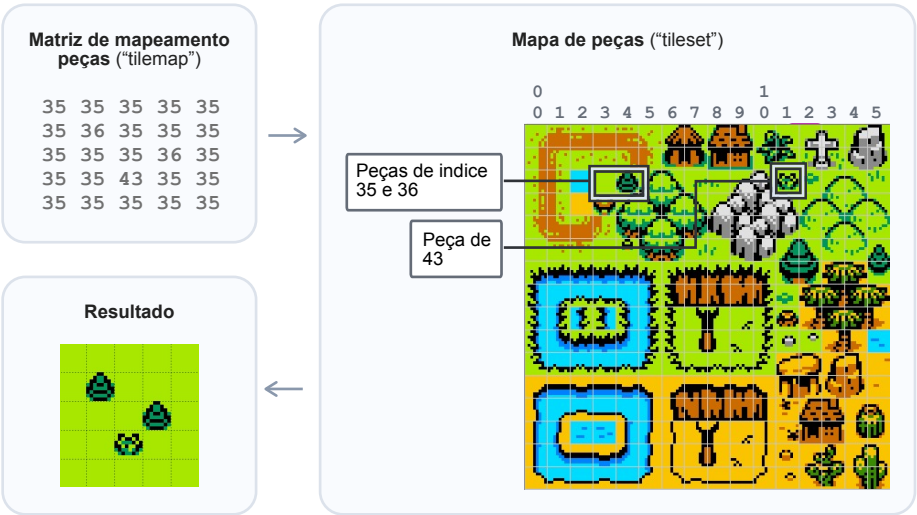
Figura 55 – Interface projetada para prova de conceito 2. Utiliza gráficos de domínio público, sob licença CC-0, criados por "GrafxKid", "ansimuz" e "surt", do portal online "Open Game Art".

Fonte: autoria própria.

No plano de fundo, um cenário desliza constantemente. Desta maneira validamos a capacidade do motor de atualizar totalmente o framebuffer entre quadros – dentro do limite de 33 milissegundos, para alcançar 30 atualizações por segundo. Utilizamos uma técnica de mapeamento comentada por Hugg (2019), a "Offscreen Scrolling". Ao invés de utilizar uma imagem do cenário, pequenas peças, em inglês, "tiles", de 16 por 16 pixels são posicionados lado a lado, segundo instruções de uma matriz de apoio, como mostra a Figura 57. Neste momento foi explorada também a capacidade de acesso rápido a ROM dos microcontroladores. A matriz que descreve o cenário é lida da ROM em tempo de execução, somente na área visível, sem que seja preciso antes transferi-la por completo para memória RAM.

Para implementação dos projéteis, utilizamos uma estrutura de lista ligada e alocação em tempo real de memória. A cada novo disparo, seja do jogador ou de um dos adversários, uma nova estrutura com informações do projétil é inserida ao fim da lista. Nas atualização do jogo, a cada 33 milissegundos, uma análise de todos os projéteis da lista é feita: os que estão posicionados fora da tela são removidos da lista e sua memória é liberada;

Figura 56 – Processo de construção de cenários utilizando peças. Utiliza gráficos de domínio público, sob licença CC-0, criados por “surt”, do portal online “Open Game Art”.



Fonte: autoria própria.

os que colidiram, seja com o jogador ou com um inimigo, são computados – aplicando o dano necessário e consequências da colisão – e depois removidos da lista, também liberando memória; por fim, o restante dos projéteis tem sua posição atualizada – cada um deles guarda seu vetor de aceleração individualmente. Os inimigos foram construídos da mesma maneira, utilizando uma lista ligada e alocação em tempo real de memória.

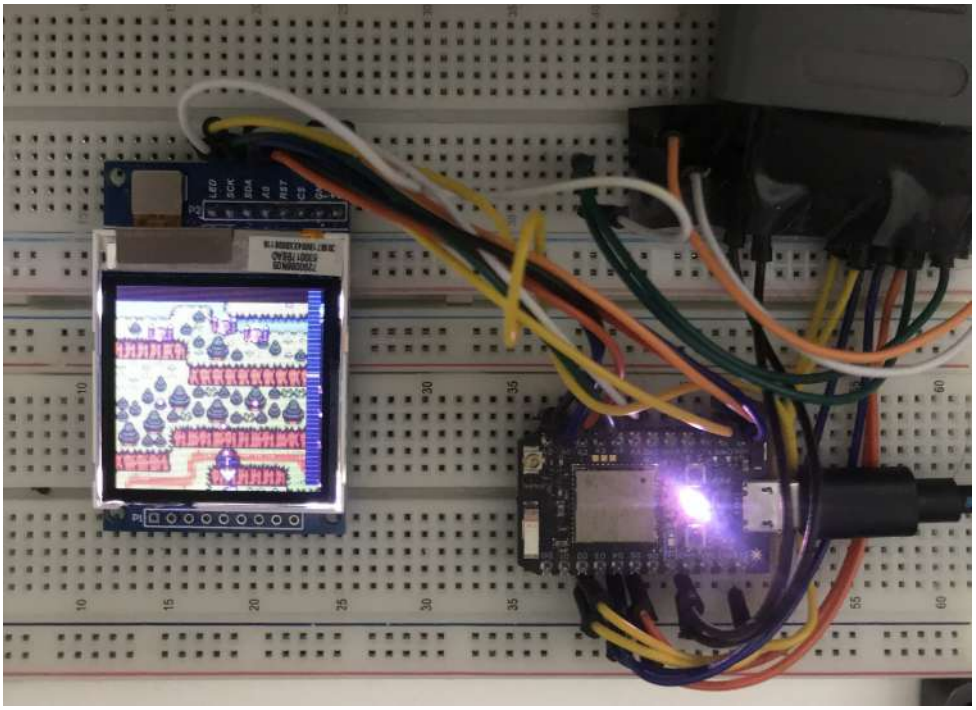
Por fim, foi implementada uma máquina de estados para os adversários. Cada um deles pode estar em um de dois estados: fora de posição, movimentando-se para seu destino; ou posicionado e atacando em uma frequência fixa própria. Quando o jogo percebe que não há inimigos na tela, executa uma rotina de reposição, adicionando mais quatro adversários ao jogo, todos eles inicialmente no estado “fora de posição”. Com isso o jogo passa a impressão de que os novos inimigos voam até sua posição.

Depois de implementado no “lvndr” e compilado dentro do contêiner, utilizando os codificadores e controladores estudados, o jogo foi executado por 8 horas seguidas. A cada segundo, 4 projéteis eram disparados, para, 2 segundos depois, serem removidos por estarem fora da região visível. No total, 115.200 alocações de memória de estruturas de 32 bytes ocorreram[15].

Após este período, foram realizadas medições da memória livre e taxa de atualização. Nos dois aparelhos, não houve falta de memória RAM, nem queda para menos de 30

[15] Utilizamos a função “sizeof” de C para obter o tamanho da estrutura.

Figura 57 – O jogo conceitual 2 sendo executado no protótipo LVNDR-01.



Fonte: autoria própria.

quadros por segundo. Apenas na entrada da cena – assim como ocorreu no primeiro jogo conceitual – foi registrado um intervalo de tempo maior do que 33 milissegundos para processar o quadro, por ser o momento em que o algoritmo do codificador de imagens é executado para descompactar o conteúdo da cena.

4 CONCLUSÃO

Para atingir o objetivo deste trabalho, de, através de um motor de jogos, tornar mais acessível o desenvolvimento de vídeo games baseados em microcontroladores, nos deixamos guiar por uma indagação: como faziam os consoles antigos? Quais algoritmos, ferramentas e estratégias eram utilizadas para se criar jogos tão divertidos, coloridos e animados como Super Mario World, da Nintendo, e Sonic The Hedgehog, da Sega? Dois autores nos ajudaram a pensar sobre esta questão:

Figura 58 – Programa de 1986, da TV japonesa NHK, sobre o desenvolvimento de jogos, mostrando a digitalização de imagens para vídeo games por meio de uma ferramenta, a Sega Digitizer System.

Fonte: NHK, do Japão.

- Hugg (2019) contribuiu, em seu texto, já contemporâneo, com uma visão histórica sobre o que era o Nintendo NES. Detalhou, exibindo exemplos de código-fonte, o que se passava internamente. Explicou como eram as estruturas de dados e quais algoritmos eram utilizados, com qual objetivo. Desta maneira pudemos perceber o quanto convenções eram importantes para que se pudesse criar aqueles jogos. O uso de tabela de cores indexadas é um exemplo disso.

- Já Gregory (2018) trouxe a ideia de motores de jogos – um acervo de programas, utilitários, rotinas otimizadas e especializadas, como visto, que facilitam o dia a dia, liberando os desenvolvedores de jogos das atividades repetitivas. Em seu livro, nos diz muito sobre como ferramentas adequadas fazem a diferença no final. Enquanto

estávamos jogando Super Nintendo no Brasil, nos anos 90, não imaginávamos como, lá no Japão, equipamentos eram criados especificamente para impulsionar o ritmo de produção de video games. A Figura 58 mostra um deles, o Sega Digitizer System, que ajuda na digitalização dos gráficos para os jogos.

Estes dois textos nos fizeram imaginar o que seria uma ferramenta ideal para facilitar a construção de jogos para microcontroladores. Algo que pudesse tirar do caminho as complicações e pormenores de se lidar com um dispositivo tão limitado; que tornasse possível se conectar componentes eletrônicos em uma placa de ensaio, abrir um programa no computador e apenas apertar um botão "play" – similar ao que o Arduino faz muito bem, mas para criação de jogos. Dai em diante, outros dois autores colaboraram para que fosse possível concretizar esta arquitetura imaginada:

- Simmonds (2017), que nos explica sobre como os controladores de dispositivos no funcionam Linux. Dá detalhes sobre a comunicação entre hardware e software, ilustrando os protocolos, como SPI e I2C. De maneira prática, inclui exemplos comentados de código e gráficos compreensíveis.
- Por sua vez, McKendrick (2020), em seu livro sobre Docker, explora em profundidade a temática de contêineres Linux. Apresenta recomendações de configuração, boas práticas de gestão dos recursos e o que precisa ser evitado, até por questões de segurança.

Assim foi construída a solução que alcançou o objetivo deste trabalho, atentando-se para o quão de baixo nível era o desenvolvimento de jogos nos consoles das gerações passadas e aos cuidados que isso exigia; percebendo como um motor de jogos tem o potencial de auxiliar nessa tarefa; entendendo o hardware e seus protocolos de comunicação; e, finalmente, utilizando ferramentas modernas para criar um ambiente de desenvolvimento integrado e extensível, onde tudo isso conversa em harmonia por detrás de um simples botão "play".

Como comentado na subseção 3.1, sobre a arquitetura, o sucesso da ferramenta construída neste trabalho vai ser proporcional a aceitação dela pela comunidade de desenvolvimento. De qualquer maneira, já empolgados com os resultados alcançados na criação dos jogos conceituais, estamos confiantes de que o primeiro passo foi dado.

5 TRABALHOS FUTUROS

O trabalho apresentado aqui, em grande parte, inspirou-se em outros estudos prévios, como os descritos na seção 2.5, dos trabalhos relacionados. Esperamos que este texto tenha despertado o interesse no assunto e a curiosidade de novas gerações de pesquisadores. Todo código-fonte pode ser encontrado na plataforma Github e está sob licença de software livre[1]. Gostaríamos de retribuir o conhecimento que nos foi passado deixando nossas sugestões de linhas de pesquisa na área de motores de jogos para microcontroladores.

5.0.1 Integrações com o ecossistema do Arduino

Tema visto na subseção 2.4.1, o ecossistema construído pelo projeto Arduino está profundamente enraizado no movimento maker, dos hobistas e estudantes que utilizam microcontroladores para projetar todo tipo de gadget. Ele, por si só, traz um legado de bibliotecas, tutorais na Internet e ferramental multiplataforma para facilitar o uso de placas de prototipação baseadas em microcontroladores. Trazer toda bagagem que o Arduino tem para próximo do desenvolvimento de jogos é um grande desafio, que envolve estudar sua arquitetura, comparar diversos tipos de hardware e planejar a troca de informação entre os sistemas.

Criar, por exemplo, um imagem de contêiner Linux para o motor de jogos que foi apresentando, preparada para fazer o intermédio com o ecossistema Arduino, tem o potencial de adicionar suporte simultâneo a uma infinidade de microcontroladores com quem o Arduino já se integra. Novos lançamentos de hardware compatível – como o Portenta H7, que começou a ser comercializado em 2020 – indicam que este ecossistema perdurará.

5.0.2 Codificação de áudio para microcontroladores

Parte essencial da experiência de jogar video games, a implementação da trilha sonora e dos efeitos especiais é um universo à parte, tanto em termos de software, quanto de hardware. Hugg (2019), sobre o assunto, explica que no video game NES, da Nintendo, um circuito dedicado faz esse trabalho. Em suas palavras, "a unidade de processamento de áudio é um gerador de som integrado com a CPU, no chip RP2A03, e possui cinco geradores independentes de som[2]". Em um console de video game baseado em microcontrolador, que

[1] Disponível no endereço eletrônico http://github.com/lv-e. Acessado em 20/11/2020

[2] "The Audio Processing Unit (APU) is a sound generator packaged with the CPU in the RP2A03 chip. It has five independent sound generators, or channels.", com tradução do autor.

não conta com um chip dedicado ao áudio, é preciso implementar por meio de software a geração dos sinais e mixagem. No caso do NES e do GameBoy, dois consoles da Nintendo, são geradores de ondas quadradas, triangulares e até ruídos, que são modelados para gerar diferentes sons. Mais sobre o tema pode ser visto na subseção 17.

O motor de jogos apresentado aqui pode ser um ponto de partida para pesquisas neste sentido. A arquitetura de codificadores e controladores facilita a essa integração e, fazendo assim, toda parte de desenho na tela e entrada por meio de controles já estará implementada.

5.0.3 Adição de consoles baseados em Linux ao ecossistema

Entre os computadores de mesa, laptops, tablets e os microcontroladores existe uma categoria conhecida pelo termo em inglês "single-board computer", que pode ser traduzido como "computadores de uma só placa". São compactos, acessíveis e custam uma fração do preço de um laptop. Modelos contemporâneos são bem mais potentes do que os microcontroladores estudados, chegando a Gigabytes de memória RAM, Giga hertz de velocidade de CPU e com suporte a video HDMI. Raspiberry Pi é um exemplo famoso de linha de dispositivos nesta categoria. O modelo Zero W, com suporte a Internet Wi-Fi e sistema operacional Linux, pode ser encontrado em sites de comércio eletrônico no Brasil por R$ 120,00, com valores de 2020.

Assim como implementamos suporte ao sistema operacional FreeRTOS, adicionar ao ecossistema do motor um Pi W Zero é, basicamente, escrever um código de controlador. Apesar da abundância de recursos de hardware em um dispositivos destes, uma pesquisa nesse sentido é desafiadora por ser preciso estudar profundamente o sistema operacional Linux, suas bibliotecas gráficas, interfaces USB e Bluetooth para controles de videogame. Em teoria, seria possível ainda deslocar todo o ecossistema para o Raspiberry Pi, que suporta contêineres Docker, criando uma dinâmica interessante de trabalho, onde o jogo pode ser editado dentro do próprio video game.

REFERÊNCIAS

24 SATA. Forbes: Albert na čelu 'uradi sam' revolucije. p. 24, Jan 2018. Citado na página 16.

AJMIRE, P. E.; KHANDWANI, F. I. A survey of lossless image compression techniques. *International Journal of Electrical Electronics & Computer Science Engineering*, v. 5, 02 2018. Citado na página 31.

ALLEN, E.; TRIANTAPHILLIDOU, S. *The manual of photography and digital imaging*. Burlington: Focal Press, 2012. Citado na página 30.

AMOS, B. *Hands-on Rtos with Microcontrollers*. [S.l.]: PACKT Publishing Limited, 2020. Citado 2 vezes nas páginas 62 e 63.

ANITHA, S. An overview of image compression. *International Journal of Advanced Research in Computer Science*, v. 5, n. 5, 2014. Citado na página 30.

BAIG, E. C.; LEVITUS, B. *IPhone for dummies*. Hoboken: John Wiley and Sons, Inc., 2019. Citado na página 15.

BARKER, S. *Horizon: Zero Dawn Cost a Reported €45 Million to Make*. 2017. Disponível em: <http://www.pushsquare.com/news/2017/03/horizon_zero_dawn_cost_a_reported_e45_million_to_make>. Citado na página 19.

BARRAGÁN, H. Wiring: Prototyping physical interaction design. *Interaction Design Institute, Ivrea, Italy*, 2004. Citado na página 37.

BOLTJES, E. 'horizon zero dawn': A game design postmortem. In: . Estados Unidos: Game Developers Conference, 2018. Citado na página 19.

CHEN, R. H. *Liquid crystal displays: fundamental physics and technology*. [S.l.]: John Wiley & Sons, 2011. Citado na página 28.

CHUDASAMA, D. V. et al. Survey of image compression method lossless approach. *International Journal of Engineering Research and*, V4, 03 2015. Citado na página 31.

COMPUTER ACTIVE. Makerbuino. play and make your own games. p. 29, Sep 2017. Citado na página 16.

CORBET, J.; RUBINI, A.; KROAH-HARTMAN, G. *Linux device drivers*. [S.l.]: OReilly, 2010. Citado na página 53.

ESPRESSIF SYSTEMS. *ESP32 Datasheet Version 2.2*. [S.l.], 2018. Citado 2 vezes nas páginas 41 e 63.

FAN, L. et al. A tangible jigsaw puzzle prototype for attention-deficit hyperactivity disorder children. In: SPRINGER. *The Euro-China Conference on Intelligent Data Analysis and Applications*. [S.l.], 2018. p. 275–286. Citado 2 vezes nas páginas 48 e 49.

FILIPEFLOP. *Placa de Desenvolvimento Micro:bit*. 2019. Disponível em: <https://www.filipeflop.com/produto/placa-de-desenvolvimento-micro-bit/>. Citado 2 vezes nas páginas 39 e 41.

FREITAS, L. G. d. Gear2d: um motor extensivel de jogos baseado em componentes. 2013. Citado 2 vezes nas páginas 47 e 48.

GDC. State of game industry. In: . San Francisco: Game Developers Conference, 2018. Citado na página 21.

GREEN, D. *Procedural content generation for C++ game development*. Birmingham: Packt Publishing, 2016. Citado na página 33.

GREGORY, J. *Game engine architecture*. Boca Raton: Taylor & Francis, CRC Press, 2018. Citado 17 vezes nas páginas 18, 19, 20, 23, 24, 25, 26, 27, 28, 29, 56, 57, 58, 65, 70, 89 e 95.

GRUBB, J. *The console alternatives: Everything you need to know about Ouya, GameStick, and Shield*. 2013. Disponível em: <https://venturebeat.com/2013/05/30/microconsoles-explained>. Citado na página 16.

GUANA, V.; STROULIA, E.; NGUYEN, V. Building a game engine: A tale of modern model-driven engineering. In: IEEE. *2015 IEEE/ACM 4th International Workshop on Games and Software Engineering*. [S.l.], 2015. p. 15–21. Citado 2 vezes nas páginas 46 e 80.

HOFFMAN, J. *Mastering Arduino*. [S.l.]: Packt Publishing, 2018. Citado 4 vezes nas páginas 36, 37, 38 e 39.

HUGG, S. *Making Games for the NES*. [S.l.]: Puzzling Plans LLC, 2019. Citado 7 vezes nas páginas 29, 58, 67, 77, 91, 95 e 97.

IENNE, P.; LEUPERS, R. *Customizable embedded processors design technologies and applications*. [S.l.]: Morgan Kaufmann, 2006. Citado na página 41.

KACZMAREK, S.; LEES, B.; BENNETT, G. *Swift 4 for Absolute Beginners: Develop Apps for iOS*. Berkeley: Apress, 2018. Citado na página 15.

KELLY, C. *Programming 2D games*. [S.l.]: CRC Press, 2012. Citado na página 20.

KIST, R. *Horizon Zero Dawn: hoe de grootste Nederlandse mediaproductie tot stand kwam*. NRC, 2017. Disponível em: <https://www.nrc.nl/nieuws/2017/02/26/hoe-de-grootste-mediaproductie-van-nederland-is-gemaakt-6962270-a1547873>. Citado na página 19.

LOGUIDICE, B.; BARTON, M. *Vintage games*. [S.l.]: Focal Press, 2012. Citado na página 19.

MAAN, A. J. Analysis and comparison of algorithms for lossless data compression. *International Journal of Information and Computation Technology*, v. 3, n. 3, p. 139–146, 2013. Citado 2 vezes nas páginas 31 e 32.

MADHAV, S. *Game programming algorithms and techniques: a platform-agnostic approach*. [S.l.]: Addison Wesley, 2014. Citado na página 28.

MAHONEY, M. Data compression explained. *mattmahoney. net, updated May*, v. 7, p. 1, 2012. Citado na página 31.

MCKENDRICK, R. *Mastering Docker*. [S.l.]: Packt Publishing, 2020. Citado 3 vezes nas páginas 54, 73 e 96.

MELONI, W. Rev up your engines. *Computer Graphics World*, v. 38, n. 1, Jan 2015. Citado 3 vezes nas páginas 19, 20 e 21.

MOLTENBREY, K. Space training. *Computer Graphics World*, v. 40, n. 4, Aug 2017. Citado na página 20.

NINTENDO. *Iwata Asks*. Kyoto: Nintendo, 2009. Disponível em: <https://iwataasks.nintendo.com/interviews/#/wii/nsmb/0/0>. Citado na página 34.

NINTENDO OF AMERICA INC. *SNES Development Manual*. [S.l.], 1993. Book 1. Citado na página 25.

NINTENDO OF AMERICA INC. *GameBoy Programming Manual*. [S.l.], 1999. DMG-06-4216-001-B. Citado na página 36.

PEREA, F. *Arduino essentials*. [S.l.]: Packt Publishing, 2015. ISBN 9781784398569. Citado na página 16.

POHLMANN, K. C. *Principles of digital audio*. [S.l.]: McGraw-Hill, 2011. Citado na página 35.

POUNDER, L. Single board computers. *Linux Format*, Future Publishing, Reino Unido, p. 45–48, Nov 2016. Citado 2 vezes nas páginas 36 e 39.

POWELL, J. The story of S.S.I. *Antic*, v. 4, n. 3, Jul 1985. Citado na página 22.

PROCEDURAL. *Cambridge Advanced Learners Dictionary*. Cambridge: Cambridge University Press, 2013. Citado na página 33.

RODOT, A. *PongMETA*. 2017. Disponível em: <https://github.com/Rodot>. Citado na página 22.

ROELOFS, G.; KOMAN, R. *PNG: the definitive guide*. Sebastopol: O'Reilly & Associates, Inc., 1999. Citado na página 29.

SANTIAGO, B. *DIY Super Nintendo Breadboard Controller*. Digi-Key, 2009. Disponível em: <https://www.digikey.com/en/maker/projects/diy-super-nintendo-breadboard-controller/788fd6f521824fb58715ca0b95b92840>. Citado na página 67.

SANTOS, T. A. d. Biblioteca para desenvolvimento de jogos para nintendo ds. 2009. Citado na página 46.

SCHAATHUN, H. G. *Machine learning in image steganalysis*. [S.l.]: Wiley, 2012. Citado na página 78.

SHEKAR, S.; HANEY, S. *Swift Game Development*. [S.l.]: Packt Publishing, 2018. Citado na página 26.

SIMMONDS, C. *Mastering embedded Linux programming: unleash the full potential of embedded Linux*. [S.l.]: Packt, 2017. Citado 3 vezes nas páginas 61, 62 e 96.

STACKOVERFLOW. *2019 Developer Survey*. 2019. Disponível em: <https://insights.stackoverflow.com/survey/2019>. Citado na página 84.

STACKOVERFLOW. *2020 Developer Survey*. 2020. Disponível em: <https://insights.stackoverflow.com/survey/2020>. Citado na página 71.

SUAREZ, F. G.; LIMOR, F. *How Computers Work: Binary & Data*. Code.org, 2018. Disponível em: <https://www.youtube.com/watch?v=USCBCmwMCDA>. Citado na página 35.

WEILLER, T. A. *Game Start: Lições de Game Design para seu videogame*. [S.l.: s.n.], 2015. Citado na página 19.

WEINBERG, M. *Tetris Copyright Decision Shows How Complicated Copyright for Games Can Be*. 2012. Disponível em: <https://www.publicknowledge.org/blog/tetris-copyright-decision-shows-how-complicated-copyright-for-games-can-be/>. Citado na página 87.

YIU, J. *Definitive Guide to ARM Cortex-M0 and Cortex-M0 Processors*. 2. ed. [S.l.]: Newnes, 2015. Citado 3 vezes nas páginas 36, 40 e 61.

ZARRAD, A. Game engine solutions. In: *Simulation and Gaming*. [S.l.]: InTech, 2018. Citado 3 vezes nas páginas 19, 23 e 24.